

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Факультет інформатики та обчислювальної техніки
(повна назва інституту/факультету)

Кафедра інформатики та програмної інженерії
(повна назва кафедри)

«До захисту допущено»

Завідувач кафедри

_____ Едуард Жаріков
(підпис) (ім'я прізвище)

“ ” _____ 2024 р.

Дипломний проєкт

на здобуття ступеня бакалавра
за освітньо-професійною програмою «Інженерія програмного забезпечення
інформаційних систем»
спеціальності «121 Інженерія програмного забезпечення»

на тему: Розподілена система агрегації, управління та поширення матеріалів

Виконав студент IV курсу, групи _____ ПІ-02
(шифр групи)

Балаценко Олександр Вячеславович _____
(прізвище, ім'я, по батькові) (підпис)

Керівник асистент, Храмченко М. С. _____
(посада, науковий ступінь, вчене звання, прізвище та ініціали) (підпис)

Рецензент Доцент каф.ІСТ, доц., к.т.н Сперкач М.О. _____
(посада, науковий ступінь, вчене звання, прізвище та ініціали) (підпис)

Засвідчую, що у цій дипломній роботі
немає запозичень з праць інших
авторів без відповідних посилань.

Студент _____
(підпис)

Київ –2024

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії
Рівень вищої освіти – перший (бакалаврський)
Спеціальність – 121 Інженерія програмного забезпечення
Освітньо-професійна програма – Інженерія програмного забезпечення
інформаційних систем

ЗАТВЕРДЖУЮ

Завідувач кафедри

(підпис) Едуард ЖАРІКОВ
(ім'я прізвище)

“ ____ ” _____ 2024 р.

ЗАВДАННЯ на дипломний проєкт студенту

Балаценко Олександр Вячеславович
(прізвище, ім'я, по батькові)

1. Тема проєкту Розподілена система агрегації, управління та поширення матеріалів

керівник проєкту Храмченко Микола Сергійович, асистент
(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджені наказом по університету від «27» травня 2024 р. № 2112-с

2. Термін подання студентом проєкту «17» червня 2024 року

3. Вихідні дані до проєкту: технічне завдання

4. Зміст пояснювальної записки

1) Загальні положення: основні визначення та терміни, опис предметного середовища, огляд ринку програмних продуктів, постановка задачі.

2) Інформаційне забезпечення: вхідні дані, вихідні дані, опис структури бази даних.

3) Програмне та технічне забезпечення: засоби розробки, вимоги до технічного забезпечення, архітектура програмного забезпечення.

4) Технологічний розділ: керівництво користувача, методика випробувань програмного продукту.

5. Перелік графічного матеріалу

1) Схема структурна варіантів використань

2) Схема бази даних

3) Схема структурна класів програмного забезпечення

6. Консультанти розділів проєкту

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання «11» березня 2024 року _____

Календарний план

№ з/п	Назва етапів виконання дипломного проєкту	Термін виконання етапів проєкту	Примітка
1	Вивчення рекомендованої літератури	11.03.2024	
2	Аналіз існуючих методів розв'язання задачі	21.03.2024	
3	Постановка та формалізація задачі	30.03.2024	
4	Розробка інформаційного забезпечення	05.04.2024	
5	Алгоритмізація задачі	15.04.2024	
6	Обґрунтування вибору використаних технічних засобів	20.04.2024	
7	Розробка програмного забезпечення	25.04.2024	
8	Налагодження програми	20.05.2024	
9	Виконання графічних документів	25.05.2024	
10	Оформлення пояснювальної записки	26.05.2024	
11	Подання ДП на попередній захист	02.06.2024	
12	Подання ДП рецензенту	10.06.2024	
13	Подання ДП на основний захист	14.06.2024	

Студент

(підпис)

Олександр БАЛАЦЕНКО

(ініціали, прізвище)

Керівник

(підпис)

Микола ХРАМЧЕНКО

(ініціали, прізвище)

АНОТАЦІЯ

Пояснювальна записка дипломного проєкту складається з п'яти розділів, містить 71 таблицю, 22 рисунки та 13 джерел – загалом 73 сторінки.

Дипломний проєкт присвячений створенню системи для поширення та управління навчальних матеріалів

Мета створення розподіленої системи з можливістю автоматизованого віддаленого керування навчанням клієнтів та агрегації даних про оцінювання отриманих знань.

Об'єкт дослідження: система з можливістю автоматизованого віддаленого керування навчанням клієнтів

Предмет дослідження: системи навчання клієнтів з можливістю збору статистики

У першому розділі виконано аналіз предметної області, відомих технічних рішень, сформульовано функціональні та нефункціональні вимоги до розробленого програмного забезпечення.

Другий розділ присвячений моделюванню та проектуванню структури програмного продукту.

У третьому розділі описано планування та виконання тестування розробленого програмного забезпечення.

У четвертому розділі описано керівництво користувача.

КЛЮЧОВІ СЛОВА: WEB ДОДАТОК, REST API ,DATABASE, ,SQL, POSTGRESQL, БАЗА ДАНИХ, СТАТИСТИКА, АДМІНІСТРУВАННЯ, НАВЧАННЯ.

ABSTRACT

The explanatory note of the diploma project consists of five sections, contains 71 tables, 22 figures and 13 sources - a total of 73 pages.

The diploma project is devoted to the creation of a system for distribution and management of educational materials

The purpose of creating a distributed system with the possibility of automated remote management of customer training and aggregation of data on the evaluation of the acquired knowledge.

Research object: a system with the possibility of automated remote management of customer training

Research subject: customer training systems with the possibility of collecting statistics

In the first section, an analysis of the subject area, known technical solutions, and functional and non-functional requirements for the developed software are formulated.

The second section is devoted to modeling and designing the structure of the software product.

The third chapter describes the planning and execution of testing developed software.

The fourth chapter describes the user guide.

KEYWORDS: WEB APPLICATION, REST API, DATABASE, SQL, POSTGRESQL, DATABASE, STATISTICS, ADMINISTRATION, LEARNING.

№ з/п	Формат	Позначення	Найменування	Кількість листів	Примітка
1	A4		Завдання на дипломний проєкт	2	
2	A4	КПІ.ІП-8201.045440.01.91	Технічне завдання	19	
3	A4	КПІ.ІП-8201.045440.02.81	Пояснювальна записка	73	
4	A4	КПІ.ІП-8201.045440.03.12	Текст програми	58	
5	A4	КПІ.ІП-8201.045440.04.51	Програма та методика тестування	6	
6	A4	КПІ.ІП-8201.045440.05.34	Керівництво користувача	10	
7	A3	КПІ.ІП-8201.045440.06.99	Графічний матеріал	3	

				КПІ.ІП-8201.045440.00.90									
Змін.	Арк.	№ докум.	Підп.	Дата	Відомість дипломного проєкту					Літ.	Аркуш	Аркушів	
Розроб.	Балащенко О.В											1	
Перевір.	Храмченко М.С												
Н.контр.	Шулькевич Т.В												
Затв.	Жаріков Е.В												
					КПІ ім. Ігоря Сікорського ФІОТ каф. ІПІ гр. ІП-02								

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ” _____ 2024 р.

**РОЗПОДІЛЕНА СИСТЕМА АГРЕГАЦІЇ, УПРАВЛІННЯ ТА
ПОШИРЕННЯ НАВЧАЛЬНИХ МАТЕРІАЛІВ**

Технічне завдання

КПІ.ПІ-8201.045440.01.91

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Микола ХРАМЧЕНКО

Нормоконтроль:

_____ Тетяна ШУЛЬКЕВИЧ

Виконавець:

_____ Олександр БАЛАЦЕНКО

Київ – 2024

ЗМІСТ

1	НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ	2
2	ПІДСТАВА ДЛЯ РОЗРОБКИ	3
3	ПРИЗНАЧЕННЯ РОЗРОБКИ	4
4	ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	5
4.1	Вимоги до функціональних характеристик	5
4.1.1	Користувацького інтерфейсу	5
4.1.2	Для користувача:	13
4.1.3	Для адміністратора організації:	14
4.1.4	Додаткові вимоги:	14
4.2	Вимоги до надійності	14
4.3	Умови експлуатації.....	14
4.3.1	Вид обслуговування	14
4.3.2	Обслуговуючий персонал.....	14
4.4	Вимоги до складу і параметрів технічних засобів	14
4.5	Вимоги до інформаційної та програмної сумісності	15
4.5.1	Вимоги до мови розробки.....	15
4.5.2	Вимоги до представленню вихідних кодів	15
4.6	Вимоги до маркування та пакування	15
4.7	Вимоги до транспортування та зберігання	15
5	ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ.....	16
5.1	Попередній склад програмної документації	16
5.2	Спеціальні вимоги до програмної документації.....	16
6	СТАДІЇ І ЕТАПИ РОЗРОБКИ	17
7	ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ.....	18

1 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ

Назва розробки: РОЗПОДІЛЕНА СИСТЕМА АГРЕГАЦІЇ,
УПРАВЛІННЯ ТА ПОШИРЕННЯ НАВЧАЛЬНИХ МАТЕРІАЛІВ

Галузь застосування:

Наведене технічне завдання поширюється на розробку програмного забезпечення поширення навчальних матеріалів, котре використовується для оптимізації процесів навчання та призначена для користувачів або співробітників компанії яким необхідно провести навчання

2 ПІДСТАВА ДЛЯ РОЗРОБКИ

Підставою для розробки розподіленої системи агрегації, управління та поширення навчальних матеріалів є завдання на дипломне проєктування, затверджене кафедрою інформатики та програмної інженерії Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського».

3 ПРИЗНАЧЕННЯ РОЗРОБКИ

Розробка призначена для полегшення навчання клієнтів або співробітників організації.

Метою розробки є створення розподіленої системи з можливістю автоматизованого віддаленого керування навчанням клієнтів та агрегації даних про оцінювання отриманих знань.

4 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Вимоги до функціональних характеристик

Програмне забезпечення повинно забезпечувати виконання наступних основних функцій:

4.1.1 Користувацького інтерфейсу

- Мати інтерфейс користувача
- Мати інтерфейс для адміністратора організації

Сторінка аутентифікації показана на рисунку 4.1. Користувач відкривши адресу веб-сторінки у браузері має побачити цю сторінку у випадку якщо він вже не авторизований. Після натискання на кнопку Sign In має перейти на сторінку My Courses, яка показана на рисунку 4.2, якщо дані для входу у систему були введені некоректно, має відображатись відповідне повідомлення.

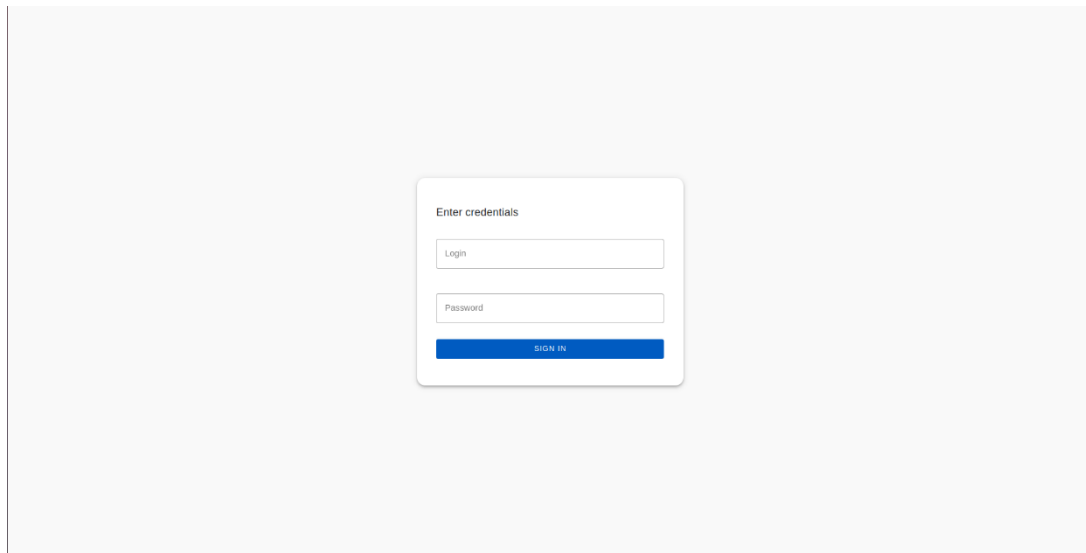


Рисунок 4.1 – Сторінка входу в систему

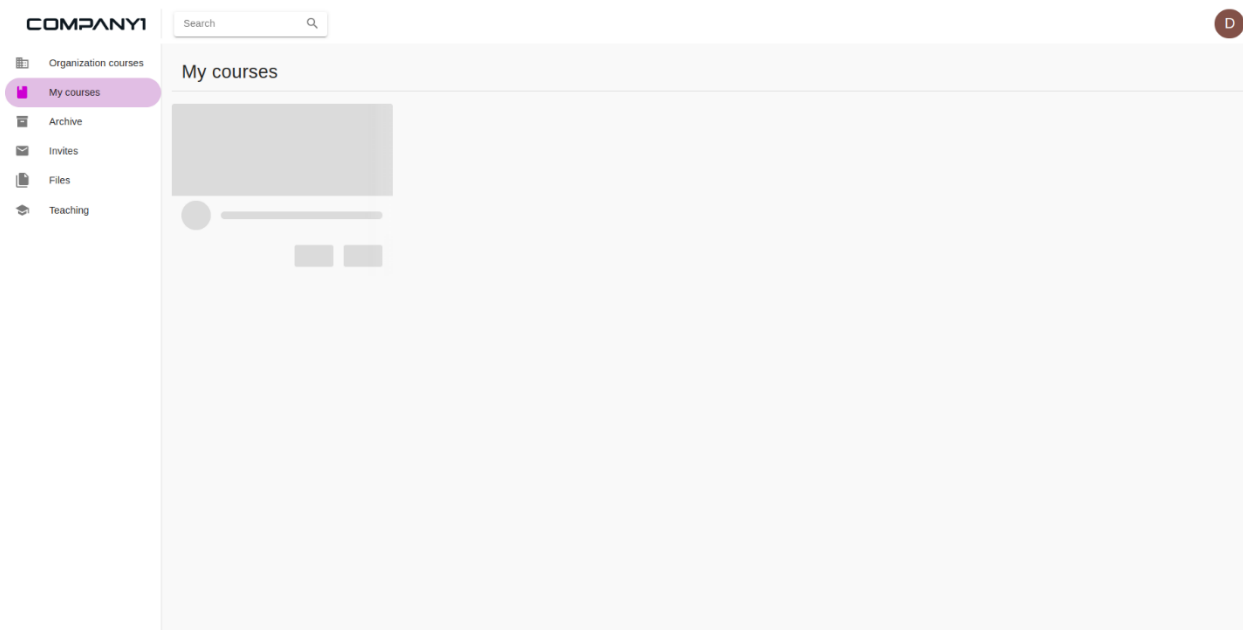


Рисунок 4.2 – Сторінка мої курси

Після успішної авторизації користувач потрапляє на цю сторінку. На цій сторінці відображаються доступні для користувача курси. З цієї сторінки користувач може або перейти в один із пунктів меню, або відкрити курс натиснувши на кнопку Open на карті, яка показана на рисунку 4.3.

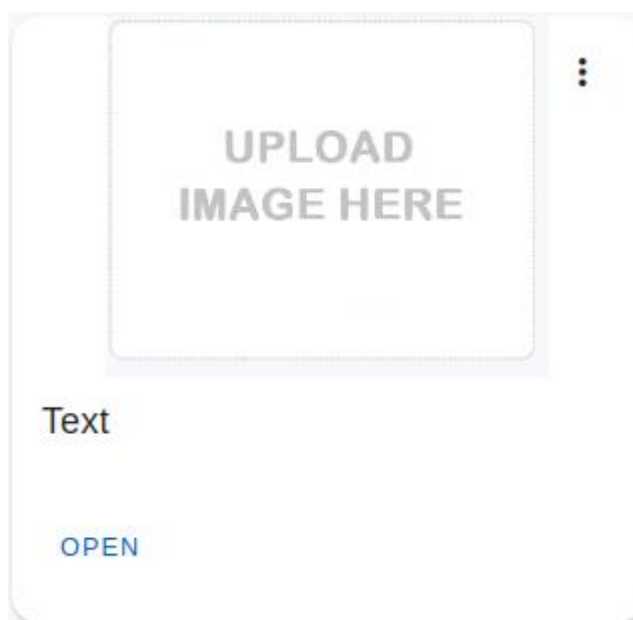


Рисунок 4.3 – Макет карти

Натиснувши на кнопку Open, користувач потрапляє на сторінку опису курсу, яка показана рисунку 4.4.

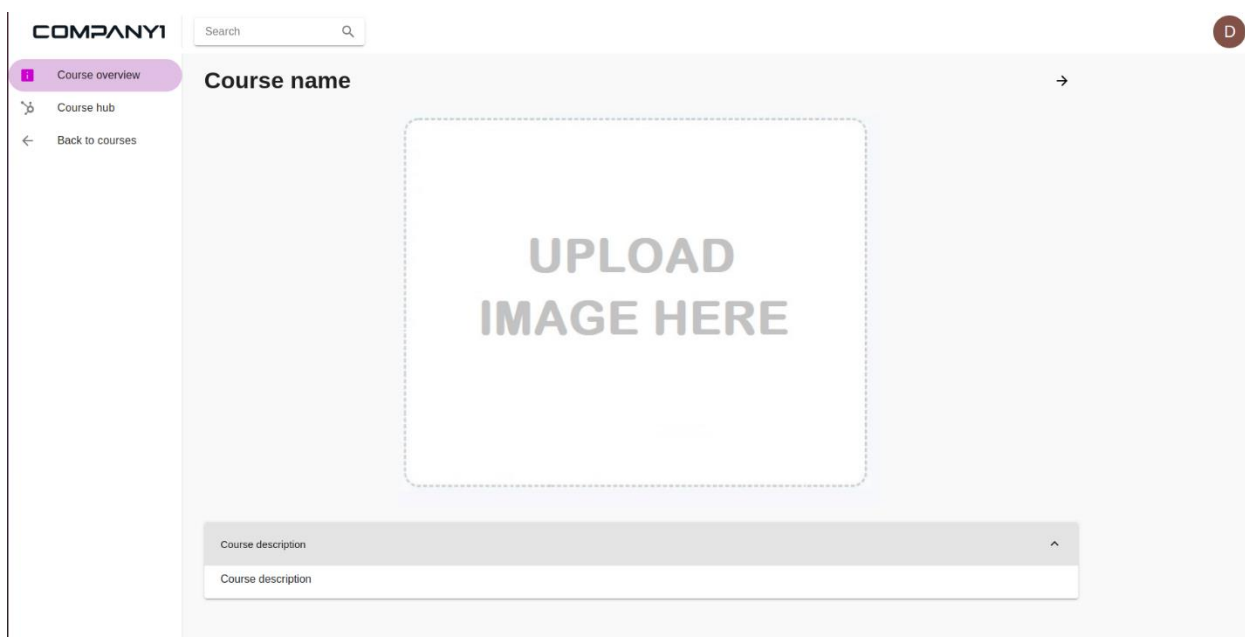


Рисунок 4.4 – Сторінка опису курсу

З цієї сторінки користувач може перейти до сторінки з програмою курсів натиснувши на стрілку в правому верхньому куту, або натиснути на пункт Course Hub в меню. Натиснувши на один з цих варіантів користувач потрапить на сторінку з програмою курсу, яка показана на рисунку 4.5

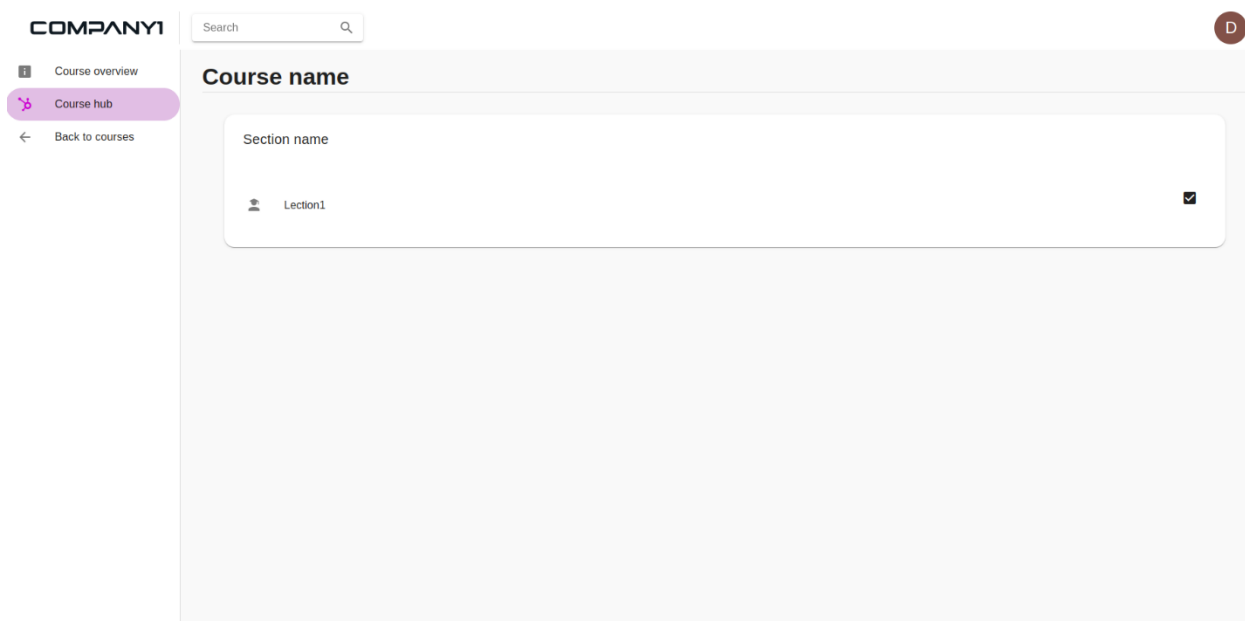


Рисунок 4.5 – Сторінка змісту курсу

З цієї сторінки користувач може завантажити файл, подивитись лекцію, або пройти тестування процес проходження якого показано на рисунках 4.6-4.9

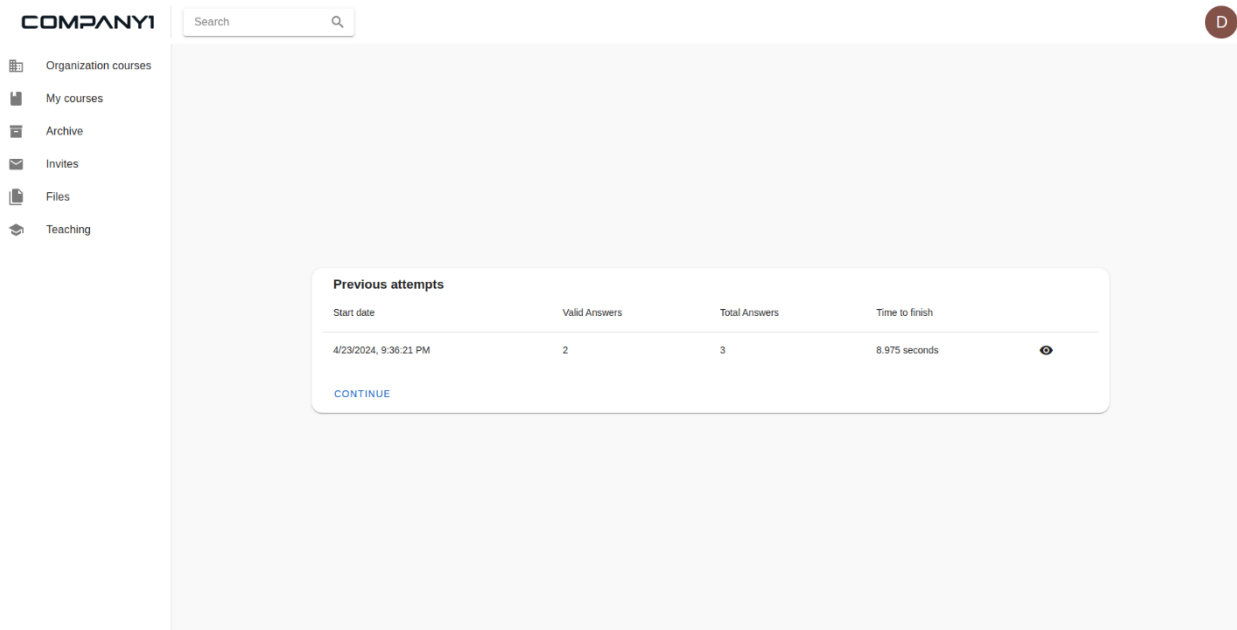


Рисунок 4.6 – Сторінка з минулими спробами

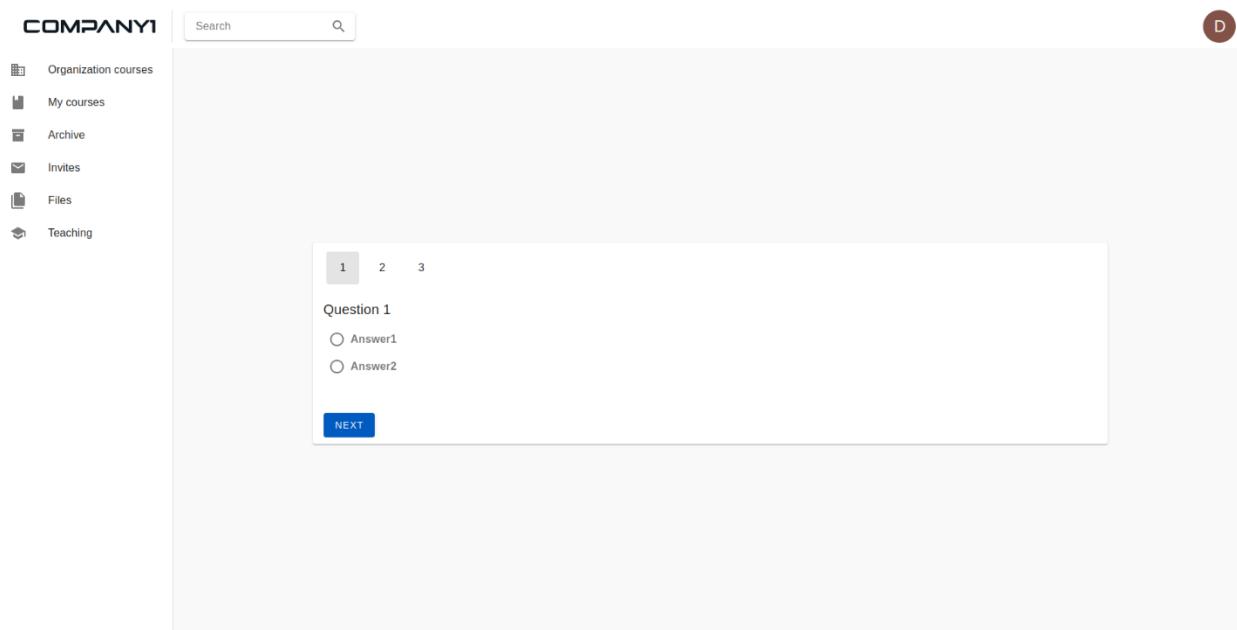


Рисунок 4.7 – Початок тесту

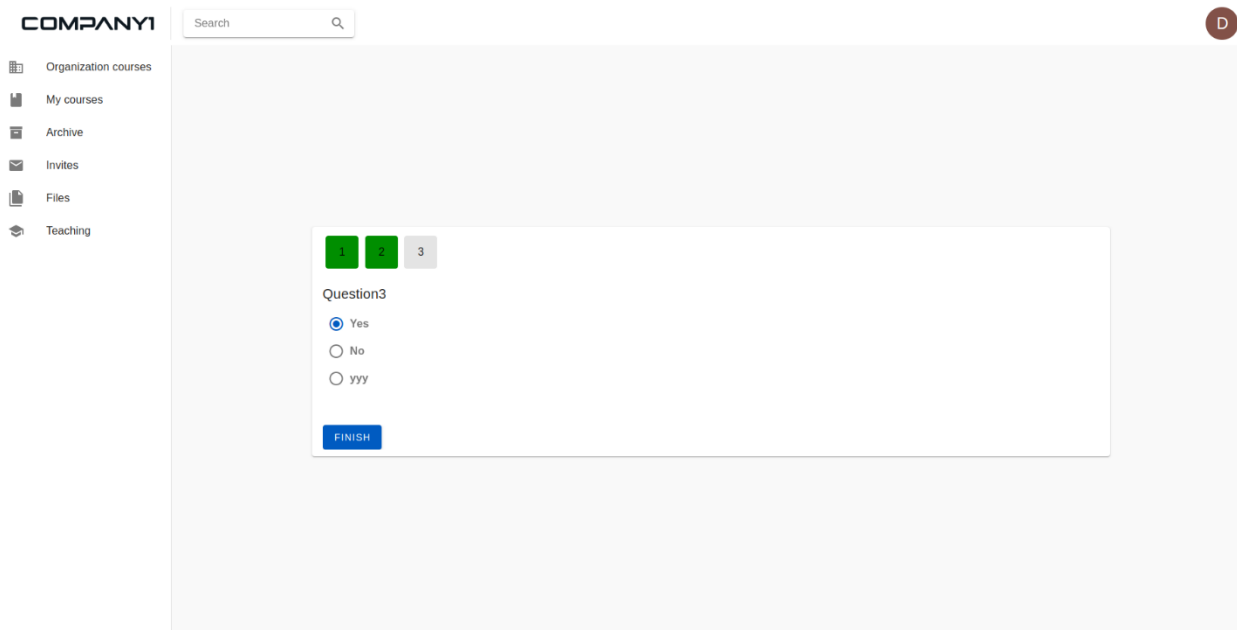


Рисунок 4.8 – Останнє питання

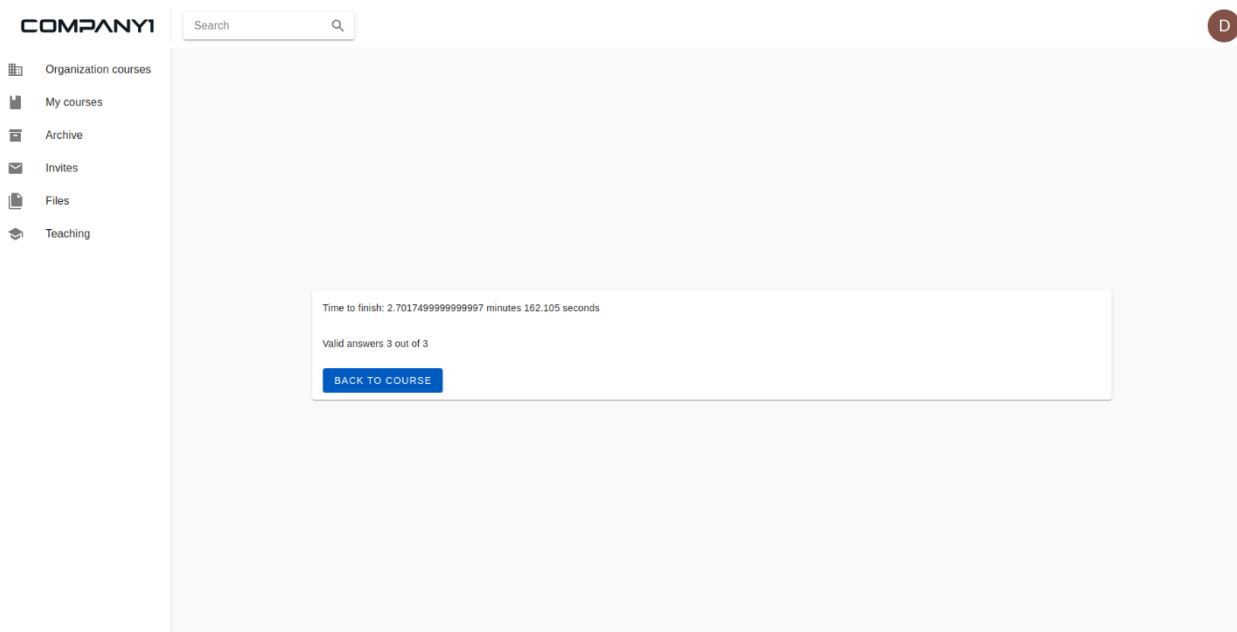


Рисунок 4.9 – Завершення тесту

Сторінка Files, з цієї сторінки користувач може завантажити на свій комп'ютер файли, до яких він має доступ, цей процес показано на рисунку 4.10. Для завантаження файлу достатньо один раз натиснути лівою кнопкою миші.

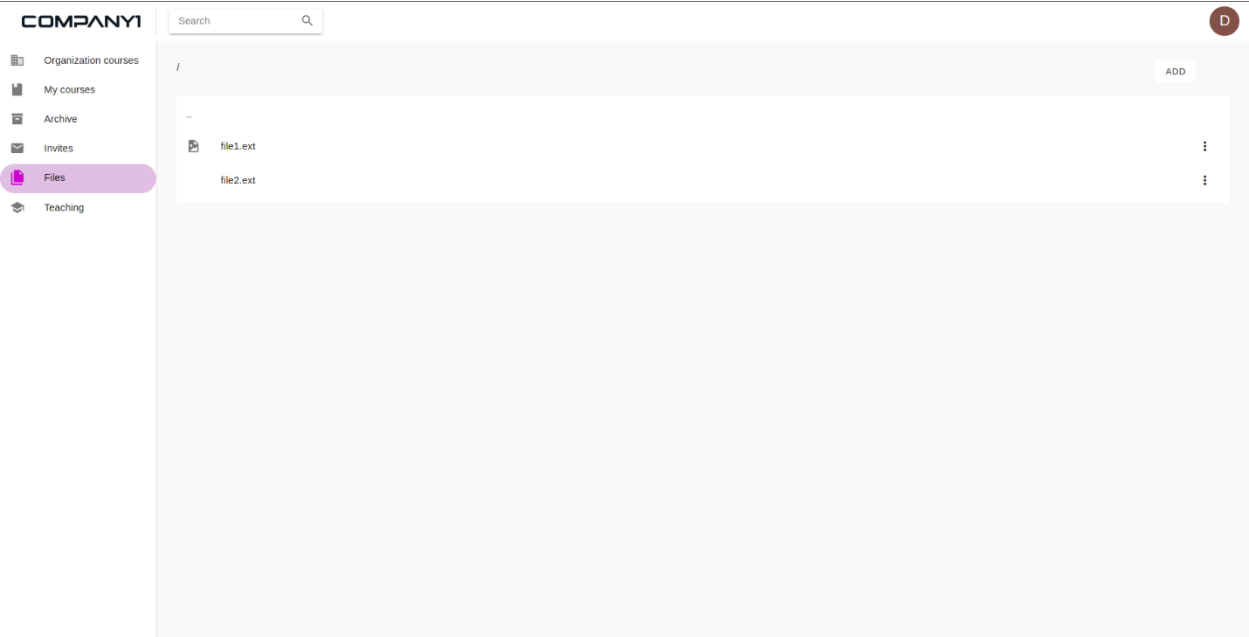


Рисунок 4.10 – Файловий менеджер

Для користувачів з правами локального адміністратора, також є додатковий пункт меню Organization Courses, при натисканні на цей пункт буде відкрита сторінка, яка показана на рисунку 4.11.

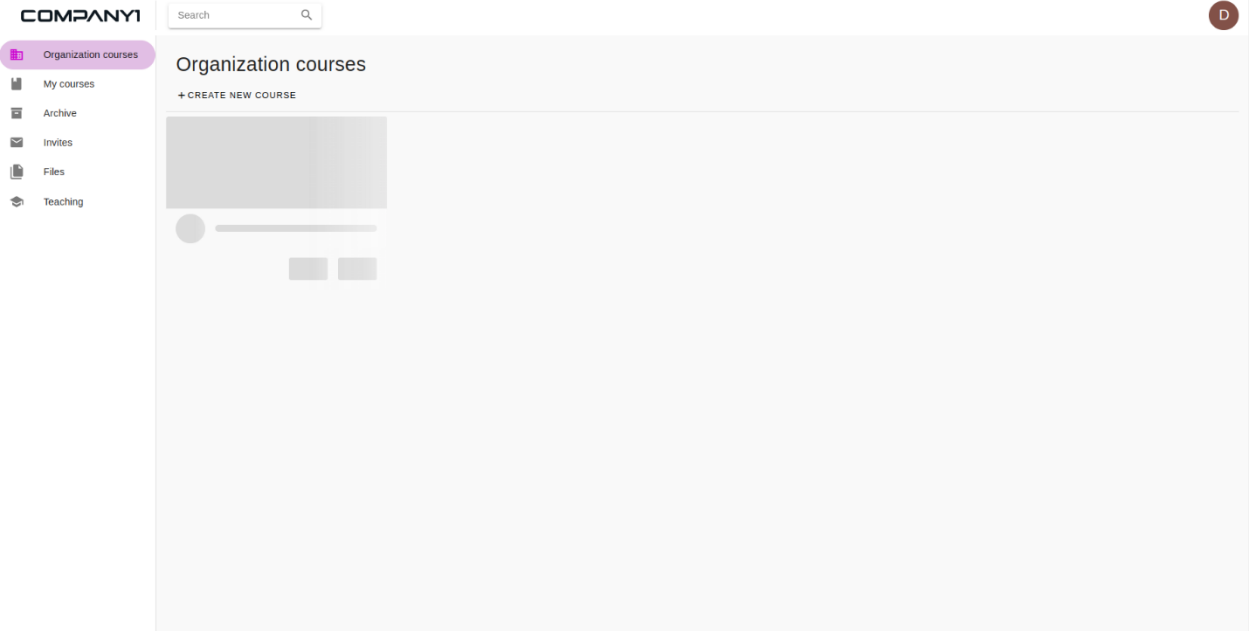


Рисунок 4.11 – Сторінка адміністрування курсів

На цій сторінці відображаються курси, які належать до організації користувача. Курси відображаються таким самим чином, як і в меню My Courses за допомогою карт з рисунку 4.3. Натиснувши на кнопку Open

користувач потрапить на сторінку редагування курсу, яка показана на рисунку 4.12

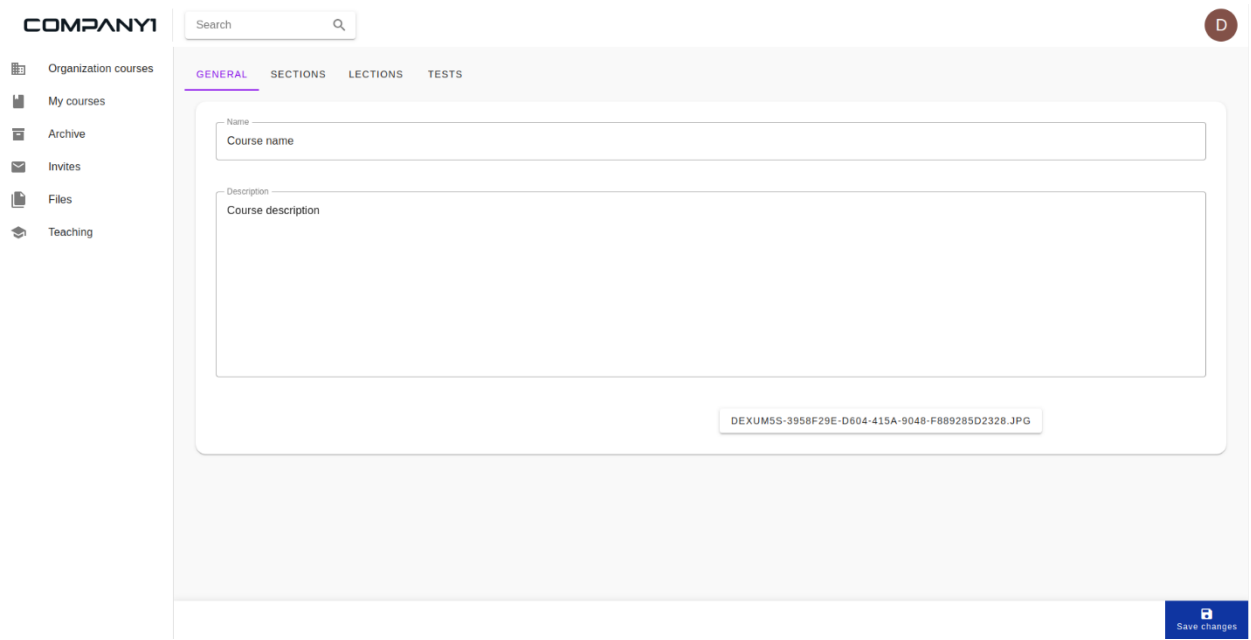


Рисунок 4.12 – Сторінка створення курсу

На цій сторінці користувач може вибрати фотографію, яка буде відображатися для курсу, редагувати його назву та опис. Також користувач може перейти в одну з вкладок Sections, Lections та Tests, на яких можна створити відповідно, програму курсу, лекції та тести. Створення програми курсу показано на рисунку 4.13.

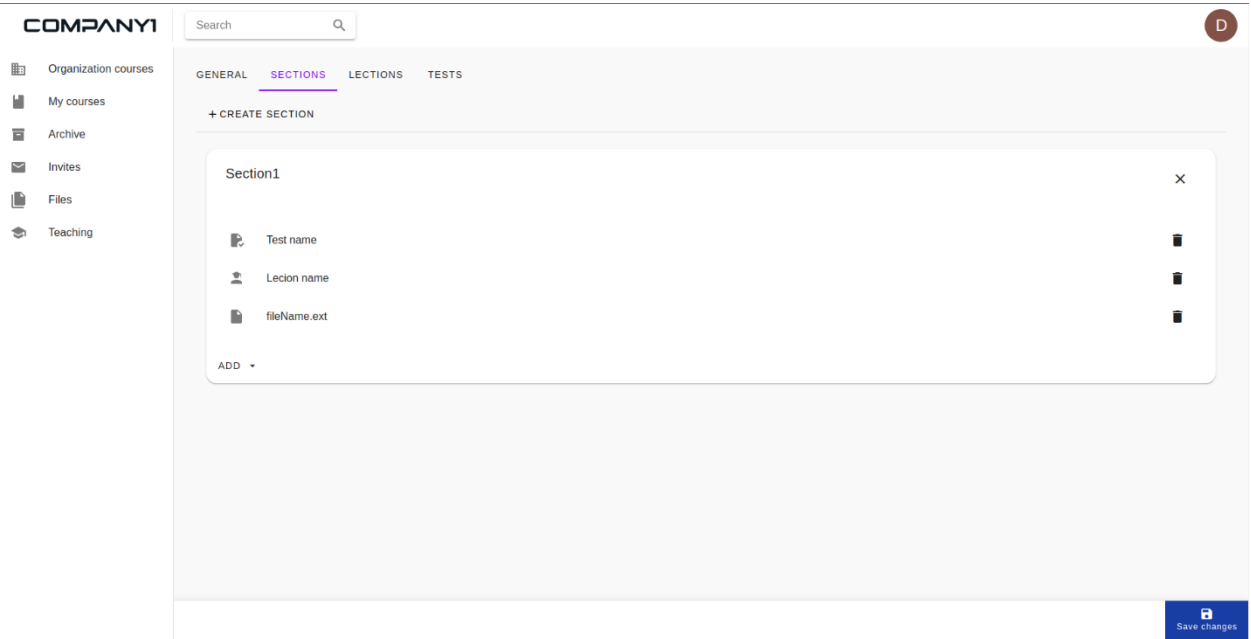


Рисунок 4.13 – Створення програми курсу

Створення тесту показано на рисунках 4.14 та 4.15

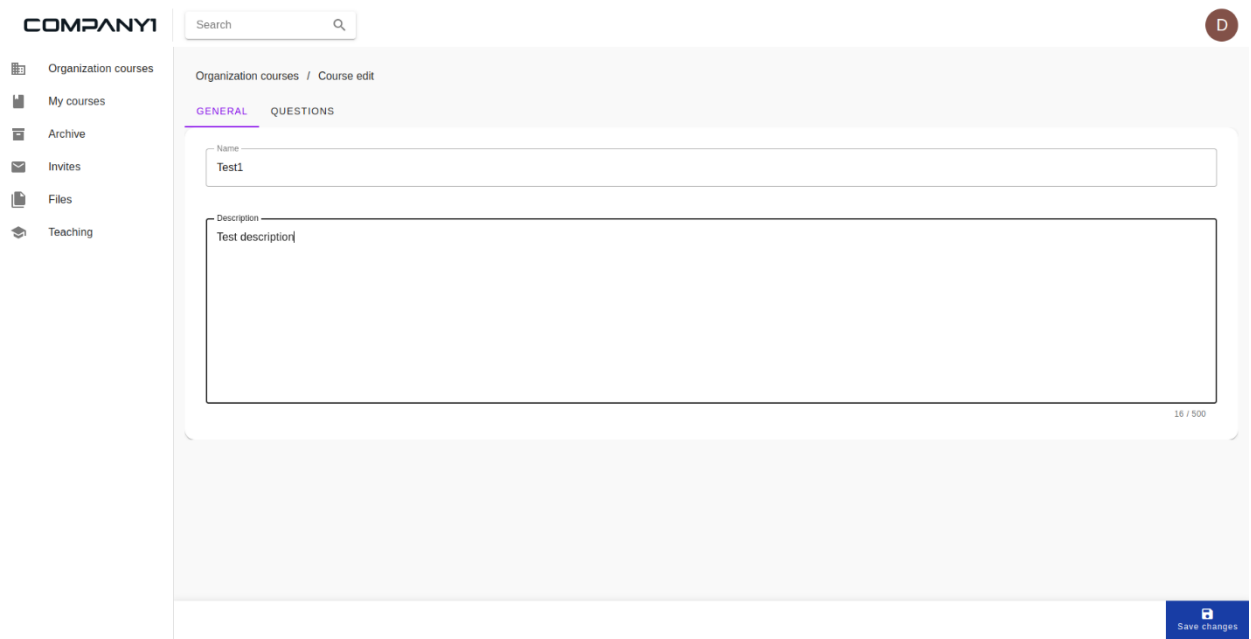


Рисунок 4.14 – Опис та назва тесту

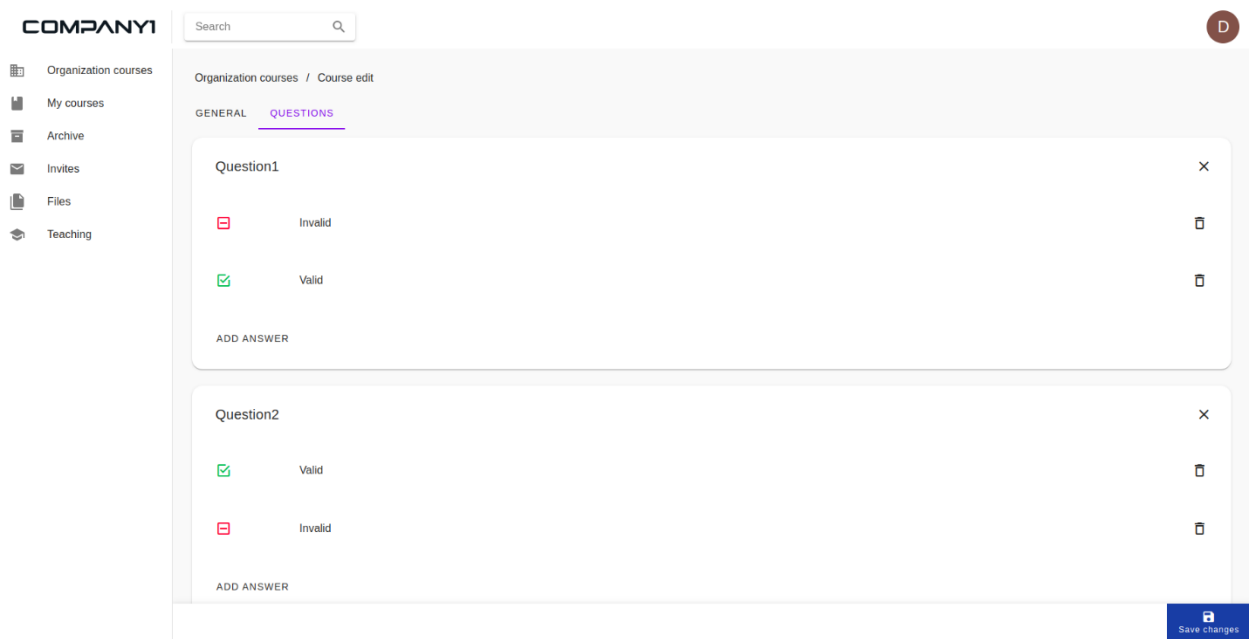


Рисунок 4.15 – Додавання питань

Для користувачів з правами адміністратора системи доступна панель адміністратора, для редагування користувачів та організацій. Сторінка зі списком користувачів показана на рисунку 4.16. З цього списку адміністратор може, створити нового користувача, редагувати існуючого, або видалити користувача з системи. Процес редагування користувача показано на рисунку 4.17

Name	Actions
SimpleUser	
Tester1	
Tester2	

Items per page: 10 1-3 of 3 |< < > >|

Рисунок 4.16 – Реєстр користувачів

Name: SimpleUser Phone: +380688888888

Email: simpleUser@test.test

ROLES

Invites

Name ↑

No data available

Items per page: 10 0-0 of 0 |< < > >|

Рисунок 4.17 – Створення користувача

4.1.2 Для користувача:

- Відкрити курс;
- Пройти курс.
- Пройти тестування
- Редагувати профіль

4.1.3 Для адміністратора організації:

- Можливість створення користувачів
- Можливість редагування користувачів
- Можливість подивитись статистику по курсам

4.1.4 Додаткові вимоги:

- вимога 1;
- вимога 2.

4.2 Вимоги до надійності

Передбачити контроль введення інформації та захист від некоректних дій користувача. Забезпечити цілісність інформації в базі даних

4.3 Умови експлуатації

Умови експлуатації згідно СанПін 2.2.2.542 – 96.

Не висуваються

4.3.1 Вид обслуговування

Вимоги до виду обслуговування не висуваються

4.3.2 Обслуговуючий персонал

Вимоги до обслуговуючого персоналу не висуваються

4.4 Вимоги до складу і параметрів технічних засобів

Програмне забезпечення повинно функціонувати на всіх сучасних браузерах на персональних комп'ютерах.

Мінімальна конфігурація технічних засобів:

- тип процесору: Intel Core i3;
- об'єм ОЗП: 4 Гб;

- підключення до мережі Інтернет зі швидкістю від 20 мегабіт;
- Пристрої введення

Рекомендована конфігурація технічних засобів:

- тип процесору: Intel Core i7;
- об'єм ОЗП: 16 Гб;
- підключення до мережі Інтернет зі швидкістю від 1000 мегабіт;

4.5 Вимоги до інформаційної та програмної сумісності

Програмне забезпечення повинно працювати під управлінням операційних систем сімейства Windows, Linux, Unix на всіх сучасних браузерях

4.5.1 Вимоги до мови розробки

Розробку виконати на мовах програмування C# та Javascript

4.5.2 Вимоги до представленню вихідних кодів

Вихідний код програми має бути представлений у вигляді тексту

4.6 Вимоги до маркування та пакування

Вимоги до маркування та пакування не висуваються.

4.7 Вимоги до транспортування та зберігання

Вимоги до транспортування та зберігання не висуваються.

5 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ

5.1 Попередній склад програмної документації

У склад супроводжувальної документації повинні входити наступні документи на аркушах формату А4:

- пояснювальна записка;
- технічне завдання;
- текст програми;
- програма та методика тестування;

Графічна частина повинна бути виконана на аркушах формату А3 та містити наступні документи:

- схема програмного забезпечення;
- схема бази даних;

5.2 Спеціальні вимоги до програмної документації

Програмні модулі, котрі розробляються, повинні бути задокументовані, тобто тексти програм повинні містити всі необхідні коментарі.

6 СТАДІЇ І ЕТАПИ РОЗРОБКИ

№	Назва етапу	Строк	Звітність
1.	Вивчення літератури за тематикою проекту	21.02	
2.	Розробка технічного завдання	03.03	Технічне завдання
3.	Аналіз вимог та уточнення специфікацій	19.03	Специфікації програмного забезпечення
4.	Проектування структури програмного забезпечення, проектування компонентів	30.03	Схема структурна програмного забезпечення та специфікація компонентів (діаграма класів, схема алгоритму)
5.	Програмна реалізація програмного забезпечення	05.04	Тексти програмного забезпечення
6.	Тестування програмного забезпечення	10.04	Тести, результати тестування
7.	Розробка матеріалів текстової частини проекту	14.04	Пояснювальна записка
8.	Розробка матеріалів графічної частини проекту	20.04	Графічний матеріал проекту
9.	Оформлення технічної документації проекту	29.04	Технічна документація

7 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ

Тестування розробленого програмного продукту виконується відповідно до “Програми та методики тестування”.

Пояснювальна записка до дипломного проєкту

на тему: Розподілена система агрегації, управління та поширення навчальних
матеріалів

КПІ.ПІ-8201.045440.02.81

ЗМІСТ

ВСТУП	2
1 ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ	4
1.1 Аналіз предметної області	4
1.2 Аналіз існуючих рішень	4
1.2.1 Аналіз відомих програмних продуктів	5
1.2.2 Аналіз відомих алгоритмічних та технічних рішень	6
1.3 Опис бізнес-процесів	9
1.4 Постановка задачі	10
Висновки до розділу	11
2 РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	12
2.1 Варіанти використання програмного забезпечення	12
2.2 Аналіз системних вимог	18
2.3 Розроблення функціональних вимог	19
2.4 Розроблення нефункціональних вимог	21
Висновки до розділу	22
3 КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	24
3.1 Архітектура програмного забезпечення	24
3.2 Обґрунтування вибору засобів розробки	25
3.3 Конструювання програмного забезпечення	27
3.4 Аналіз безпеки даних	57
Висновки до розділу	59
4 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	61
4.1 Аналіз якості ПЗ	61
4.2 Опис процесів тестування	61
4.3 Опис контрольного прикладу	67
Висновки до розділу	69
5 РОЗГОРТАННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	70

5.1	Розгортання програмного забезпечення.....	70
5.2	Супровід програмного забезпечення	70
	Висновки до розділу	72
	ВИСНОВКИ.....	72
	СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	76
	ДОДАТКИ.....	77

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

IDE	– Integrated Development Environment – інтегроване середовище розробки.
API	– Application programming interface, прикладний програмний Інтерфейс
CLR	– Common language runtime
HTML	– Hypertext markup language
ER	– Entity-Relation diagram
ОС	– Операційна система.
БД	– База даних.

ВСТУП

Сучасний світ знаходиться у стадії швидкого розвитку, тому все більшої актуальності набуває проблема якісного та доступного навчання. Особливо це стосується сфери професійної освіти та навичок, без яких складно уявити швидку та сучасну освіту, особливо у сферах які зараз стрімко розвиваються, тому важливою задачею є пошук оптимальних форм і методів навчання, здатних задовольнити потреби в отриманні якісних знань та навичок.

Одним з рішень цієї проблеми є створення платформи для навчання людей. Такі платформи дозволяють забезпечити гнучкість, доступність та індивідуалізацію процесу навчання.

Актуальність теми дипломної роботи обумовлена рядом факторів. По-перше, інформаційні технології вже суттєво змінили методи та форми навчання, і розвиток цифрових платформ для навчання може сприяти реалізації освітніх потенціалів людини на ще вищому рівні. По-друге, глобальна пандемія COVID-19, яка призвела до масового переходу на дистанційне навчання, підкреслила важливість створення ефективних рішень для забезпечення якісного навчання в онлайн-форматах.

Ця дипломна робота спрямована на розробку платформи для навчання людей, яка буде враховувати потреби різних категорій користувачів та надавати можливість для гнучкого вибору навчальних матеріалів, методів та форматів. Основна мета роботи полягає у розробці концепції та дизайну платформи, що максимально сприятиме отриманню якісних знань та навичок, задовольняючи потреби сучасної освіти.

У рамках дипломної роботи будуть проаналізовані існуючі платформи для навчання та їх ключові переваги та недоліки. Також буде проведено дослідження потреб користувачів, яке допоможе виявити вимоги до функціональності, інтерфейсу та змісту платформи. Окрім того, буде

розроблений прототип платформи, що враховує висновки дослідження та сучасні тенденції в освіті.

Тому можна сказати що, розробка платформи для навчання людей є актуальною і важливою задачею сучасної освіти, що вимагає всебічного аналізу та інноваційного підходу. Успішне вирішення цієї проблеми може сприяти підвищенню якості освіти, розвитку професійних компетенцій та забезпечити доступ до навчання для всіх верств населення.

1 ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ

1.1 Аналіз предметної області

Масові відкриті онлайн-курси — це інтернет-курс з великомасштабною інтерактивною участю та відкритим доступом через інтернет. На додаток до традиційних матеріалів навчального курсу, такі як відео, читання, і домашніх завдань, MOOC надає можливість використання інтерактивного форуму користувачів, які допомагають створити спільноту студентів, викладачів та асистентів (TAS) [1].

Перші відеозаписи лекцій різних навчальних закладів почали з'являтися у мережі Інтернет ще наприкінці 1990-х років, однак лише MOOC надали змогу зручного інтерактивного спілкування між студентами та викладачами, а також можливість викладачам приймати іспитів в режимі онлайн.

На сьогоднішній день це одна із найновіших форм дистанційного навчання, яка все ще активно розвивається у світовій освіті. Подібні платформи розраховані на студентів різних попередніх рівнів підготовки — як новачків, так і досвідчених фахівців. Також є широкий вибір серед цінових категорій – від дешевого сегменту за маленькі курси до великих курсів які розраховані на декілька місяців.

1.2 Аналіз існуючих рішень

Проаналізуємо відоме на сьогодні алгоритмічне забезпечення у даній області та технічні рішення, що допоможуть у реалізації розподіленої системи агрегації, управління та поширення навчальних матеріалів. Далі будуть розглянуті допоміжні програмні засоби, засоби розробки та готові програмні рішення.

1.2.1 Аналіз відомих програмних продуктів

Аналізуючи відомі платформи для навчання, можна виділити декілька ключових гравців на ринку, які пропонують різні особливості та функціонал:

Coursera [2]: Це одна з найбільш популярних платформ, що пропонує курси від провідних університетів та компаній світу. Coursera пропонує широкий спектр курсів з різних галузей знань, сертифікаційні програми та спеціалізації. Вони також пропонують ступеневі програми, які можуть бути еквівалентом традиційного ступеня вищої освіти.

edX [3]: Це ще одна відома платформа, заснована Гарвардським університетом та Массачусетським технологічним інститутом. edX пропонує безліч курсів з різних дисциплін, а також сертифікаційні програми. edX прагне поширювати якісну освіту по всьому світу, надаючи доступ до курсів безкоштовно, але з можливістю отримання сертифікатів за плату.

Udemy [4]: У порівнянні з Coursera та edX, Udemy фокусується на наданні платформи для інструкторів, які хочуть створювати та продавати свої курси. Курси на Udemy покривають широкий діапазон тем, включаючи бізнес, технології, особистий розвиток та багато іншого. Ціни на курси встановлюються самими інструкторами, а платформа часто пропонує знижки.

Moodle [5]: Відрізняється від інших платформ, оскільки це відкрите програмне забезпечення, що дозволяє організаціям створювати власні платформи для навчання. Він використовується в основному університетами, школами та корпораціями для створення власних навчальних платформ, що відповідають їх специфічним вимогам. Moodle пропонує гнучкість у налаштуванні, інтеграції з іншими системами та можливість контролювати весь процес навчання. Завдяки активній спільноті, Moodle має численні додатки та плагіни, що розширюють його можливості.

Для порівняння курсової роботи з аналогом можна скористатись таблицею 1.3.

Таблиця 1.3 – Порівняння з аналогом

Функціонал	Проект	edX	Coursera	Moodle	Confluence
Реєстрація/вхід	Так (тільки вхід)	Так	Так	Так	Так
Відео-лекції	Ні	Так	Так	Так	Ні (можливі відео в статтях)
Тести	Так	Так	Так	Так	Ні
Отримання сертифікатів	Ні	Так	Так	Так (залежно від курсу)	Ні
Редагування/створення контенту	Так	Ні	Ні	Так (для викладачів)	Так (для користувачів з правами доступу)
Відслідковування прогресу	Так	Так	Так	Так	Ні (окрім перегляду статей)
Оцінювання	Так (автоматичне)	Так (автоматичне та ручне)	Так (автоматичне та ручне)	Так (автоматичне та ручне)	Ні
Редагування/створення контенту	Так	Ні	Ні	Так (для викладачів)	Так (для користувачів з правами доступу)

1.2.2 Аналіз відомих алгоритмічних та технічних рішень

Для початку аналізу необхідно визначити цільову платформу для якої буде розроблятися система, на даний момент створення веб-додатків є одним з найкращих варіантів. По-перше веб-розробка є одним з найпопулярніших напрямів – це можна побачити на рисунку 1.1, де перші 2 місця по популярності мов займають специфічні для веб-розробки мови, що в свою чергу збільшує кількість інформації, яку можна знайти в інтернеті, а також значно зменшує швидкість розробки додатку та часу, який потрібен

розробнику для того щоб орієнтуватись в новому середовищі. По-друге не треба витрачати ресурси на підтримку окремих платформ, оскільки всі сучасні операційні системи мають можливість запускати сучасні браузері. Варто зазначити, що існують деякі складності підтримки веб-застосунків, пов'язані з застарілими браузерами, або з особливостями роботи віртуальних машин Javascript у різних браузерах, але зазвичай ці проблеми досить тривіальні, та достатньо просто вирішуються за допомогою бібліотек, які є у відкритому доступі.

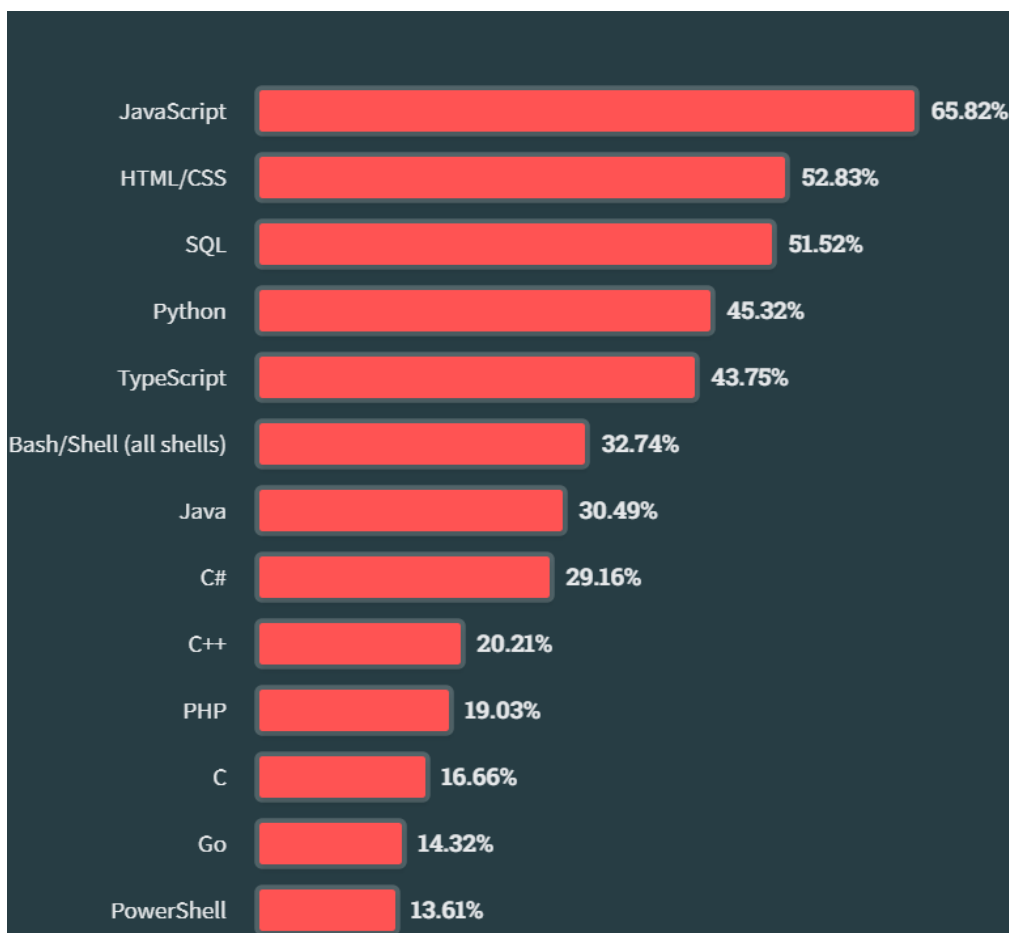


Рисунок 1.1 – опитування ресурсу stack overflow

Також, існує варіант створення мобільного додатку, який виглядає досить оптимальним, але тільки на перший погляд, не дивлячись на те, що існують засоби розробки з можливістю створення додатку на декілька платформ, наприклад Flutter [6], варто розуміти, що для використання API, який є специфічним для платформи варто використовувати мови, які є основними мовами розробки для конкретної платформи – а саме Java, Kotlin

для Android та Objective-C, Swift для IOS, що в свою чергу зобов'язує нас не тільки паралельно вивчати інші мови програмування, а і слідкувати за останніми версіями API операційних систем для того щоб код який ми писали підтримувався на нашій цільовій платформі. Продовжуючи приклад Flutter, треба зазначити, що цей фреймворк може також збирати веб-версію, але поки ця технологія ще розвивається, на даний момент результатом збірки додатку є Javascript код, який генерується в процесі збірки, до якого важко підключити великий набір бібліотек, які існують в різних репозиторіях, що в підсумку дає більш складний процес розробки та більш складний процес відлагоджування.

Тому вибір є розробляти веб-версію є більш оптимальним, як мінімум на початку життєвого циклу додатку.

Основою платформи має бути надійна система зберігання даних, у випадку з платформою для навчання, повинна бути можливість роботи як з базою даних, так і з файловою системою. Це є критично важливою задачею, оскільки крім простих даних, такі як, наприклад, дані користувача, ми можемо мати справу з великими бінарними файлами на кшталт відеолекцій, такі дані недоречно зберігати в БД тому необхідно розробити функціонал який забезпечить цілісність при роботі з файлами.

Спочатку проаналізуємо СУБД які існують на ринку. По-перше необхідно поділити на типи. Існують два основних типи – це SQL та NoSQL [7] системи. Зазвичай вибір між SQL та NoSQL СУБД залежить від конкретних потреб та вимог проекту. Перше про що необхідно сказати це структури зберігання даних та їх цілісність, часто кажуть що NoSQL не має структури, але це не зовсім так, при читанні даних з бази існує структура, наприклад JSON об'єкт, просто ця структура не є сталою, як в SQL БД, де дані зберігаються у вигляді таблиць, які були попередньо описані. Цілісність даних в різних типах баз досягається різними шляхами, так в SQL, за цим слідкує рушій, в NoSQL БД, це обов'язок клієнта.

В контексті БД, треба розуміти, наскільки добре вона може масштабуватися, якщо йде мова про NoSQL, то зазвичай використовується «горизонтальне» масштабування, тобто додавання серверів, або віртуальних машин, SQL використовують «вертикальне» масштабування, тобто покращення характеристик серверу, зазвичай додаванням ресурсів.

Останнім важливим параметром є підтримка транзакцій, більшість NoSQL БД її не мають, зазвичай підтримка послідовних операцій є обов'язком клієнта, SQL підтримують транзакції, що дозволяє забезпечити послідовність, ізоляцію та стабільність операцій з даними. Це означає, що вони забезпечують високий рівень надійності та цілісності даних.

Підсумовуючи все вищесказане про типи БД, можна зробити такі висновки. По-перше, структура даних, залишається незміною впродовж версії, тому варіант структурованих таблиць є досить непоганим. По-друге, підтримка транзакцій та цілісності даних, на рівні БД, є дуже важливою, оскільки знімає навантаження з клієнта, який ними користується, а це в свою чергу полегшує його розробку. Останнє, що варто зазначити, в системі буде багато зв'язків типу one-to-many, або many-to-many, які в свою чергу не дуже легко підтримувати в NoSQL. В реляційних базах, оновлення даних буде заключатися в оновленні однієї таблиці, а в нереляційних, необхідно оновити всі структури де були використані ці дані, або ж робити симуляції з'єднань, що знову ж таки, ускладнює код додатку, оскільки має симулюватися клієнтом. Тому враховуючи усе перелічено, використання SQL БД буде доречним в даному проєкті.

1.3 Опис бізнес-процесів

Для опису бізнес процесу програмного забезпечення використовується BPMN моделі рисунок 1.2.

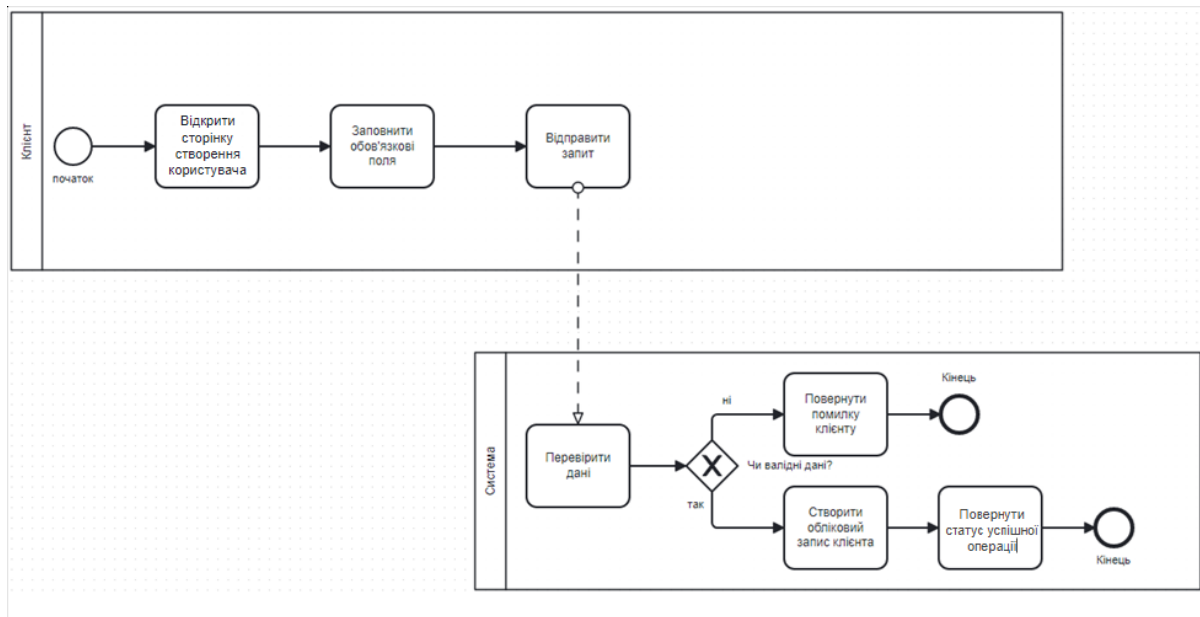


Рисунок 1.2 – Схема бізнес-процесу створення користувача

Опис послідовності створення користувача:

- Адміністратор системи переходить на сторінку створення користувача;
- Заповнює поля реєстрації;
- якщо введені поля, не відповідають шаблону заповнення на клієнтській стороні, відповідні поля підсвічуються помилкою;
- Також відбувається додаткова перевірка на серверній частині програмного забезпечення, при виявленні помилок, клієнтська сторона має відобразити помилку в зручному вигляді в залежності від її типу.

1.4 Постановка задачі

Для реалізації поставленої задачі необхідно реалізувати онлайн платформу для навчання. Розробку можна узагальнити та поділити на 2 основні частини:

- Клієнтська
- Серверна

Серверна частина є ядром системи і повинна реалізувати всі головні функції системи такі як:

- Авторизація та автентифікація
- Робота з СУБД
- Робота з файловим сховищем
- Робота з клієнтами
- Управління користувачами
- Реалізація серверної частини набору функцій платформи
- Надання інтерфейсу для роботи з зовнішніми програмами

Клієнтська частина повинна реалізувати зручну взаємодію з платформою для клієнтів, шляхом створення зручного інтерфейсу користувачу через який буде відбуватися основна взаємодія з серверною частиною.

Висновки до розділу

Підбиваючи підсумки даного розділу можна сказати, що була проаналізована предметна область, поставлені задачі, які необхідно реалізувати. Були проаналізовані технічні рішення та алгоритми, було проведено порівняння існуючих SQL та NoSQL СУБД, розписані їх плюси та недоліки.

2 РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1 Варіанти використання програмного забезпечення

Головною функцією програмного забезпечення є проходження курсів, більше функцій можна побачити на рисунку 2.1

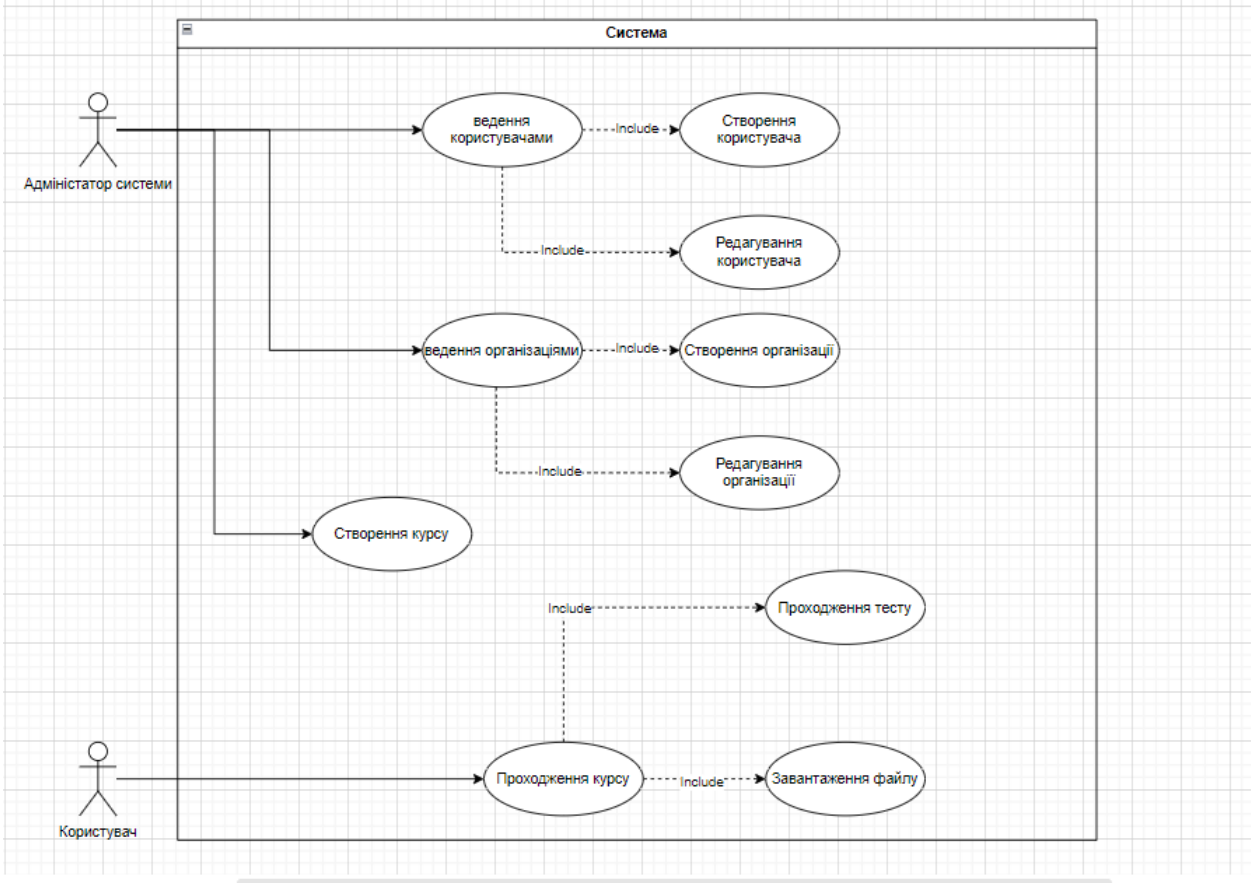


Рисунок 2.1 – Діаграма варіантів використання

В таблицях 2.1 – 2.11 наведені варіанти використання програмного забезпечення.

Таблиця 2.1 - Варіант використання UC-01

Use case name	Створення користувача
Use case ID	UC-01
Goals	Реєстрація нового користувача в системі
Actors	Адміністратор
Trigger	Адміністратор бажає створити нового користувача
Pre-conditions	-
Flow of Events	Користувач переходить до панелі адміністратора. В поля для створення вводяться відповідні дані: логін користувача, пошту, ім'я, прізвище. Після заповнення даних адміністратор натискає кнопку реєстрації. Після цього з'являється повідомлення про успішну реєстрацію, і користувачу на пошту відправляється повідомлення.
Extension	В випадку введення не коректних даних, червоним підсвічуються неправильно введені поля та стає неактивною.
Post-Condition	Створення запису користувача в БД

Таблиця 2.2 - Варіант використання UC-02

Use case name	Автентифікація користувача
Use case ID	UC-02
Goals	Автентифікація нового користувача в системі
Actors	Гість (неzareєстрований користувач)
Trigger	Користувач бажає увійти в систему
Pre-conditions	-
Flow of Events	Користувач переходить на сторінку входу. Вводить логін та пароль Після заповнення полів користувача натискає кнопку входу. Після цього користувач перенаправляється у систему.
Extension	В випадку введення не коректних даних, кнопка входу стає неактивною. Якщо якесь конкретне поле введено некоректно, то воно підсвічується червоним надписом.
Post-Condition	Перехід у систему

Таблиця 2.3 - Варіант використання UC-03

Use case name	Перегляд курсів
Use case ID	UC-03
Goals	Відображення списку курсів
Actors	Користувач
Trigger	Користувач бажає подивитись список курсів
Pre-conditions	Користувач має бути авторизований
Flow of Events	Користувач переходить на сторінку курсів. Після цього користувач повинен бачити усі курси до яких у нього є доступ
Extension	
Post-Condition	

Таблиця 2.4 - Варіант використання UC-04

Use case name	Редагування профілю
Use case ID	UC-04
Goals	Змінити дані в профілю
Actors	Користувач
Trigger	Користувач бажає відредагувати профіль
Pre-conditions	Користувач має бути авторизований
Flow of Events	Користувач відкриває меню та переходить на сторінку профілю. Після цього користувач повинен мати змогу змінити дані які можна редагувати.
Extension	Якщо користувач ввів неправильні дані в якесь поле, воно повинно стати червоним та кнопка зберегти повинна стати неактивною.
Post-Condition	Дані профілю змінені, відбувся update в БД

Таблиця 2.5 - Варіант використання UC-05

Use case name	Скачування файлу
Use case ID	UC-05
Goals	Скачати файл
Actors	Користувач
Trigger	Користувач бажає скачати файлу
Pre-conditions	Користувач має бути авторизований

Продовження таблиці 2.5

Flow of Events	Користувач відкриває меню та переходить на сторінку з доступними курсами. Після цього користувач вибирає курс який він хоче переглянути. Користувача перенаправляє на сторінку курсу де він повинен побачити файли які він має змогу відкрити та скачати необхідний файл.
Extension	
Post-Condition	Користувач бачить зміст файлу

Таблиця 2.6 - Варіант використання UC-06

Use case name	Проходження тесту
Use case ID	UC-06
Goals	Пройти тестування
Actors	Користувач
Trigger	Користувач бажає пройти тест
Pre-conditions	Користувач має бути авторизований
Flow of Events	Користувач відкриває меню та переходить на сторінку з доступними курсами. Після цього користувач вибирає курс який він хоче переглянути. Користувача перенаправляє на сторінку курсу де він повинен побачити тести до яких він має доступ та вибрати один з доступних тестів. Після цього пройти тестування та нажити кнопку зберегти відповідь.
Extension	
Post-Condition	Користувач бачить зміст файлу

Таблиця 2.7 - Варіант використання UC-07

Use case name	Надання доступу користувачу до курсу
Use case ID	UC-07
Goals	Надати доступ користувачу до курсу
Actors	Адміністратор організації
Trigger	Адміністратор організації хоче надати доступ користувачу до курсу
Pre-conditions	Адміністратор організації має бути авторизований

Продовження таблиці 2.7

Flow of Events	Адміністратор організації відкриває меню та переходить до реєстру користувачів. Після переходу Адміністратор організації до реєстру він натискає на кнопку створення нового користувача. Після цього адміністратор вводить обов'язкові дані користувача, вводить одноразовий пароль та натискає кнопку створити.
Extension	Якщо Адміністратор організації ввів неправильні дані в якесь поле, воно повинно стати червоним та кнопка створити повинна стати неактивною.
Post-Condition	Створений новий користувач

Таблиця 2.8 - Варіант використання UC-08

Use case name	Редагування користувачів організації
Use case ID	UC-08
Goals	Відредагувати дані користувача організації
Actors	Адміністратор організації
Trigger	Адміністратор організації хоче відредагувати користувача організації
Pre-conditions	Адміністратор організації має бути авторизований
Flow of Events	Адміністратор організації відкриває меню та переходить до реєстру користувачів. Після переходу до реєстру він відкриває користувача якого хоче відредагувати, та вносить необхідні дані.
Extension	Якщо Адміністратор організації ввів неправильні дані в якесь поле, воно повинно стати червоним та кнопка зберегти повинна стати неактивною.
Post-Condition	Користувач відредагований

Таблиця 2.9 - Варіант використання UC-09

Use case name	Редагування прав користувачів організації
Use case ID	UC-09
Goals	Відредагувати права користувача організації
Actors	Адміністратор організації

Продовження таблиці 2.9

Trigger	Адміністратор організації хоче відредагувати права користувача організації
Pre-conditions	Адміністратор організації має бути авторизований
Flow of Events	Адміністратор організації відкриває меню та переходить до реєстру користувачів. Після переходу до реєстру він відкриває користувача якого хоче відредагувати, та змінює його права.
Extension	
Post-Condition	Користувач відредагований

Таблиця 2.10 - Варіант використання UC-10

Use case name	Створення організації
Use case ID	UC-10
Goals	Створити організацію
Actors	Адміністратор системи
Trigger	Адміністратор системи хоче створити нову організацію
Pre-conditions	Адміністратор системи має бути авторизований
Flow of Events	Адміністратор системи відкриває меню та переходить до реєстру організацій. Після переходу до реєстру він натискає на кнопку створення вводить необхідні дані та натискає кнопку створити.
Extension	Якщо Адміністратор організації ввів неправильні дані в якесь поле, воно повинно стати червоним та кнопка створити повинна стати неактивною.
Post-Condition	Створена нова організація

Таблиця 2.11 - Варіант використання UC-11

Use case name	Створення користувача
Use case ID	UC-11
Goals	Створити користувача
Actors	Адміністратор системи
Trigger	Адміністратор системи бажає створити користувача
Pre-conditions	Адміністратор системи має бути авторизований

Продовження таблиці 2.11

Flow of Events	Адміністратор відкриває меню та переходить до реєстру користувачів. Після переходу адміністратор до реєстру адміністратор натискає на кнопку створення нового користувача. Після цього адміністратор вводить обов'язкові дані користувача, вводить одноразовий пароль та натискає кнопку створити.
Extension	Якщо адміністратор ввів неправильні дані в якесь поле, воно повинно стати червоним та кнопка створити повинна стати неактивною.
Post-Condition	Створений новий користувач

2.2 Аналіз системних вимог

Основною вимогою системи є необхідність запуску на всіх сучасних браузерях, оскільки цільовою платформою є браузер, необхідно максимально розширити список користувачів, які б могли користуватися системою, також сюди входить необхідність розробити адаптивний інтерфейс користувача, тобто можливість користування на різних пристроях.

Для запуску серверної частини додатку необхідно мати комп'ютер з встановленою операційною системою на ядрі Linux, стабільний інтернет зі швидкістю не менше 100 мбіт/сек, 16 ГБ ОЗУ та процесор i5 мінімум з 4 ядрами. Така конфігурація необхідна для того, щоб була можливість швидкої обробки великою кількістю запитів. Варто окремо виділити кількість оперативної пам'яті, 16 Гб пам'яті повинно бути достатньо для того, щоб на перших етапах життя додатку дані з запитів могли повністю завантажуватись в оперативну пам'ять, без додаткових звернень до диску.

2.3 Розроблення функціональних вимог

Функціональні вимоги описані у таблицях 2.13 – 2.23

Таблиця 2.13 – Функціональна вимога FR-1

Назва	Створення користувача
Опис	Система повинна надавати можливість створювати користувача адміністратором системи.

Таблиця 2.14 – Функціональна вимога FR-2

Назва	Автентифікація користувача
Опис	Система повинна надавати можливість заходити в систему користувачеві шляхом введення логіну та паролю

Таблиця 2.15 – Функціональна вимога FR-3

Назва	Вихід з системи
Опис	Система повинна надавати можливість виходити з системи користувачеві

Таблиця 2.16 – Функціональна вимога FR-4

Назва	Редагування профілю
Опис	Система повинна надати можливість користувачу редагувати свій профіль

Таблиця 2.17 – Функціональна вимога FR-5

Назва	Прогрес навчання
Опис	Система повинна надати можливість відображення прогресу користувачів.

Таблиця 2.18 – Функціональна вимога FR-6

Назва	Управління курсами
Опис	Система повинна надати можливість створювати, редагувати та видаляти курси, розділи, лекції та завдання.

Таблиця 2.19 – Функціональна вимога FR-7

Назва	Редагування користувачів організації
Опис	Система повинна надати можливість редагувати користувачів організації адміністратору організації

Таблиця 2.20 – Функціональна вимога FR-8

Назва	Редагування прав користувачів організації
Опис	Система повинна надати можливість редагувати права користувачів організації адміністратору організації

Таблиця 2.21 – Функціональна вимога FR-9

Назва	Блокування користувачів організації
Опис	Система повинна надати можливість блокувати користувачів організації адміністратору організації

Таблиця 2.22 – Функціональна вимога FR-10

Назва	Створення користувачів
Опис	Система повинна надати можливість створювати користувачів адміністратору системи

Таблиця 2.23 – Функціональна вимога FR-11

Назва	Створення організації
Опис	Система повинна надати можливість створювати організації адміністратору системи

Таблиця 2.24 – Функціональна вимога FR-12

Назва	Зміна власника організації
Опис	Система повинна надати можливість змінювати власника організації адміністратору системи

	fr-1	fr-2	fr-3	fr-4	fr-5	fr-6	fr-7	fr-8	fr-9	fr-10	fr-11
uc-1	+										
uc-2		+	+								
uc-3				+							
uc-4				+							
uc-5											
uc-6											
uc-7						+					
uc-8							+	+	+		
uc-9							+	+	+		
uc-10										+	
uc-11											+

Рисунок 2.2 – Матриця трасування вимог

2.4 Розроблення нефункціональних вимог

Нефункціональні вимоги описані у таблицях 2.24 – 2.29

Таблиця 2.24 – Нефункціональна вимога NfR-1

Назва	Можливість запустити інтерфейс у сучасних браузерів
Опис	Інтерфейс системи повинен працювати на сучасних браузерах

Таблиця 2.25 – Нефункціональна вимога NfR-2

Назва	Забезпечення безпеки і конфіденційності даних користувачів
Опис	Дані користувача повинні зберігатися безпечно

Таблиця 2.26 – Нефункціональна вимога NfR-3

Назва	Автоматизоване керування файловим сховищем
Опис	Система повинна мати функціонал для керуванням файловим сховищем у автоматичному режимі

Таблиця 2.27 – Нефункціональна вимога NfR-4

Назва	Написана на C# та JavaScript. СУБД має бути PostgreSQL
Опис	програмне забезпечення повинно бути написано з використанням мов програмування C# та JavaScript, а також на базі системи керування базами даних PostgreSQL.

Таблиця 2.28 – Нефункціональна вимога NfR-5

Назва	Підтримка програмних інтерфейсів для інтеграції з зовнішніми системами
Опис	Система повинна мати програмний інтерфейс для інтеграції з зовнішніми системами

Таблиця 2.29 – Нефункціональна вимога NfR-6

Назва	Підтримка Linux
Опис	Серверна частина повинна запускатись на операційній системі Linux

Висновки до розділу

В даному розділі були описанні основні функціональні та нефункціональні вимоги, для користувачів з різними правами в системі, також була побудована діаграма використання, де було описано дії які можуть робити користувачі системи. Також був проведений аналіз вимог до системи в якому було розписано необхідні вимоги яких необхідно дотриматись при розробці програмного забезпечення.

За результатами розділу сформовано технічне завдання на розробку програмного забезпечення.

3 КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 Архітектура програмного забезпечення

- DB: СУБД PostgreSQL
- File Storage: Сховище для зберігання файлів
- Server: Головний сервер онлайн платформи
- Message Broker: RabbitMQ
- Reverse Proxy: Nginx
- Clients: Користувачі які підключаються через клієнт написаний на VueJs

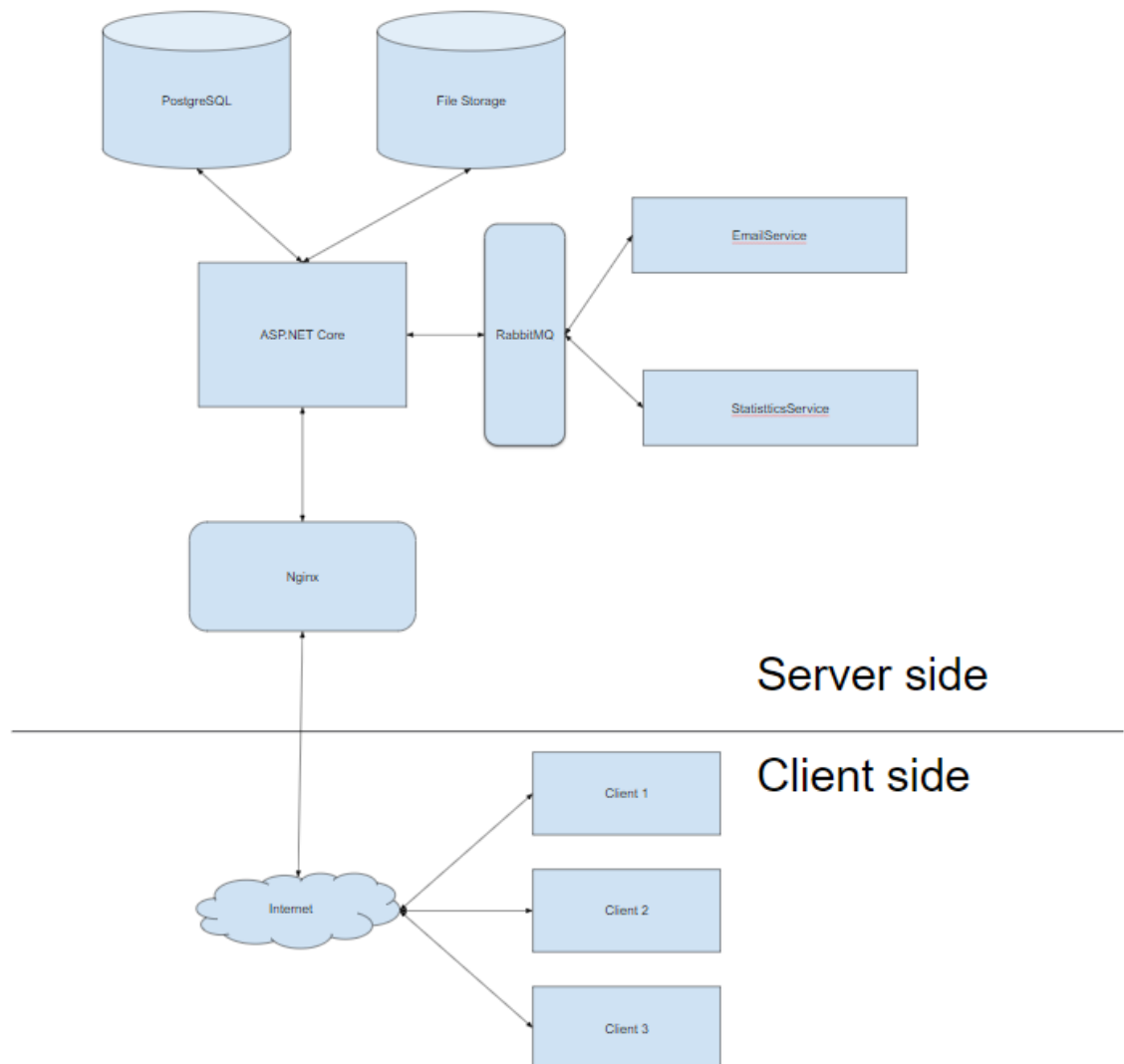


Рисунок 3.1 – Загальна архітектура

3.2 Обґрунтування вибору засобів розробки

Оскільки розробка серверної частини буде вестися на платформі .NET core, необхідно підібрати веб-фреймворк, який може розкрити потенціал цієї платформи. Тому був обраний ASP.NET Core як основний веб-фреймворк для розробки. Завдяки структурі фреймворку процес розробки нового функціоналу триває достатньо швидко, в більшості випадків розробнику необхідно створити клас контролера запиту, який буде автоматично знайдений внутрішнім функціоналом ASP.NET, вирішенням залежностей для контролером також займається фреймворк оскільки має вбудований IoC-container [8], тому зі сторони розробника достатньо один раз зареєструвати залежність яка йому необхідна, і вона буде передана у конструктор контролера за допомогою впровадження залежностей [9]. Звісно ж що проблема маршрутизації запитів також вирішується внутрішніми інструментами.

Після вирішення структурних питань далі необхідно вирішити інше важливе питання, а саме питання авторизації та автентифікації. Для того щоб вирішити це питання існує модуль для ASP.NET Core, який має назву Identity. Цей модуль надає графічний інтерфейс для автентифікацію та керує доступами та правами користувача. Також цей модуль має підтримку сторонніх провайдерів для автентифікації таких як Google, Microsoft, та інших.

Останнє що необхідно зробити для того щоб завершити формування ядра серверної частини це вибрати ORM для роботи з базою даних. Для цього є, Entity Framework Core (EF Core), він є офіційною ORM бібліотекою для .NET Core, розробленою Microsoft, яка дозволяє розробникам працювати з СУБД, використовуючи .NET-об'єкти, і автоматично відображає їх на таблиці бази даних. EF Core добре інтегрується з ASP.NET Core, роблячи його відмінним вибором для розробки сучасних веб-застосунків. Він пропонує підтримку різних баз даних, забезпечує високу продуктивність і

масштабованість, а також мінімізує необхідність написання SQL-запитів вручну.

Підсумовуючи вище написано можемо виділити головні переваги:

- Модульність: це означає що немає потреби одразу включати у збірку всі модулі фреймворку, тобто можна використовувати лише те що треба на момент розробки.
- IoC та DI «з коробки»: це означає що питання вирішення залежностей для класів вирішується фреймворком, що зменшує кількість коду який необхідно писати розробникам та значно полегшує тестування
- Identity core: Цей модуль вирішує більшість питань які виникають при керуванні користувачами.
- EF Core: Офіційна ORM розроблена Microsoft, гарно інтегрується з ASP.NET Core та має широкий набір інструментів для роботи з БД.

Тепер можна перейти до опису клієнтської частини, оскільки інтерфейс буде розроблятися на мові програмування JavaScript, то необхідно зробити вибір фреймворку для розробки інтерфейсу. Виходячи з рисунку 1.3.1.1 можна побачити, що три найпопулярніші фреймворки це Vue, Angular, React, тому зробимо їх аналіз.

Vue.js відрізняється гнучкістю, швидкістю та зручністю в освоєнні, що робить його хорошим варіантом для малих та середніх проєктів. Завдяки поступовій адаптації можна використовувати лише ті частини фреймворка, які потрібні для проєкту і при збільшенні складності проєкта додавати інші.

Angular - це повноцінний фреймворк, розроблений Google, що підтримує розробку великих та складних застосунків. Він використовує TypeScript, що забезпечує статичну типізацію та поліпшену підтримку помилок. Однак, він має крутіший кут навчання порівняно з Vue та React.

React, розроблений Facebook, є гнучким фреймворком, зорієнтованим на роботу з компонентами. Завдяки віртуальному DOM, він забезпечує відмінну продуктивність та широкі можливості для розширення та налаштування за допомогою численних додаткових бібліотек та пакетів.

Vue.js може бути відмінним вибором для розробки платформи для навчання, оскільки він є легким та гнучким фреймворком, що дозволяє швидко створювати застосунки з невеликим навантаженням на браузер. Це забезпечує плавну роботу платформи навчання навіть на пристроях з обмеженими ресурсами, що є важливим аспектом для доступності освіти.

Крім того, Vue.js має простий та зрозумілий синтаксис, що сприяє швидкому освоєнню розробниками. Завдяки інтуїтивно зрозумілому середовищу розробки, команда розробників зможе ефективно працювати над створенням платформи для навчання, адаптувати її до вимог користувачів та швидко впроваджувати нові функції.

3.3 Конструювання програмного забезпечення

Для кращого розуміння того як працює основний сервер, варто зрозуміти як працюють його окремі компоненти, тому опис архітектури краще почати з опису допоміжних сервісів. Перший сервіс – це сервіс для надсилення нотифікацій по електронній пошті. Зазвичай операції по надсиленню електронної пошти є досить повільними за рахунок операцій, які треба зробити для підключення до поштового серверу. Також ситуація може погіршитись від чинників які часто не залежать від роботи додатків, такі як антивіруси, погане з'єднання з мережею та ін. Тому доцільно винести операції пов'язані з надсиленням поштових нотифікацій таким чином, щоб вони мінімально впливали на основну роботу сервера, в архітектурі додатку це буде окремий сервіс. На рисунку 3.2 можна побачити діаграму класів сервісу.

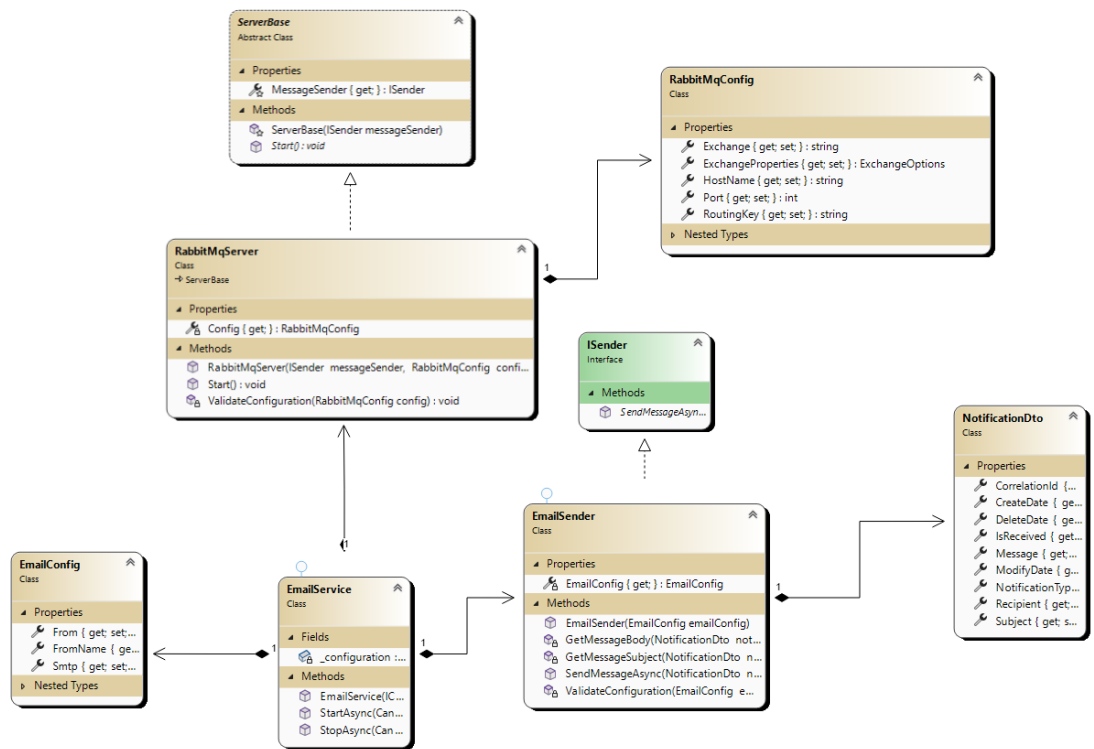


Рисунок 3.2 – Діаграма класів сервісу для відправки поштових нотифікацій

Перед запуском серверу необхідно налаштувати конфігураційний файл application.json. Приклад конфігурації вказаний на рисунку 3.3

```

{
  "RabbitMq": {
    "HostName": "localhost",
    "Port": 5672,
    "Exchange": "main",
    "RoutingKey": "notification.mail",
    "ExchangeProperties": {
      "Durable": false,
      "AutoDelete": false
    }
  },
  "Email": {
    "From": "mail@example.com",
    "FromName": "Example name",
    "Smtp": {
      "UserName": "",
      "Password": "",
      "Host": "server.domain.com",
      "Port": 587
    }
  }
}

```

Рисунок 3.3 – Файл конфігурації

В секції «RabbitMq» конфігурується адреса сервера де встановлений брокер повідомлень та параметри роботи. Розглянемо налаштування черги трохи детальніше. Почнемо з параметру «Exchange», він відповідає на назву обміну. У випадку RabbitMq обмін – це сутність яка отримує повідомлення та перенаправляє його у черги в залежності від конфігурації. В сервісі можна вказати параметр «Durable», для того щоб вказати чи повинно повідомлення зберігатися в разі, якщо сервер брокеру вимкнеться з якоїсь причини. Параметр «AutoDelete» потрібний для того, щоб повідомити брокеру, чи потрібно видаляти чергу коли її останній підписант відписався.

Секція «Email» потрібна для опису параметрів підключення до поштового серверу. Параметри «From» та «FromName» потрібні для вказування адреси відправника. В секції «Smtp» задаються параметри підключення до Smtp сервера, де поля «Username» та «Password» потрібні для аутентифікації, а поля «Host» та «Port» для адреси самого серверу. Тепер можна перейти до огляду основних файлів у сервісі.

Розпочнемо з файлу «Program.cs», це точка старту додатку він запускає сервіс. Код можна побачити на рисунку 3.4.

```
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;

var host = Host.CreateDefaultBuilder(args)
    .ConfigureServices((_, services) =>
    {
        services.AddHostedService<EmailService.EmailService>();
    })
    .Build();

await host.RunAsync();
```

Рисунок 3.4 – Файл Program.cs

Метод «CreateDefaultBuilder» створює об'єкт класу будівельника. В якому реєструємо сервіс «EmailService». «EmailService» – це клас, який реалізує

інтерфейс «IHostedService», він є частиною бібліотеки .Net и потрібен для того щоб клас «EmailService» можна було використовувати в методі «AddHostedServices». Для розуміння роботи класу, достатньо фрагменту коду з рисунку 3.5. Метод «StartAsync» це метод інтерфейсу «IHostedService» цей метод використовується для запуску сервісу. В цьому методі зчитується конфігураційний файл та створюються 2 класи. Перший клас – це «EmailSender» та «RabbitMqServer».

```
public Task StartAsync(CancellationToken cancellationToken)
{
    var rabbitMqConfig = new RabbitMqConfig();
    _configuration.GetSection("RabbitMq").Bind(rabbitMqConfig);

    var emailConfig = new EmailConfig();
    _configuration.GetSection("Email").Bind(emailConfig);

    var messageSender = new EmailSender(emailConfig);
    var server = new RabbitMqServer(messageSender, rabbitMqConfig);
    server.Start();
    return Task.CompletedTask;
}
```

Рисунок 3.5 – Метод StartAsync

Перед тим як перейти до головного класу «RabbitMqServer» залишилося розглянути клас «EmailSender», для цього розглянемо код з рисунку 3.6.

```

public async Task SendMessageAsync(NotificationDto notificationDto)
{
    var message = new MailMessage();
    message.IsBodyHtml = true;
    message.From = new MailAddress(EmailConfig.From!, EmailConfig.FromName!);
    message.To.Add(new MailAddress(notificationDto.Recipient));
    message.Subject = GetMessageSubject(notificationDto);
    message.Body = GetMessageBody(notificationDto);

    using var client = new SmtpClient(EmailConfig.Smtp!.Host, EmailConfig.Smtp.Port);
    client.Credentials = new NetworkCredential(EmailConfig.Smtp.UserName, EmailConfig.Smtp.Password);
    client.EnableSsl = true;

    try
    {
        Console.WriteLine($"Attempting to send email to {notificationDto.Recipient}");
        client.Send(message);
        Console.WriteLine("Email sent!");
    }
    catch (Exception ex)
    {
        Console.WriteLine("The email was not sent.");
        Console.WriteLine("Error message: " + ex.Message);
    }
}

```

Рисунок 3.6 – Метод SendMessageAsync

Можна побачити, що при виклику метода «SendMessageAsync» формується лист та відправляється по Smtp протоколу. Всі інструменти для відправки, які використовуються на рисунку 3.6, є класами стандартної бібліотеки .net, тому деталі їх реалізації можна подивитись в офіційній документації. Залишилось розглянути роботу тільки класу «RabbitMqServer». Для запуску сервера, необхідно викликати метод «Start», після виклику методу створюється обмін та черга для взаємодії з RabbitMq, це можна побачити на рисунку 3.7

```

public override void Start()
{
    ValidateConfiguration(Config);

    var factory = new ConnectionFactory
    {
        HostName = Config.HostName,
        Port = Config.Port
    };
    using var connection = factory.CreateConnection();
    using var channel = connection.CreateModel();
    channel.ExchangeDeclare(
        Config.Exchange,
        "topic",
        Config.ExchangeProperties!.Durable,
        Config.ExchangeProperties!.AutoDelete
    );

    var queueName = channel.QueueDeclare().QueueName;
    channel.BasicQos(0, 1, false);
    channel.QueueBind(queueName, Config.Exchange, Config.RoutingKey);
}

```

Рисунок 3.7 – Підключення до RabbitMQ

Після створення черги сервіс підписується на подію отримання нотифікації з брокеру, після отримання повідомлення, програма намагається його десерілізувати з json рядку у об'єкт класу NotificationDto, якщо конвертація вдала, то відправляється повідомлення за допомогою об'єкта EmailSender, який був створений раніше.

```

public class NotificationDto
{
    public string CorrelationId { get; set; }
    public string Subject { get; set; }
    public string Message { get; set; }
    public string Recipient { get; set; }
    public string NotificationType { get; set; }
    public bool IsReceived { get; set; }
    public DateTime CreateDate { get; set; }
    public DateTime ModifyDate { get; set; }
    public DateTime? DeleteDate { get; set; }
}

```

Рисунок 3.8 – клас NotificationDto

```

var consumer = new EventingBasicConsumer(channel);
consumer.Received += async (model, ea) =>
{
    Console.WriteLine($"Received message. On exchange: {ea.Exchange} with routingKey: {ea.RoutingKey}");
    try
    {
        var dataString = Encoding.UTF8.GetString(ea.Body.ToArray());
        var notificationDto = JsonSerializer.Deserialize<NotificationDto>(dataString);
        if (notificationDto is null)
            throw new Exception("Error on message deserialize");

        await MessageSender.SendMessageAsync(notificationDto);
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
    }
    finally
    {
        channel.BasicAck(ea.DeliveryTag, multiple: false);
    }
};
channel.BasicConsume(queueName, false, consumer);

Console.WriteLine("Awaiting for requests. Press [enter] to exit");
}

```

Рисунок 3.9 – Обробник отриманих повідомлень з брокеру

В наступних таблицях описані класи які представлені на рисунку 3.2.

Таблиця 3.1 – Опис членів класу EmailService

Назва	Тип	Опис
_configuration	IConfiguration	Доступ до параметрів конфігурації.
StartAsync	Task	Запускає службу.
StopAsync	Task	Зупиняє службу.
_configuration	IConfiguration	Доступ до параметрів конфігурації.

Таблиця 3.2 – Опис членів класу ServerBase

Назва	Тип	Опис
MessageSender	ISender	Механізм для відправки повідомлень.
Start	void	Абстрактний метод, який повинен бути реалізований у похідному класі для запуску сервера.

Таблиця 3.3 – Опис членів класу RabbitMqServer

Назва	Тип	Опис
Config	RabbitMqConfig	Конфігурація для сервера RabbitMq.
Start	void	Запускає сервер RabbitMq, приймає та обробляє повідомлення.
ValidateConfiguration	void	Перевіряє коректність конфігурації, у разі, якщо вона неправильна кидає помилку

Таблиця 3.4 – Опис членів класу EmailSender

Назва	Тип	Опис
EmailConfig	EmailConfig	Конфігурація для відправки електронних листів.
SendMessageAsync	Task	Відправляє електронне повідомлення.
ValidateConfiguration	void	Перевіряє коректність конфігурації, у разі, якщо вона неправильна кидає помилку

Таблиця 3.5 – Опис членів класу NotificationDto

Назва	Тип	Опис
CorrelationId	string	Ідентифікатор кореляції для відстеження повідомлень.
Subject	string	Тема повідомлення.
Message	string	Тіло повідомлення.
Recipient	string	Отримувач повідомлення.
NotificationType	string	Тип повідомлення.
IsReceived	bool	Чи отримано повідомлення.

Продовження таблиці 3.5

CreateDate	DateTime	Дата створення повідомлення.
ModifyDate	DateTime	Дата останньої модифікації повідомлення.
DeleteDate	DateTime?	Дата видалення повідомлення.

Таблиця 3.6 – Опис членів класу RabbitMqConfig

Назва	Тип	Опис
HostName	string	Ім'я хоста для RabbitMQ.
Port	int	Порт для RabbitMQ.
Exchange	string	Обмінник RabbitMQ для відправки повідомлень.
RoutingKey	string	Ключ маршрутизації для відправки повідомлень до відповідної черги.
ExchangeProperties	ExchangeOptions	Налаштування обмінника.

Таблиця 3.7 – Опис членів класу EmailConfig

Назва	Тип	Опис
From	string	Адреса електронної пошти відправника.
FromName	string	Ім'я відправника.
Smtp	SmtpOptions	Налаштування для протоколу SMTP.

Таблиця 3.8 – Опис членів класу SmtpOptions

Назва	Тип	Опис
UserName	string	Ім'я користувача для SMTP.
Password	string	Пароль для SMTP.
Host	string	Хост для SMTP.
Port	int	Порт для SMTP.

Переходимо до зберігання даних у нашому застосунку, опис почнемо зі зберігання файлу. Для роботи серверу необхідна була бібліотека, яка могла би не тільки зберігати файли та створювати директорії, а й у випадку нештатних ситуацій робити повний відкат усіх дій, це потрібно для ситуацій, де, наприклад, при додаванні декількох файлів з одним сталася проблема при записі, всі інші файли мають видалитися. Тому була створена бібліотека, яка підтримує усі вимоги, які були описані вище.

Тепер необхідно детальніше розібратися з тим як працює бібліотека. За основу було вирішено взяти шаблон проектування Команда [10] – його ідея полягає у створенні маленьких класів, які зазвичай реалізують інтерфейс з двома методами, де перший метод потрібен для того щоб зробити дію, а другий для того щоб цю дію відмінити, або інвертувати, приклад такого інтерфейсу показано на рисунку 3.10.

```
namespace FileStorage.Commands;

public interface ICommand : IDisposable
{
    public ErrorCodes Do();
    public ErrorCodes Undo();
}
```

Рисунок 3.10 – Інтерфейс ICommand

Для того щоб створити дії з файловим сховище достатньо створити клас, який реалізує цей інтерфейс, на рисунку 3.11 показано приклад програмного файлу CreateFile.cs.


```

public ErrorCodes Do()
{
    try
    {
        var location = Path.Join(_config.Location, StorageFile.Path);
        var fileInfo = new FileInfo(location);
        if (fileInfo.Directory is null || !fileInfo.Directory.Exists) return ErrorCodes.DirectoryNotFound;
        if (File.Exists(location)) return ErrorCodes.FileAlreadyExists;
        File.WriteAllBytes(location, StorageFile.Content);
    }
    catch (DirectoryNotFoundException e)
    {
        Console.Error.Write(e);
        return ErrorCodes.DirectoryNotFound;
    }

    return ErrorCodes.None;
}

public ErrorCodes Undo()
{
    return new RemoveFile(StorageFile, _config, true).Do();
}

```

Рисунок 3.11 – Створення файлу

За такою антологією створені інші дії з файловою системою. Перевага такого підходу полягає в тому, що при необхідності можна комбінувати команди, для цієї мети був створений спеціальний клас, в нього передається черга команд, яку він намагається виконати, і у випадку якщо одна з команд не була успішно виконано, то цей клас проходить по всім виконаним командам та викликає у них метод Undo, для того щоб повернути сховище у стан в якому воно було до початку виконання черги команд. Схема алгоритму показана на рисунку 3.12

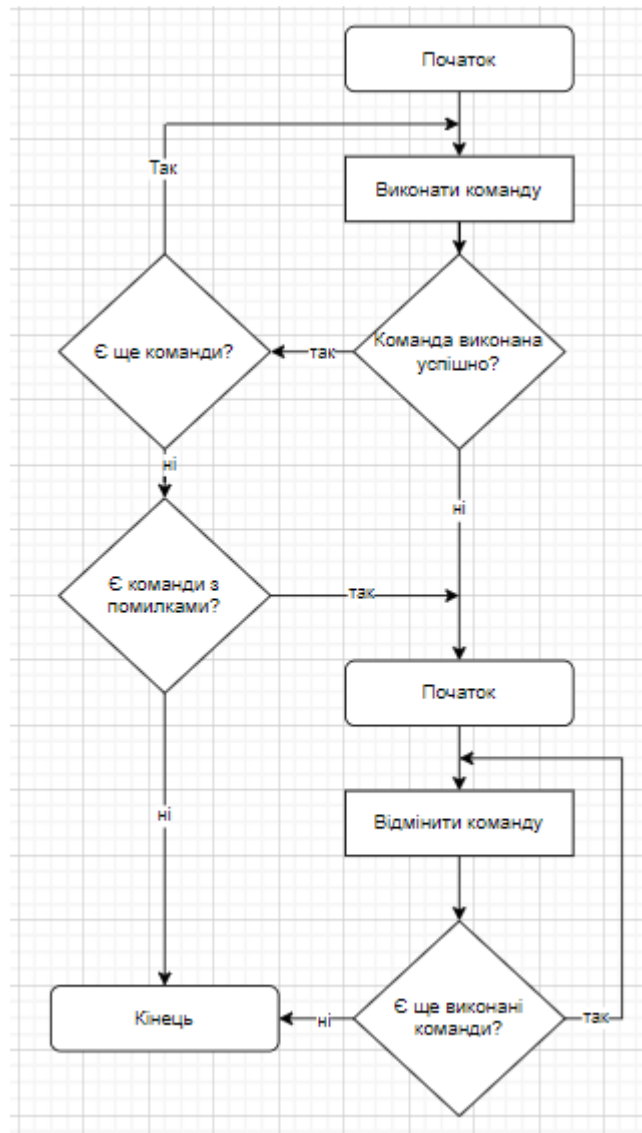


Рисунок 3.12 – Схема роботи алгоритму виконання команд

Також важливим аспектом є можливість обходити директорію. Директорія може бути зручно представлена у вигляді дерева. Важливим є те що логіка обходу дерева має бути представлена клієнтом, який користується бібліотекою. Для буде використано шаблон проектування відвідувач [11]. Особливість цього шаблону полягає в тому, що він дозволяє додавати логіку при обході дерева, при цьому не змінюючи сам алгоритм. Для його реалізації достатньо створити інтерфейс, який буде передаватися у функцію обходу структури сховища, приклад інтерфейсу показано на рисунку 3.13

```
using FileStorage.Dto;

namespace FileStorage;

public interface IStorageVisitor<T>
{
    public T VisitDirectory (StorageDirectory directory);
    public T VisitFile(StorageFile file);
}
```

Рисунок 3.13 – Інтерфейс IStorageVisitor

Тепер можна перейти до опису класів для взаємодії з бібліотекою. Класи описані в таблицях 3.9-3.19

Таблиця 3.9 – StorageDirectory

Властивість/Метод	Опис
Id	Цілочисельний ідентифікатор директорії.
Path	Шлях до директорії.
ParentId	Цілочисельний ідентифікатор батьківської директорії.
ParentDir	Посилання на батьківську директорію.
Meta	Словник, що містить метадані директорії у вигляді пар ключ-значення.
Accept<T>(IStorageVisitor<T> visitor)	Метод, який приймає відвідувача IStorageVisitor<T> та викликає його метод VisitDirectory, передаючи власний екземпляр. Повертає результат відвідувача.

Таблиця 3.10 – StorageFile

Назва	Опис
Id	Цілочисельний ідентифікатор файлу.
Name	Ім'я файлу.
Path	Шлях до файлу.
Content	Масив байтів, що представляє вміст файлу.
Directory	Директорія, в якій знаходиться цей файл.
Meta	Словник, що містить метадані файлу у вигляді пар ключ-значення.

Таблиця 3.11 – ICommand

Назва	Опис
Do()	Виконує операцію, пов'язану з командою. Повертає код помилки ErrorCodes, якщо операція не вдалася.
Undo()	Відміняє виконану операцію, пов'язану з командою. Повертає код помилки ErrorCodes, якщо операція не вдалася.

Таблиця 3.12 – IStorageVisitor

Назва	Опис
VisitDirectory(StorageDirectory directory)	Метод, що відвідує об'єкт типу StorageDirectory та повертає об'єкт типу T

Продовження таблиці 3.12

VisitFile(StorageFile file)	Метод, що відвідує об'єкт типу StorageFile та повертає об'єкт типу T.
-----------------------------	---

Таблиця 3.13 – Config

Властивість/Метод	Опис
Location	Шлях до місцезнаходження, яке використовується для зберігання даних.
AutoCommit	Логічне значення, що вказує, чи потрібно автоматично виконувати збереження змін у системі зберігання.
MetaFileExtension	Рядок, що визначає розширення файлу метаданих за замовчуванням.

Таблиця 3.14 – ConfigurationManager

Назва	Опис
_configPath	Приватне поле, що зберігає шлях до файлу конфігурації.
ConfigurationManager(string configPath)	Конструктор класу, який приймає шлях до файлу конфігурації та зберігає його у відповідному приватному полі.
GetConfig()	Метод, який зчитує конфігураційний файл з вказаного шляху, створює об'єкт Config і повертає його.

Таблиця 3.15 – CmdExecutor

Назва	Опис
Execute(List<ICommand> commands)	Алгоритм з рисунку 3.12
Dispose(List<ICommand> commands)	Приватний метод, який викликає метод <code>Dispose()</code> для кожної команди у списку. Викликається для звільнення ресурсів після виконання всіх команд

Таблиця 3.16 - FileScope

Назва	Опис
CreateFile(StorageFile file)	Створює новий файл та відповідний метафайл, використовуючи конфігурацію, отриману з ConfigurationManager. Додає виконавчі команди до списку команд.
CreateDirectory(StorageDirectory directory)	Створює нову директорію та відповідний метафайл, використовуючи конфігурацію, отриману з ConfigurationManager. Додає виконавчі команди до списку команд.

Продовження таблиці 3.16

RemoveDirectory(StorageDirectory directory)	Видаляє директорію та відповідний метафайл, використовуючи конфігурацію, отриману з ConfigurationManager. Додає виконавчі команди до списку команд.
RemoveFile(StorageFile file)	Видаляє файл та відповідний метафайл, використовуючи конфігурацію, отриману з ConfigurationManager. Додає виконавчі команди до списку команд.
ClearCommands()	Очищує список виконавчих команд.
Commit()	Виконує всі виконавчі команди, які зберігаються у списку, використовуючи CmdExecutor. Якщо операція пройшла успішно, очищує список команд та повертає порожній список помилок.
Rollback()	Скасовує (очищує) список виконавчих команд.

Таблиця 3.17 – StorageContext

Назва	Опис
_configurationManager	Приватне поле для зберігання екземпляру ConfigurationManager, який використовується для отримання конфігурації.

Продовження таблиці 3.17

<code>_commandExecutor</code>	Приватне поле для зберігання екземпляру <code>CmdExecutor</code> , який використовується для виконання команд.
<code>Scopes</code>	Словник, що містить області зберігання файлів.
<code>StorageContext(ConfigurationManager configurationManager, CommandExecutor commandExecutor)</code>	Конструктор, який приймає екземпляри <code>ConfigurationManager</code> та <code>CmdExecutor</code> для використання в класі.
<code>CreateScope(string name)</code>	Метод для створення нової області зберігання з вказаним ім'ям.
<code>RemoveScope(string name)</code>	Метод для видалення області зберігання за іменем.
<code>AddDirectory(StorageDirectory directory)</code>	Метод для додавання нової директорії у глобальну область зберігання.
<code>RemoveDirectory(StorageDirectory directory)</code>	Метод для видалення директорії з глобальної області зберігання.
<code>AddFile(string scope, StorageFile file)</code>	Метод для додавання нового файлу до області зберігання з вказаним ім'ям.
<code>AddFile(StorageFile file)</code>	Метод для додавання нового файлу до глобальної області зберігання.
<code>RemoveFile(string scope, StorageFile file)</code>	Метод для видалення файлу з області зберігання з вказаним ім'ям.
<code>RemoveFile(StorageFile file)</code>	Метод для видалення файлу з глобальної області зберігання.

Кінець таблиці 3.17

Commit()	Метод для збереження всіх змін у всіх областях зберігання.
----------	--

Таблиця 3.18 – StoreUtils

Властивість/Метод	Опис
_configurationManager	Приватне поле для зберігання екземпляру ConfigurationManager, який використовується для отримання конфігурації.
StorageUtils(ConfigurationManager configurationManager)	Конструктор, який ініціалізує об'єкт StorageUtils. Отримує екземпляр ConfigurationManager для використання в класі.
Deserialize(string meta)	Статичний метод, який десеріалізує метадані з рядка у форматі JSON у словник.
Walk<T>(StorageDirectory storageDirectory, IStorageVisitor<T> storageVisitor)	Метод для обходу дерева файлової системи з використанням заданого відвідувача та викликом його методів відповідно до потреб.
GetMeta(string path)	Метод для отримання метаданих за вказаним шляхом.
GetMetaAsync(string path)	Асинхронний метод для отримання метаданих за вказаним шляхом.
GetFullPath(string path)	Метод для отримання повного шляху до файлу або директорії.

Кінець таблиці 3.18

GetRelativePath(string fullPath)	Метод для отримання відносного шляху відносно кореневої директорії.
----------------------------------	---

Таблиця 3.19 – FileStorage

Назва	Опис
Context	Об'єкт <code>StorageContext</code> , який використовується для управління областями зберігання та виконання змін.
ConfigurationManager	Приватне поле для зберігання екземпляру <code>ConfigurationManager</code> , який використовується для отримання конфігурації.
Utils	Приватне поле для зберігання екземпляр <code>StorageUtils</code> , який надає різні допоміжні функції для роботи з файловою системою.
FileStorage(string config)	Конструктор, який ініціалізує об'єкт <code>FileStorage</code> .
GetFullPath(string p)	Метод для отримання повного шляху до файлу або директорії з відносного шляху.
GetDirectory(string path)	Метод для отримання об'єкта <code>StorageDirectory</code> за вказаним шляхом.

Продовження таблиці 3.19

GetDirectoryContent(StorageDirectory directory)	Метод для отримання вмісту директорії у вигляді списку об'єктів IStorageEntity
GetFile(string path)	Метод для отримання об'єкта StorageFile за вказаним шляхом.
Walk<T>(StorageDirectory directory, IStorageVisitor<T> visitor)	Метод для виконання обходу дерева файлової системи з використанням заданого відвідувача.
GetFileContent(StorageFile file)	Метод для отримання вмісту файлу у вигляді масиву байтів.

Перед тим як описувати головний сервер, необхідно описати структуру бази даних, її опис наведений в таблицях 3.20 – 3.27

Таблиця 3.20 – ТаблицяAspNetUsers

Id	Унікальний ідентифікатор для кожного користувача.
CreateDate	Мітка часу, що вказує на час створення користувача.
ModifyDate	Мітка часу, що вказує на час останньої модифікації користувача.
DeleteDate	Мітка часу, що вказує на час видалення користувача. (Необов'язково)
IsSoftDeleted	Булеве значення, яке вказує, чи був користувач видалений "м'яким" способом.
UserName	Ім'я користувача.
NormalizedUserName	Нормалізоване ім'я користувача.
Email	Адреса електронної пошти користувача.

Продовження таблиці 3.20

NormalizedEmail	Нормалізована адреса електронної пошти користувача.
EmailConfirmed	Булеве значення, що підтверджує, що адреса електронної пошти користувача підтверджена.
PasswordHash	Хеш пароля користувача.
SecurityStamp	Значення безпеки для управління конфіденційністю пароля та аутентифікацією користувача.
ConcurrencyStamp	Токен, який використовується для обробки конфліктів з паралельною роботою під час оновлень.
PhoneNumber	Номер телефону користувача.
PhoneNumberConfirmed	Булеве значення, що підтверджує, що номер телефону користувача підтверджено.
TwoFactorEnabled	Булеве значення, що вказує, чи увімкнена двофакторна аутентифікація для користувача.
LockoutEnd	Мітка часу, що вказує на час закінчення блокування облікового запису користувача. (Необов'язково)
LockoutEnabled	Булеве значення, що вказує, чи увімкнене блокування облікового запису користувача.

Таблиця 3.21 – таблицяAspNetRoles

Id	Унікальний ідентифікатор для кожної ролі.
CreateDate	Мітка часу, що вказує на час створення ролі.
ModifyDate	Мітка часу, що вказує на час останньої модифікації ролі.

Продовження таблиці 3.21

DeleteDate	Мітка часу, що вказує на час видалення ролі. (Необов'язково)
IsSoftDeleted	Булеве значення, яке вказує, чи була роль видалена "м'яким" способом.
Name	Назва ролі.
NormalizedName	Нормалізована назва ролі.
ConcurrencyStamp	Токен, який використовується для обробки конфліктів з паралельною роботою під час оновлень.

Таблиця 3.22 – Таблиця Tenants

Id	Унікальний ідентифікатор для кожної організації.
CreateDate	Мітка часу, що вказує на час створення організації.
ModifyDate	Мітка часу, що вказує на час останньої модифікації організації.
DeleteDate	Мітка часу, що вказує на час видалення орендаря. (Необов'язково)
IsSoftDeleted	Булеве значення, яке вказує, чи був орендар видалений "м'яким" способом.

Таблиця 3.23 – Таблиця PlatformRoles

Атрибут	Опис
Id	Унікальний ідентифікатор ролі на платформі.
Name	Назва ролі.
CreateDate	Мітка часу створення ролі.
ModifyDate	Мітка часу останньої модифікації ролі.
DeleteDate	Мітка часу видалення ролі.
IsSoftDeleted	Булеве значення, яке вказує, чи була роль видалена "м'яким" способом.

Продовження таблиці 3.23

TenantUserId	Зовнішній ключ, що посилається на користувача організації, до якого належить ця роль.
UserId	Зовнішній ключ, що посилається на користувача, якому надана ця роль.
Code	Код ролі.

Таблиця 3.24 – Таблиця Courses

Id	Унікальний ідентифікатор для кожного курсу.
Name	Назва курсу.
Description	Опис курсу.
TenantId	Зовнішній ключ, який посилається на організацію, до якої належить цей курс.
IsPublic	Булеве значення, яке вказує, чи є курс доступним для загального перегляду.
IsPublished	Булеве значення, яке вказує, чи був курс опублікований.
CreateDate	Мітка часу, що вказує на час створення курсу.
ModifyDate	Мітка часу, що вказує на час останньої модифікації курсу.
DeleteDate	Мітка часу, що вказує на час видалення курсу. (Необов'язково)
IsSoftDeleted	Булеве значення, яке вказує, чи був курс видалений "м'яким" способом.

Таблиця 3.25 – Таблиця Slides

Атрибут	Опис
Id	Унікальний ідентифікатор слайда.
Name	Назва слайда.

Продовження таблиці 3.25

Markup	Зміст слайда.
TenantId	Зовнішній ключ, що посилається на організацію, до якого належить цей слайд.
CreateDate	Мітка часу створення слайда.
ModifyDate	Мітка часу останньої модифікації слайда.
DeleteDate	Мітка часу видалення слайда.
IsSoftDeleted	Булеве значення, яке вказує, чи був слайд видалений "м'яким" способом.
Order	Порядковий номер слайда.
LectureId	Зовнішній ключ, що посилається на лекцію, до якої належить цей слайд.

Таблиця 3.26 – Таблиця Enrolled

Атрибут	Опис
Id	Унікальний ідентифікатор запису про запис на курс.
UserId	Зовнішній ключ, що посилається на користувача, який записався на курс.
TenantId	Зовнішній ключ, що посилається на орендаря, до якого належить цей запис.
CourseId	Зовнішній ключ, що посилається на курс, на який записався користувач.
CreateDate	Мітка часу створення запису.
ModifyDate	Мітка часу останньої модифікації запису.
DeleteDate	Мітка часу видалення запису.
IsSoftDeleted	Булеве значення, яке вказує, чи був запис видалений "мяким" способом.

Продовження таблиці 3.26

IsBanned	Булеве значення, яке вказує, чи заблокований користувач на цьому курсі.
IsArchived	Булеве значення, яке вказує, чи архівований цей запис.

Таблиця 3.27 – таблиця Invites

Атрибут	Опис
Id	Унікальний ідентифікатор запрошення.
InvitedUserId	Зовнішній ключ, що посилається на користувача, який був запрошений.
InviterId	Зовнішній ключ, що посилається на користувача, який відправив запрошення.
TenantId	Зовнішній ключ, що посилається на організацію, до якого належить це запрошення.
CourseId	Зовнішній ключ, що посилається на курс, до якого стосується це запрошення.
CreateDate	Мітка часу створення запрошення.
ModifyDate	Мітка часу останньої модифікації запрошення.
DeleteDate	Мітка часу видалення запрошення.
IsSoftDeleted	Булеве значення, яке вказує, чи було запрошення видалено "м'яким" способом.

Таблиця 3.28 – Таблиця Mentors

Атрибут	Опис
Id	Унікальний ідентифікатор

Продовження таблиці 3.28

CourseId	Зовнішній ключ, що посилається на курс, до якого належить цей ментор.
UserId	Зовнішній ключ, що посилається на користувача, який є ментором.
CreateDate	Мітка часу створення запису.
DeleteDate	Мітка часу видалення запису.

Тепер опишемо головний сервер нашого застосунку, він запускається у середовищі ASP.NET Core, в загальному випадку саме фреймворк буде управляти роботою коду, в основі його роботи лежать контролери – це спеціальні класи які створюються при умові що URL шлях співпадає з налаштованим для конкретного контролера за допомогою внутрішніх інструментів. Клієнтський код для обробки даних, які потрапляють до контролеру при правильній реалізації, повинен бути наданий у конструктор класу за допомогою ін’єкції залежності, яка також вбудована у фреймворк, таким чином розробку нового функціоналу сервера можна виділити основні етапи:

1. Створення контролера
2. Створення класу для обробки даних, що надійшли в контролер при необхідності
3. Реєстрація створеного класу для обробки даних в DI-контейнер

Після цих дій сервер автоматично зможе знайти контролер, якщо він відповідає хоча б одній з наступних умов:

- Назва класу закінчується на Controller
- Клас наслідує ControllerBase
- На клас встановлений атрибут Controller

Наприклад, якщо необхідно створити контролер для обробки URL-шляху *api/v1/courses* потрібно створити клас *CoursesController*, для обробки даних,

створюємо клас `CoursesService` та реєструємо його в контейнері. Для того щоб отримати екземпляр класу `CoursesService` в конструкторі `CoursesController` додається атрибут який має тип `CoursesService`, якщо необхідні інші залежності то їх необхідно також додати у конструктор.

Більшість операцій по керуванню користувачами виконуються завдяки бібліотеці `Identity Core Framework`, так для створення і редагування прав користувача використовується клас `userManager` з бібліотеки `Identity Core Framework`, основні методи якого описані у таблиці 3.29. Для автентифікації користувача використовується клас `SignInManager`, його методи описані у таблиці 3.30. Для авторизації користувача використовується клас `RoleManager`, його методи описані у таблиці 3.31.

Таблиця 3.29 – Головні методи класу `userManager`

Назва методу	Опис
<code>CreateAsync(IdentityUser user, string password)</code>	Створює нового користувача з вказаним паролем.
<code>DeleteAsync(IdentityUser user)</code>	Видаляє користувача.
<code>FindByIdAsync(string userId)</code>	Знаходить користувача за ID.
<code>FindByNameAsync(string userName)</code>	Знаходить користувача за ім'ям користувача.
<code>UpdateAsync(IdentityUser user)</code>	Оновлює користувача.
<code>AddToRoleAsync(IdentityUser user, string role)</code>	Додає користувача до вказаної ролі.
<code>IsInRoleAsync(IdentityUser user, string role)</code>	Перевіряє, чи належить користувач до вказаної ролі.
<code>RemoveFromRoleAsync(IdentityUser user, string role)</code>	Видаляє користувача з вказаної ролі.
<code>GetRolesAsync(IdentityUser user)</code>	Отримує список ролей, до яких належить користувач.

Продовження таблиці 3.29

CheckPasswordAsync(IdentityUser user, string password)	Перевіряє, чи вказаний пароль вірний для користувача.
--	---

Таблиця 3.30 – Головні методи класу SignInManager

Назва методу	Опис
PasswordSignInAsync(IdentityUser user, string password, bool isPersistent, bool lockoutOnFailure)	Авторизує користувача з вказаним паролем.
SignOutAsync()	Здійснює вихід користувача.
SignInAsync(IdentityUser user, bool isPersistent, string? authenticationMethod = null)	Здійснює вхід користувача без пароля.
IsSignedIn(ClaimsPrincipal principal)	Перевіряє, чи користувач авторизований.
ValidateSecurityStampAsync(ClaimsPrincipal principal)	Перевіряє актуальність печатки безпеки користувача.
CheckPasswordSignInAsync(IdentityUser user, string password, bool lockoutOnFailure)	Перевіряє пароль користувача та авторизує його.
RefreshSignInAsync(IdentityUser user)	Оновлює користувача який вже автентифікований.
GetExternalLoginInfoAsync(string? expectedXsrf = null)	Отримує інформацію про зовнішню авторизацію.
ExternalLoginSignInAsync(string loginProvider, string providerKey, bool isPersistent, bool bypassTwoFactor)	Здійснює вхід користувача через зовнішній сервіс авторизації.
UserManager(IdentityUser user)	Повертає об'єкт UserManager, що використовується цим SignInManager.

Описавши роботу фреймворка та його бібліотек переходимо до опису контролерів та класів для обробки даних які надходять до цих контролерів, опис контролерів для авторизації та автентифікації на мою думку можна опустити оскільки вони складаються з викликів методів які наведені в таблицях 3.29 – 3.30.

Таблиця 3.31 – Опис утиліт

№ п/п	Назва утиліти	Опис застосування
1	IntelliJ Rider	Середовище розробки програмного забезпечення серверної частини.
2	IntelliJ WebStorm	Середовище розробки програмного забезпечення клієнтської частини.
3	IntelliJ DataGrip	Програмне забезпечення яке надає легкий графічний інтерфейс для доступу до бази даних.
4	Insomnia	Використовувалось для тестування API інтерфейсів, та клієнтських запитів.

Тексти програмного коду наведені в окремому документі «Текст програми».

3.4 Аналіз безпеки даних

Для аналізу безпеки необхідно поділити програмне забезпечення на дві частини: серверну та клієнтську.

Серверну частину почнемо описувати зі взаємодією з базою даних. В загальному випадку якщо СУБД має домен, або публічний IP для підключення до неї, то з'єднання повинно відбуватися за протоколом HTTPS, в нашому випадку необхідності прямого підключення до бази даних немає і вся взаємодія відбувається через головний сервер, в нашому випадку достатньо створити приватну мережу та підключатися до БД за через неї. Оскільки прямого доступу до СУБД немає, за виключенням команди, яка підтримує програмне забезпечення, вся взаємодія з СУБД повинна відбуватися за допомогою ORM, тобто за допомогою Entity Framework Core.

Перше про що необхідно подбати – це SQL-ін'єкції [12]. SQL-ін'єкцій – це тип атаки коли у вхідні дані замість правильних даних, передається код на необхідному діалекті SQL, який робить якусь шкідливу дію. При роботі з фреймворком запобігти цим атакам достатньо просто, по перше не можна використовувати функціонал який запускає SQL команди, які не були згенеровані через спеціальні засоби для їх побудови, в ідеальному випадку слід взагалі відмовитись від взаємодії з базою таким чином та використовувати засоби LinqToEntity які не вразливі до таких атак, оскільки на рівні програми взаємодія з базою відбувається в контексті створених об'єктів.

По друге, згідно з рекомендаціями безпеки компанії Microsoft, при створенні API для програмного забезпечення заборонено повертати об'єкт IQueryable для потенційно небезпечних користувачів нашого інтерфейсу. Головна проблема – при поверненні цього об'єкта в тому, що він може використовуватися для побудови запитів, тобто якщо потенційний злоумисник має на меті отримати якісь дані до якого у нього не має доступу від може це зробити за допомогою цього об'єкту наприклад додавши виклик

методу Include. У цій роботі такий API не входить в план розробки, тому цей тип атак не є можливим.

Наступне, необхідно зробити аналіз безпеки при роботі з ASP.Net Core. Найголовніше про що необхідно подбати – це безпека при автентифікації. Процес автентифікації відбувається достатньо тривіально, клієнт надсилає дані для автентифікації, серверна частина перевіряє дані та при успішній перевірці повертає токен. Оскільки автентифікація відбувається за допомогою інструментів Identity, то участь розробника є мінімальною і складається майже повністю з викликів менеджерів фреймворку, але все ж таки необхідно розібратися більш детально у тому як працює бібліотека, для цього розпишемо процес авторизації детальніше. Після того як клієнт відправив запит з даними для перевірки за допомогою об'єкту UserManager шукає користувача, у випадку коли користувач існує в системі необхідно перевірити його пароль який зберігається в СУБД в хешованому вигляді, якщо паролі співпадають, то за допомогою SignInManager, автентифікуємо користувача. В програмному забезпеченні для цієї роботи токен зберігається у куках браузера, звичайно для більш безпечного зберігання куки повинні бути http-only та secure, це означає що до куків не буде доступу через клієнтський JavaScript код і вони можуть бути встановлені лише за допомогою HTTP запитів. Сам токен зашифровується за допомогою інструментів ASP.NET Core та передається на клієнт у зашифрованому вигляді. Також, непоганою практикою буде створення додатково поле в таблиці користувачів з позначкою «Чи були дані скомпрометовані», це поле можна поставити в значення true чи 1 у випадку якщо СУБД була вкрадена, або в цілому дані були скомпрометовані, що в залежності від ситуації може, наприклад, зобов'язати користувача змінити пароль.

Також, варто зазначити, що для зберігання чутливої інформації такої як, наприклад, інформацію про кредитні картки, можна використовувати зовнішні сервіси для керування ключами, або ж власною реалізацією для шифрування даних.

Для клієнтської частини можна виділити основні вимоги до безпеки:

- Не запускати код користувача – такі виклики функцій як eval або додавання html у розмітку компонентів не є безпечними і повинні уникатися.
- З'єднання тільки по HTTPS
- Операції з чутливими даними не повинні виконуватись на стороні клієнта
- Критичні дані (хеші паролів, ключі) не повинні надсилатися до клієнтів.

Висновки до розділу

В цьому розділі були проаналізовані головні моменти на які варто звернути увагу при розробці програмного забезпечення, була надана та описана діаграма загальної архітектури програмного забезпечення та описані деяких сервісів, також були створені таблиці з детальним описом класів, які включають в себе описи властивостей та методів. Був проведений детальний опис структури бази даних та таблиці в яких описана кожна таблиця. Також був проведений аналіз безпеки програмного застосунку, в якому було проаналізовано найбільш вразливі місця як з серверною сторони, так і зі сторони клієнта. Аналіз даних включав в себе такі пункти:

- База даних
 - Підключення
 - Взаємодія
 - SQL-ін'єкції та як їм запобігти
- Сервер
 - Автентифікація
 - Авторизація
 - Генерація токenu
 - Зберігання даних

- Клієнт
 - Взаємодія з сервером
 - Безпека при роботі з даними
 - Безпека при роботі з користувачами

Отже, був проведений комплексний аналіз який включає в себе всі етапи розробки програмного забезпечення

4 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Аналіз якості ПЗ

Метою тестування є наступне:

- перевірка правильності роботи програмного забезпечення відповідно до функціональних вимог;
- перевірка збереження даних;
- перевірка сумісності веб-додатку з останніми версіями сучасних браузерів
- знаходження проблем, помилок і недоліків з метою їх усунення;
- перевірка зручності графічного інтерфейсу.

Метриками для оцінки якості ПЗ обрано наступні:

- швидкість;
- надійність;

4.2 Опис процесів тестування

Тестування виконується згідно документу «Програма та методика тестування».

Було виконане мануальне тестування програмного забезпечення, опис відповідних тестів наведено у таблицях 4.1 – 4.10.

Таблиця 4.1 – Тест 1.1 Аутентифікація користувача

Початковий стан системи	Користувач знаходиться на сторінці входу
Вхідні дані	Логін, пароль
Опис проведення тесту	Користувач знаходиться на сторінці входу в систему. Вводить логін та пароль у відповідні поля. Натискає кнопку увійти в систему.

Продовження таблиці 4.1

Очікуваний результат	Вхід проходить успішно, користувач перенаправляється на сторінку з курсами.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.2 – Тест 1.2 Створення користувача

Початковий стан системи	Адміністратор системи знаходиться в реєстрі користувачів.
Вхідні дані	Дані користувача, такі як, логін, електронна пошта, телефон, організація
Опис проведення тесту	Адміністратор системи знаходиться на сторінці реєстру користувачів. Натискає кнопку додати, яка знаходиться в правому куті екрану. На сторінці, яка відкрилася адміністратор вводить дані користувача та натискає на кнопку зберегти.
Очікуваний результат	З'являється повідомлення, що користувач був успішно створений, на вказану пошту відправляється лист з даними для входу.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.3 – Тест 1.3 Створення організації

Початковий стан системи	Адміністратор системи знаходиться в реєстрі організацій.
Вхідні дані	Дані організації, такі як, назва, електронна пошта, телефон

Продовження таблиці 4.3

Опис проведення тесту	Адміністратор системи знаходиться на сторінці реєстру організацій. Натискає кнопку додати, яка знаходиться в правому куті екрану. На сторінці, яка відкрилася адміністратор вводить дані організації та натискає на кнопку зберегти.
Очікуваний результат	З'являється повідомлення, що організація була успішно створена, на вказану пошту відправляється лист з інформацією що організація була створена в системі.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.4 – Тест 1.4 Створення курсу адміністратором системи

Початковий стан системи	Адміністратор системи знаходиться в реєстрі курсів.
Вхідні дані	Дані курсу, такі як, назва, опис, фото
Опис проведення тесту	Адміністратор системи знаходиться на сторінці реєстру курсів. Натискає кнопку додати, яка знаходиться в правому куті екрану. На сторінці, яка відкрилася адміністратор вводить дані курсу та натискає на кнопку зберегти.
Очікуваний результат	З'являється повідомлення, що курс був успішно створений.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.5 – Тест 1.5 Додавання ментора до курсу адміністратором системи

Початковий стан системи	Адміністратор системи знаходиться на сторінці редагування курсу.
Вхідні дані	
Опис проведення тесту	Адміністратор системи знаходиться на сторінці редагування курсів. Переходить на вкладку ментори. Натискає на кнопку додати, яка знаходиться в правому куті таблиці. З'являється модальне вікно з випадаючим списком користувачів з роллю Mentor. Адміністратор вибирає користувача та натискає на кнопку Ок.
Очікуваний результат	З'являється повідомлення, що ментор був успішно доданий до курсу.
Фактичний результат	Збігається з очікуваним.

Таблиця 4.6 – Тест 1.6 Додавання запрошення до курсу адміністратором системи

Початковий стан системи	Адміністратор системи знаходиться на сторінці редагування курсу.
Вхідні дані	
Опис проведення тесту	Адміністратор системи знаходиться на сторінці редагування курсів. Переходить на вкладку з запрошеннями. Натискає на кнопку додати, яка знаходиться в правому куті таблиці. З'являється модальне вікно з випадаючим списком користувачів організації до якої належить курс. Адміністратор вибирає користувача та натискає на кнопку Ок.

Продовження таблиці 4.6

Очікуваний результат	З'являється повідомлення, що запрошення було успішно відправлено. Користувачу надсилається лист на пошту.
Фактичний результат	Збігається з очікуваним.

Таблиця 4.7 – Тест 1.7 Прийняття запрошення до курсу користувачем

Початковий стан системи	Користувач знаходиться на сторінці з запрошеннями пройти курс.
Вхідні дані	Запрошення до курсу
Опис проведення тесту	Користувач знаходиться на сторінці з запрошеннями пройти курс, та бачить запрошення до курсів. Вибирає потрібний курс та натискає кнопку прийняття запрошення.
Очікуваний результат	Користувач перенаправляється на сторінку опису курсу.
Фактичний результат	Збігається з очікуваним.

Таблиця 4.8 – Тест 1.8 Відповідь на правильне питання

Початковий стан системи	Користувач знаходиться на сторінці зі змістом курсу.
Вхідні дані	
Опис проведення тесту	Користувач знаходиться на сторінці знаходиться на сторінці зі змістом курсу. Користувач натискає на тест, який він хоче пройти. Після цього натискає на кнопку початку тестування. Користувач відповідає на правильну відповідь питання.

Продовження таблиці 4.8

Очікуваний результат	Номер питання підсвічується зеленим кольором.
Фактичний результат	Збігається з очікуваним.

Таблиця 4.9 – Тест 1.9 Відповідь на неправильне питання

Початковий стан системи	Користувач знаходиться на сторінці зі змістом курсу.
Вхідні дані	
Опис проведення тесту	Користувач знаходиться на сторінці знаходиться на сторінці зі змістом курсу. Користувач натискає на тест, який він хоче пройти. Після цього натискає на кнопку початку тестування. Користувач відповідає на неправильну відповідь питання.
Очікуваний результат	Номер питання підсвічується червоним кольором.
Фактичний результат	Збігається з очікуваним.

Таблиця 4.10 – Тест 1.10 Вихід з аккаунту користувача

Початковий стан системи	Користувач знаходиться в системі
Вхідні дані	
Опис проведення тесту	Користувач знаходиться в системі. Натискає на кнопку профілю у правому верхньому куті. Натискає кнопку вийти з системи.
Очікуваний результат	Вихід проходить успішно, користувач перенаправляється на сторінку аутентифікації.

Продовження таблиці 4.10

Фактичний результат	Збігається з очікуванням.
---------------------	---------------------------

4.3 Опис контрольного прикладу

В якості контрольного прикладу розглянемо функціонал створення користувача адміністратором системи.

Перше, що необхідно зробити – це перейти до списку користувачів в панелі адміністратора, як це показано на рисунку 4.1

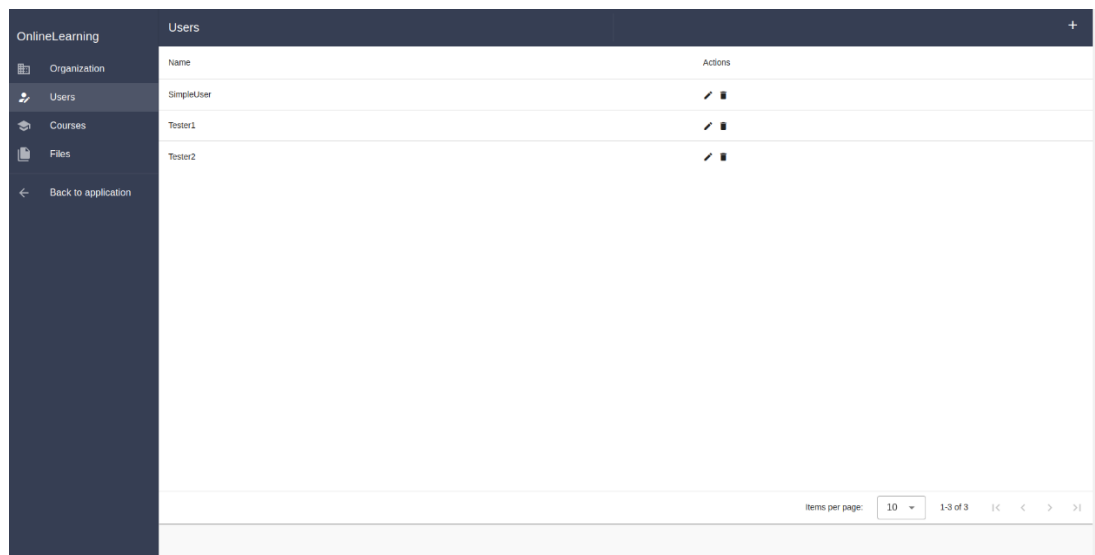


Рисунок 4.1 – Реєстр користувачів

Далі потрібно натиснути на кнопку створення користувача, яка знаходиться в правому верхньому куту. Після цього система перенаправляє адміністратора на форму створення користувача, далі адміністратору необхідно ввести дані користувача. Система повинна перевірити валідність даних, приклад перевірки показано на рисунку 4.2

Рисунок 4.2 – Перевірка при створенні користувача

Після успішного введення даних адміністратор натискає на кнопку збереження даних та користувач отримує повідомлення на вказану електронну адресу, як це показано на рисунках 4.3 та 4.4.

Рисунок 4.3 – Створений користувач



OL Platform з домену amazonses.com
КОМУ МЕНІ ▾

 [Перекласти такою мовою: українська](#) 

Profile added to OLP:
Username: SashaB
Password: 123qweQ*

Рисунок 4.4 – Лист с даними для входу

Висновки до розділу

В цьому розділі було описано тестування системи. Було зазначено мету тестування та описані основні сценарії мануального тестування, такі як:

- 1) Аутентифікація в системі
- 2) Робота з функціоналом курсів
- 3) Основні функції адміністратора системи
- 4) Основні функції адміністратора організації

5 РОЗГОРТАННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

5.1 Розгортання програмного забезпечення

Створений програмний продукт є повноцінним рішенням і надається для використання у вигляді пакета для самостійного розгортання і подальшої підтримки. При цьому користувач самостійно несе відповідальність як за виділення і підтримку апаратного, так і відповідного програмного забезпечення. Рекомендованим підходом є використання віртуального виділеного серверу від довіреного провайдера або особисто підтримуваного апаратного рішення з операційною системою GNU/Linux Debian або похідною від неї. При цьому етапами розгортання є:

- 1) Розгортання системи керування базами даних.** Підтримуваним продуктом рішенням є СКБД PostgreSQL. При цьому може застосовуватися як один апаратний сервер для розгортання всіх модулів продукту, так і кожна архітектурна частина на окремому обчислювальному блоці.

В першому випадку на встановленій системі виконується команда

“apt install postgresql” для встановлення основних пакетів PostgreSQL, після чого виконується *“sudo -u postgres psql -c 'create database lms_main’”*.

В останньому випадку, окрім попередніх команд ще додається налаштування зовнішнього доступу до PostgreSQL шляхом редагування файла конфігурації *postgresql.conf* та додавання *“listen_addresses = ‘*’”* після чого редагується файл *pg_hba.conf* з додаванням *“host all all <Адреса мережі для доступу>/<маска підмережі для доступу> md5”*

- 2) Розгортання головного серверу керування логікою.** Після отримання пакету рішення *lms_<Номер версії>.deb* виконується його встановлення стандартними засобами з використанням команди *“sudo dpkg -i lms_<Номер версії>.deb”* після чого відбувається автоматичне розміщення файлів рішення за директоріями системи, створення

конфігурації за замовчуванням та файлового сховища. Після чого виконується додавання даних для підключення до СКБД з попереднього етапу у файл конфігурації *config.json*.

- 3) **Розгортання проксі-серверу nginx.** Модуль встановлюється виконанням команди “*sudo apt install nginx*” та додаванням налаштувань фаєрволу “*sudo ufw allow 'Nginx HTTP'*” та додаванням можливості надання статичних файлів шляхом редагування конфігу *nginx* з додаванням блоку:

“

```
server {
    root /etc/lms/www/data;

    location / {
    }

    ...
}
```

”

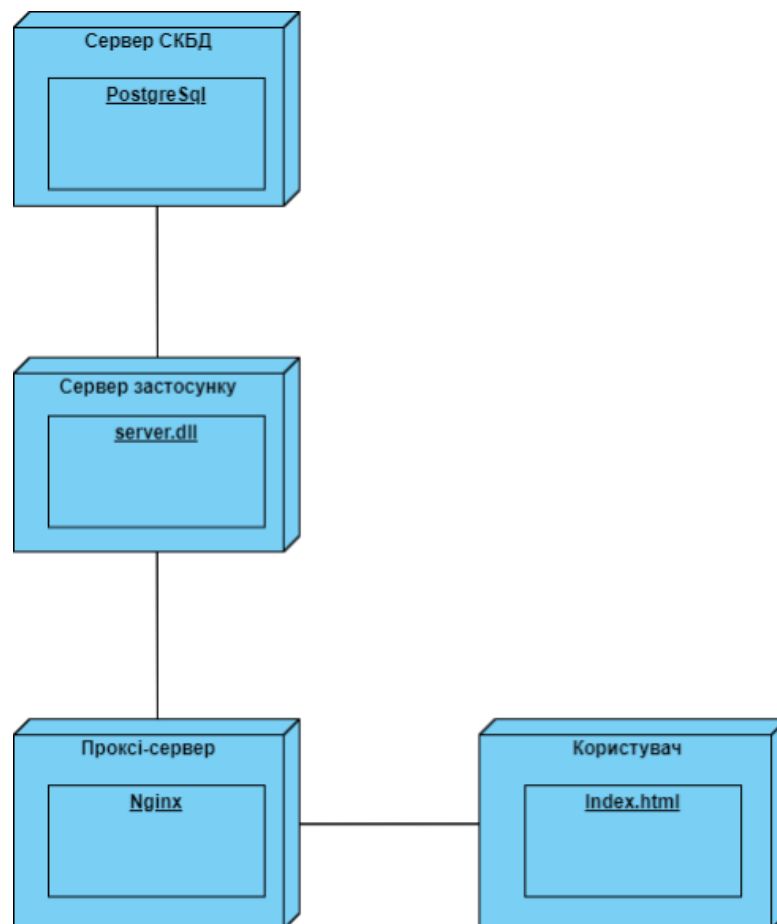


Рисунок 5.1 - Діаграма розгортання

5.2 Супровід програмного забезпечення

Користувачам надається можливість для отримання останньої стабільної версії застосунку для оновлення. При цьому відбувається поширення з використанням пакету DEB, створення якого відбувається автоматично при запуску відповідного завдання з використанням гілки репозиторію з номером версії. Етапами створення та доставки відповідної версії є:

- 1) Постачання нової версії до репозиторію. Коли нова версія застосунку готова для постачання, створюється нова гілка в репозиторії з необхідною версією в назві.
- 2) Автоматизоване завдання запускається, при цьому завантажуються всі необхідні для збірки пакети, такі як NodeJs, NPM, .NET та інші.
- 3) Після налаштування середовища збірки виконується дозавантаження залежностей проекту і після чого відбувається безпосередня збірка фінального виконаного продукту.
- 4) Отримані на попередньому етапі дані формуються в артефакти головного серверу у вигляді DEB пакету та клієнтської частини у вигляді архіву з мінімізованими статичними файлами.

Цей процес автоматизує створення і постачання нових версій застосунку, щоб забезпечити зручність для користувачів і зберегти послідовність та надійність процесу.

При цьому інші частини рішення, такі як СКБД PostgreSQL можуть бути оновлені окремо, про що більш детально повідомляється у файлі, що поставляється разом з кожною версією програмного застосунку.

Висновки до розділу

Підводячи підсумки цього розділу, можна зазначити, що було описано процес та етапи розгортання версії нового застосунку, при цьому було визначено відповідні кроки використовуючи окремі апаратні вузли для кожного з елементів, так і застосування одного глобального.

Розроблений програмний продукт є повноцінним рішенням і готовий до використання у вигляді пакета для самостійного розгортання і подальшої підтримки. Важливо зазначити, що користувачі самостійно несуть відповідальність за апаратну і програмну інфраструктуру при цьому рекомендованим підходом для розгортання програмного продукту є використання віртуального виділеного сервера або особисто підтримуваного апаратного рішення з операційною системою GNU/Linux Debian або похідною від неї.

Етапами розгортання є:

- Розгортання системи керування базами даних PostgreSQL.
- Розгортання головного серверу керування логікою.
- Розгортання проксі-серверу Nginx.

При цьому користувачам надається можливість отримати останню стабільну версію застосунку для оновлення. Цей процес автоматизується і включає в себе поширення через пакет DEB. Сам процес створення і постачання нових версій застосунку автоматизований для зручності користувачів і забезпечення послідовності та надійності процесу.

ВИСНОВКИ

В даній роботі було проаналізовано існуючі рішення систем та застосунків, які дозволяються поширювати навчальні матеріали розроблено систему для поширення навчальних матеріалів та була зроблена порівняльна таблиця, в якій було описані переваги та недоліки цих систем.

Також, було сконструйоване та розроблене програмне забезпечення з детальним описом всіх аспектів розробки. Перше, про що необхідно сказати, це розробка функціональних та нефункціональних вимог, які включають в себе основний функціонал, який програма повинна виконувати. Він включає в себе роботу з курсами, файловим сховищем та веденням користувачів. На основі цих вимог був розроблений клієнт-серверний застосунок, серверна частина якого була написана на мові програмування C#, з використання таких програмних засобів розробки як: ASP.NET Core, Entity Framework, Identity Framework. Клієнтська частина була написана за допомогою фреймворку VueJs з використання бібліотеки графічних компонентів vuetify.

В якості бази даних було використано PostgreSQL, вся взаємодія відбувається за допомогою Entity Framework.

Для взаємодії з файловою системою був написаний модуль, який включає в себе можливість, який включає в себе можливість взаємодіяти з системою за допомогою окремих операцій, так і за допомогою транзакцій, що дозволяє відмінити усі операції що виконувалися у транзакції.

Комунікація між клієнтом та сервером відбувається за допомогою архітектури REST.

Для забезпечення безпеки користувачів при роботі з системою був проведений аналіз безпеки даних, в якому було визначено основні можливості атак на систему та те як їм запобігти, способи безпечного зберігання даних та безпечний обмін даними між клієнтом та сервером.

Після реалізації застосунку він був протестований на екранах з різної розподільною здатністю та з використанням різних браузерів, для того щоб

переконатися, що інтерфейс відображається правильно, а всі функції системи працюють коректно.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

- 1) Learning management system (LMS). URL: <https://www.techtarget.com/searchcio/definition/learning-management-system> (дата звернення 26.02.2024)
- 2) Coursera. URL: <https://about.coursera.org/> (дата звернення 26.02.2024)
- 3) edX Documentation Resources. URL: <https://docs.edx.org/> (дата звернення 26.02.2024)
- 4) Udemy. URL: <https://www.udemy.com/> (дата звернення 26.02.2024)
- 5) Moodle Docs. URL: https://docs.moodle.org/402/en/Main_page (дата звернення 26.02.2024)
- 6) Flutter overview. URL: <https://docs.flutter.dev/resources/architectural-overview> (дата звернення 01.03.2024)
- 7) BASE analysis of NoSQL database. URL: <https://www.sciencedirect.com/science/article/abs/pii/S0167739X15001788>
- 8) Inversion of Control Containers and the Dependency Injection pattern. URL: <https://martinfowler.com/articles/injection.html>
- 9) Mark Seemann, Steven van Deursen. Dependency Injection Principles, Practices, and Patterns 2019; P. 36
- 10) Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. Design Patterns Elements of Reusable Object-Oriented Software; P. 223
- 11) Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. Design Patterns Elements of Reusable Object-Oriented Software; P. 331
- 12) A Classification of SQL Injection Attacks and Countermeasures. URL: <https://sites.cc.gatech.edu/home/orso/papers/halfond.viegas.orso.ISSSE06.pdf>

ДОДАТОК А ЗВІТ ПОДІБНОСТІ



Ім'я користувача:
Лісовиченко Олег Іванович

ID перевірки:
1016322103

Дата перевірки:
05.06.2024 09:00:07 EEST

Тип перевірки:
Doc vs Internet + Library

Дата звіту:
05.06.2024 09:33:05 EEST

ID користувача:
76913

Назва документа: ІП-02_Балащенко_ПЗ
Кількість сторінок: 68 Кількість слів: 10009 Кількість символів: 80652 Розмір файлу: 1.03 MB ID файлу: 1016120532

Виявлено модифікації тексту (можуть впливати на відсоток схожості)

11.5%
Схожість

Найбільша схожість: 3.94% з джерелом з Бібліотеки (ID файлу: 1016106058)

2.97% Джерела з Інтернету	183	Сторінка 70
11.4% Джерела з Бібліотеки	223	Сторінка 71

0% Цитат

- Вилучення цитат вимкнене
- Вилучення списку бібліографічних посилань вимкнене

0%
Вилучень

Немає вилучених джерел

Модифікації

Виявлено модифікації тексту. Детальна інформація доступна в онлайн-звіті.

Замінені символи	14
Підозріле форматування	12 сторінок

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2024 р.

**РОЗПОДІЛЕНА СИСТЕМА АГРЕГАЦІЇ, УПРАВЛІННЯ ТА
ПОШИРЕННЯ НАВЧАЛЬНИХ МАТЕРІАЛІВ**

Текст програми

КПІ.ПІ-8201.045440.03.12

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Микола ХРАМЧЕНКО

Нормоконтроль:

_____ Тетяна ШУЛЬКЕВИЧ

Виконавець:

_____ Олександр БАЛАЦЕНКО

Київ – 2024

Посилання на репозиторій з повним текстом програмного коду
<https://github.com/Whigra1/olp>

Файл UsersController.cs

Реалізація функціональної вимоги створення користувача

```
using System.Text.Json;
using OLPNet.Attributes;
using OLPNet.Controllers.Extensions;
using OLPNet.Controllers.Users.Dtos;
using OLPNet.Database;
using OLPNet.Database.Tables;
using OLPNet.RabbitMq;
using OLPNet.RabbitMq.dto;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace OLPNet.Controllers.Users;

[ApiController]
[Authorize]
[Route("/api/v1/[controller]")]
public class UsersController(DatabaseContext dbContext, UserManager<User> userManager, IRabbitMqContext rabbitMqContext) : ControllerBase
{
    private const string RoutingKey = "notification.mail";

    [HttpPost]
    [SystemAdmin]
    public async Task<IActionResult> AddUser(UserDto dto)
    {
        var pass = "123qweQ*";
        var user = new User
        {
            UserName = dto.Name,
            Email = dto.Email,
            PhoneNumber = dto.Phone,
            EmailConfirmed = true,
            PhoneNumberConfirmed = true,
            TenantUsers = new List<TenantUser>
            {
                new()
                {
                    Description = "Гарний хлопчина",
                    TenantId = dto.TenantId
                }
            }
        };
        var result = await userManager.CreateAsync(user, "123qweQ*");
        if (!result.Succeeded)
            return BadRequest("Error on creating user");
        rabbitMqContext.Publish(RoutingKey, JsonSerializer.SerializeToUtf8Bytes(new NotificationDto
        {
            Recipient = dto.Email,
            Subject = "Account created",
            CorrelationId = Guid.NewGuid().ToString(),
            NotificationType = "Email",
            Message = $"Profile added to OLP:<br>Username: {dto.Name}<br> Password: {pass}"
        })), null);
        return Ok(user.Id);
    }
}
```

Файл UsersController.cs

```

<template>
<v-container class="fill-height" fluid style="align-content: flex-start; padding: 0">
  <v-toolbar color="#374053">
    <v-btn v-if="!userID" @click="save" icon> <v-icon color="green">mdi-content-save</v-icon> </v-btn>
    <v-spacer></v-spacer>
    <v-btn @click="close" icon> <v-icon color="white">mdi-close</v-icon> </v-btn>
  </v-toolbar>
  <v-row style="flex-direction: column;">
    <v-col v-if="saved">
<!--    <v-alert-->
<!--      type="success"-->
<!--      title="Saved"-->
<!--      text="Successfully saved"-->
<!--      closable-->
<!--    />-->
    </v-col>
    <v-col>
      <v-form class="mt-2" style="padding: 16px">
        <v-row justify="space-between">
          <v-col>
            <v-row>
              <v-col>
                <v-text-field
                  v-model="name.value.value"
                  :counter="100"
                  :error-messages="name.errorMessage.value"
                  label="Name"
                  variant="outlined"
                  :readonly="userID"
                />
              </v-col>
            </v-row>
            <v-row class="mt-0">
              <v-col>
                <v-text-field
                  v-model="email.value.value"
                  :counter="100"
                  :error-messages="email.errorMessage.value"
                  label="Email"
                  variant="outlined"
                  :readonly="userID"
                />
              </v-col>
            </v-row>
          </v-col>
        </v-row>
      </v-form>
    </v-col>
  </v-row>
</template>

```

```

<v-col>
  <v-row>
    <v-col>
      <v-text-field
        v-model="phoneNumber.value.value"
        no-resize
        rows="10"
        :error-messages="phoneNumber.errorMessage.value"
        label="Phone"
        variant="outlined"
        :readonly="userID"
      />
    </v-col>
  </v-row>
  <v-row class="mt-0">
    <v-col>
      <v-select
        v-model="organization.value.value"
        :items="organizations"
        item-title="name"
        item-value="id"
        no-resize
        rows="10"
        :error-messages="organization.errorMessage.value"
        label="Organization"
        variant="outlined"
        :readonly="userID"
      />
    </v-col>
  </v-row>
</v-col>
</v-form>
</v-col>
<v-col>
  <v-tabs v-model="tab">
    <v-tab value="roles">Roles</v-tab>
  </v-tabs>
</v-col>
<v-col>
  <roles-table v-if="tab === 'roles'" :user-id="userID"/>
</v-col>
</v-row>
</v-container>
</template>

```

```

<script>
import { ref } from 'vue'
import { useField, useForm } from 'vee-validate'
import RolesTable from "@/components/orgAdmin/users/RolesTable.vue";
export default {
  components: { RolesTable },
  setup() {
    const {handleSubmit, handleReset} = useForm({
      validationSchema: {
        name (value) {
          if (value?.length >= 2) return true
          return 'Name needs to be at least 2 characters.'
        },
        email (value) {
          if (1 === 1) return true
          return 'Name needs to be at least 2 characters.'
        },
        phoneNumber (value) {
          if (1 === 1) return true
          return 'Name needs to be at least 2 characters.'
        },
        organization (value) {
          if (value) return true
          return 'Organization is required'
        }
      },
    })
    const name = useField('name')
    const email = useField('email')
    const phoneNumber = useField('phoneNumber')
    const organization = useField('organization')

    return {name, email, organization, phoneNumber, handleReset}
  },
  data() {
    return {
      saved: false,
      userID: null,
      tab: 'mentors',
      isPublic: false,
      organizations: []
    }
  },
  created() {

```

```

if (this.$route.params.userID) {
  this.userID = this.$route.params.userID
  this.loadUser(this.userID)
}
this.loadOrganizations()
},

methods: {
  async loadOrganizations() {
    const url = new URL(BACKEND_URL + '/Organizations')
    const result = await fetch(url, {
      method: 'GET',
      credentials: 'include',
      headers: new Headers({'content-type': 'application/json'})
    })
    if (result.status !== 200) {
      console.error("Something wrong")
    }

    this.organizations = await result.json()
  },
  async loadUser(userId) {
    let result = await fetch(new URL(BACKEND_URL + '/Users/' + userId), {
      method: 'GET',
      credentials: 'include',
      headers: new Headers({'content-type': 'application/json'})
    })
    if (result.status !== 200) {
      console.error('Something wrong')
      return
    }
    result = await result.json()
    this.name.value.value = result.userName
    this.email.value.value = result.email
    this.phoneNumber.value.value = result.phoneNumber
    this.organization.value.value = result.organization
  },

  async save() {
    if (this.saved) {
      this.saved = false
    }
    let result = await fetch(new URL(BACKEND_URL + '/Users'), {
      method: this.userID ? 'PUT' : 'POST',
      credentials: 'include',
      headers: new Headers({'content-type': 'application/json'}),

```

```

        body: JSON.stringify({
            id: this.userID,
            name: this.name.value.value,
            email: this.email.value.value,
            phone: this.phoneNumber.value.value,
            tenantId: this.organization.value.value
        })
    })
    if (result.status !== 200) {
        console.error("Something wrong")
        return
    }
    this.saved = true
    if (!this.userID) {
        this.userID = await result.json()
        this.$router.push({ name: 'usersEditAdm', params: { userID: this.userID } })
    }
    setTimeout(() => {
        this.saved = false
    }, 10000)
},
close () {
    this.$router.push('/users-adm')
}
}
}
</script>

```

<style scoped>

</style>

Файл **AuthenticationController.cs**

Реалізація функціональної вимоги автентифікації користувача

```

using System.Security.Claims;
using OLPSNet.Controllers.Dtos;
using OLPSNet.Database;
using OLPSNet.Database.Tables;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

```

```

namespace OLPSNet.Controllers.Authentication;

```

```

[ApiController]

```

```

[Route("[controller]")]

```



```

public class AuthenticationController : ControllerBase
{
    private readonly UserManager<User> _userManager;
    private readonly SignInManager<User> _signInManager;
    private readonly DatabaseContext _databaseContext;

    public AuthenticationController(
        UserManager<User> userManager,
        SignInManager<User> signInManager,
        DatabaseContext databaseContext
    )
    {
        _userManager = userManager;
        _signInManager = signInManager;
        _databaseContext = databaseContext;
    }

    [HttpPost("auth")]
    public async Task<IActionResult> Authenticate(AuthenticationDto authenticationDto)
    {
        return await Authenticate(authenticationDto, false);
    }

    private async Task<IActionResult> Authenticate(AuthenticationDto authenticationDto, bool skipPassCheck = false)
    {
        var user = await _userManager.FindByNameAsync(authenticationDto.UserName);
        if (user is null)
            return Unauthorized();

        if (!skipPassCheck)
        {
            var passCheckResult = await _signInManager.CheckPasswordSignInAsync(user, authenticationDto.Password, false);
            if (!passCheckResult.Succeeded)
                return Unauthorized();
        }

        var tenant = await _databaseContext.Tenants.FirstOrDefaultAsync(u => u.Tenant == user.Tenant);

        var tenantUser = await _databaseContext.TenantUsers
            .Include(t => t.PlatformRoles)
            .FirstAsync(t => t.User == user);

        await _signInManager.SignInWithClaimsAsync(user, true, new []
    {

```

```

        new Claim("TenantId", tenant.Id.ToString()),
        new Claim("IsAdmin", (tenantUser.PlatformRoles.FirstOrDefault(pR => pR.Code == "OrgAdmin") != null).ToString()),
        new Claim("IsMentor", (tenantUser.PlatformRoles.FirstOrDefault(pR => pR.Code == "Mentor") != null).ToString()),
        new Claim("IsSystemAdmin", (tenantUser.PlatformRoles.FirstOrDefault(pR => pR.Code == "SystemAdmin") !=
null).ToString())
    });
    return Ok();
}

await _signInManager.SignOutAsync();
return Ok();
}
}

```

Файл LoginView.vue

```

<template>
  <div class="d-flex align-center justify-center" style="height: 100vh">
    <credentials-card @submit="tryLogin" />
  </div>
</template>

<script>
import { useUserInfoStore } from '@stores/userInfo'
import CredentialsCard from "@components/CredentialsCard.vue";
export default {
  name: "LoginView",
  components: {CredentialsCard},
  data() {
  },
  methods: {
    async tryLogin(login, password) {
      let res = await fetch(new URL(BACKEND_URL + 'Authentication/auth'), {
        method: 'POST',
        credentials: 'include',
        headers: new Headers({'content-type': 'application/json'}),
        body: JSON.stringify({
          userName: login,
          password: password
        })
      })

      if (res.status !== 200) {
        console.error("Something wrong")
        return
      }

      res = await fetch(new URL(BACKEND_URL + '/Users/info'), {
        method: 'GET',

```

```

    credentials: 'include',
    headers: new Headers({'content-type': 'application/json'}),
  })

  if (res.status === 200) {
    const store = useUserInfoStore()
    store.setInfo(await res.json())
    this.$router.push(store.info.isAdmin ? '/course-admin' : '/courses')
    return
  }

  this.$router.push('/courses')
}
}
}
</script>

```

```
<style scoped>
```

```
</style>
```

Файл CredentialsCard.vue

```

<template>
  <v-card style="border-radius: 15px; padding: 2% 1%" width="500" elevation="4">
    <v-card-title>
      Enter credentials
    </v-card-title>
    <v-card-item>
      <v-form fast-fail @submit.prevent="$emit('submit', login, password)">
        <v-row class="mt-1">
          <v-col>
            <v-text-field variant="outlined" v-model="login" label="Login"/>
          </v-col>
        </v-row>
        <v-row>
          <v-col>
            <v-text-field type="password" variant="outlined" v-model="password" label="Password"/>
          </v-col>
        </v-row>
        <v-btn type="submit" color="primary" block class="mt-2">Sign in</v-btn>
      </v-form>
    </v-card-item>
  </v-card>
</template>

<script>

```

```

export default {
  name: "CredentialsCard",
  data() {
    return {
      login: "",
      password: ""
    }
  }
}
</script>

```

```
<style scoped>
```

```
</style>
```

Реалізація функціональної вимоги вихід користувача з системи

Файл **AuthenticationController.cs**

```

using System.Security.Claims;
using OLPNet.Controllers.Dtos;
using OLPNet.Database;
using OLPNet.Database.Tables;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace OLPNet.Controllers.Authentication;

[ApiController]
[Route("[controller]")]
public class AuthenticationController : ControllerBase
{
    private readonly UserManager<User> _userManager;
    private readonly SignInManager<User> _signInManager;
    private readonly DatabaseContext _databaseContext;

    public AuthenticationController(
        UserManager<User> userManager,
        SignInManager<User> signInManager,
        DatabaseContext databaseContext
    )
    {
        _userManager = userManager;
        _signInManager = signInManager;
        _databaseContext = databaseContext;
    }

    [HttpPost("logout")]
    public async Task<IActionResult> Logout()
    {
        if (HttpContext.User.Identity is null)
            return Unauthorized();

        await _signInManager.SignOutAsync();
        return Ok();
    }
}

```

Файл **UserMenu.vue**

```
<template>
```

```

<v-menu
  bottom
  min-width="300px"
  rounded
  offset-y
>
<template v-slot:activator="{ on, props }">
  <v-btn
    icon
    x-large
    v-on="on"
    v-bind="props"
    style="margin: 0 16px"
  >
    <v-avatar
      color="brown"
      size="48"
    >
      <span class="white--text text-h5">{{ (name || 'T')[0] }}</span>
    </v-avatar>
  </v-btn>
</template>
<v-list>
  <v-list-item to="/profile">
    <v-list-item-title>
      <v-icon>bi bi-person</v-icon>
      Profile
    </v-list-item-title>
  </v-list-item>
  <v-divider></v-divider>
  <v-list-item
    v-if="isAdmin"
    to="/course-adm">
    <v-list-item-title>
      <v-icon>bi bi-gear</v-icon>
      Administration
    </v-list-item-title>
  </v-list-item>
  <v-divider></v-divider>
  <v-list-item @click="logout">
    <v-list-item-title>
      <v-icon>bi bi-arrow-bar-left</v-icon>
      Logout
    </v-list-item-title>
  </v-list-item>
</v-list>

```

```

    </v-menu>
</template>
<script>
export default = {
  methods: {
    async logout () {
      const userService = new UserService()
      const successfull = await userService.logout()
      if (successfull) {
        this.$router.push('/login')
      }
    }
  }
}
</script>

```

Файл UserService.js

```

export default class UserService {
  async logout() {
    try {
      const resp = await fetch(new URL(BACKEND_URL + '/logout'), {
        method: 'POST',
        credentials: 'include',
        headers: new Headers({'content-type': 'application/json'}),
      })
      return resp.status === 200
    } catch (e) {
      return false
    }
  }
}

```

Функціональна вимога редагування профілю

Файл ProfilePage.vue

```

<template>
<v-container fluid style="background-color: #F8F9FA; align-content: start; padding: 0">
  <v-row align="start" class="mt-1" style="">
    <v-col class="ml-16" cols="auto">
      <v-avatar
        color="brown"
        size="70"
      >
      <span class="white--text text-h4">N</span>
    </v-avatar>
    </v-col>
    <v-col cols="auto" class="userdata" style="color: black">
      <v-row>
        <h2 class="username">
          {{ userName }}
        </h2>
      </v-row>
      <v-row> {{ email }} </v-row>
      <v-row> {{ "Organization" }} </v-row>
    </v-col>
  </v-row>
</template>

```

```

<v-col cols="auto" style="margin-top: auto; margin-left: 5%">
  <v-btn
    class="mb-4"
    color="primary"
    rounded
    @click="isEditMode = !isEditMode"
  >
    <v-icon small color="primary">mdi-pencil-box-outline</v-icon>
    <span style="color: white">Edit Profile</span>
  </v-btn>
</v-col>
</v-row>
<v-col class="ml-16 mr-16">
  <h2> Edit your profile</h2>
  <v-row no-gutters class="mb-6">
    <v-col cols="12">
      <v-divider />
    </v-col>
  </v-row>
  <v-sheet
    width="100%"
    class="fw-data-sheet profile"
  >
    <profile-block title="General">
      <v-form v-model="valid" ref="form">
        <v-row>
          <v-col cols="4">
            <div class="mt-1">Your Name</div>
            <v-text-field
              :disabled="!isEditMode"
              v-model="localUserName"
              dense
              variant="outlined"
              hide-details="auto"
              maxlength="60"
            />
            <div class="mt-2">Your E-mail</div>
            <v-text-field
              :disabled="!isEditMode"
              v-model="localEmail"
              dense
              variant="outlined"
              hide-details="auto"
            />
            <v-btn
              v-if="isEditMode"
              class="mt-5 mb-5 fw-button"
              color="#025870"
              rounded
              @click="updateUserData"
            >
              <v-icon small color="primary">mdi-reload</v-icon>
              <span style="color: white">Update</span>
            </v-btn>
          </v-col>
          <v-col cols="4">
            <div class="mt-1">Your Company</div>
            <v-text-field
              :disabled="!isEditMode"
              value="SportBrand.com"
              dense
              variant="outlined"
              hide-details="auto"
              maxlength="60"
            />
          </v-col>
        </v-row>
      </v-form>
    </profile-block>
  </v-sheet>
</v-col>

```

```

<profile-block title="Password Update">
  <v-form v-model="passwordFormValid" ref="passwordForm">
    <v-row>
      <v-col cols="4">
        <div class="mt-1">Current Password</div>
        <v-text-field
          :disabled="!isEditMode"
          v-model="currentPassword"
          dense
          variant="outlined"
          hide-details="auto"
        />
        <div class="mt-2">New Password</div>
        <v-text-field
          :disabled="!isEditMode"
          v-model="newPassword"
          :append-icon="show1 ? 'mdi-eye' : 'mdi-eye-off'"
          :type="show1 ? 'text' : 'password'"
          @click:append="show1 = !show1"
          @change="validatePasswords"
          dense
          variant="outlined"
          hide-details="auto"
        />
        <div class="mt-2">Repeat Password</div>
        <v-text-field
          :disabled="!isEditMode"
          v-model="repeatedPassword"
          :append-icon="show2 ? 'mdi-eye' : 'mdi-eye-off'"
          :type="show2 ? 'text' : 'password'"
          @click:append="show2 = !show2"
          @change="validatePasswords"
          dense
          variant="outlined"
          hide-details="auto"
        />
        <v-btn
          v-if="isEditMode"
          class="mt-5 mb-4 fw-button"
          color="primary"
          rounded
          @click="updateUserPassword"
        >
          <v-icon small color="white">mdi-reload</v-icon>
          <span style="color: white">Update</span>
        </v-btn>
      </v-col>
    </v-row>
  </v-form>
</profile-block>
</v-sheet>
</v-col>
</v-container>
</template>

```

```

<script>
import ProfileBlock from "./ProfileBlock.vue";
import {useUserInfoStore} from "@stores/userInfo";
const _store = useUserInfoStore()
export default {
  name: "ProfilePage",
  components: {
    ProfileBlock
  },
  data() {
    return {
      valid: true,
      passwordFormValid: false,
      localUserName: "",

```



```

    localEmail: "",
    localPhoneNumber: "",
    localCountry: "",
    localFullName: "",
    currentPassword: "",
    newPassword: "",
    repeatedPassword: "",
    show1: false,
    show2: false,
    isEditMode: false
  }
},

mounted() {
  this.localUserName = this.store.info.userName
  this.localEmail = this.store.info.email
  this.localPhoneNumber = this.store.info.phoneNumber
},

computed: {
  email() {
    return this.store.info.email
  },

  userName() {
    return this.store.info.userName
  },

  phoneNumber() {
    return this.store.info.phoneNumber
  },

  store() {
    return _store
  },
  isDirty() {
    return this.userName !== this.userName ||
      this.localEmail !== this.email ||
      this.localPhoneNumber !== this.phoneNumber ||
      this.localCountry !== this.country ||
      this.localFullName !== this.fullName
  }
},

methods: {
  async updateUserData() {
    if (!this.isDirty)
      return

    this.$refs.form.validate()

    if (!this.valid) return

    const dto = {
      userName: this.localUserName,
      fullName: this.localFullName,
      email: this.localEmail,
      phoneNumber: this.localPhoneNumber,
      country: this.localCountry
    }

    this.isEditMode = false
  },

  async updateUserPassword() {
    this.$refs.passwordForm.validate()
    if (!this.passwordFormValid) {
      return
    }
  }
}

```

```

    if (!this.currentPassword || !this.newPassword || !this.repeatedPassword) {
      throw new Error('required fields are not filled')
    }

    if (this.newPassword !== this.repeatedPassword) {
      throw new Error('New password and the repetition of the new password must match')
    }

    const dto = {
      currentPassword: this.currentPassword,
      newPassword: this.newPassword
    }

    await this.$store.dispatch('user/updateUserPassword', dto)

    this.currentPassword = ''
    this.newPassword = ''
    this.repeatedPassword = ''

    this.isEditMode = false
  },

  validatePasswords() {
    this.$refs.passwordForm.validate()
  }
}
</script>

<style scoped>
.username {
  color: black;
  margin-top: 5%;
}
.userdata {
  padding-left: 1%;
  color: #A8DFEE;
}

.profile {
  margin-top: 1%;
  margin-bottom: 1%;
}
</style>

```

Файл PageBlock.vue

```

<template>
  <v-col class="mt-4 mb-4">
    <v-row class="profile-block-title">
      <h2 class="profile-block-title-header">
        {{ title }}
      </h2>
    </v-row>
  </v-col>
</template>

<script>
export default {
  name: "ProfileBlock",

```

```

    props: {
      title: {
        type: String
      }
    }
  }
}
</script>

```

```

<style scoped>
.profile-block-title {
  background: #F8F9FA;
  height: 60px;
  align-content: center;
}
.profile-block-title-header {
  margin-left: 25px;
  color: #02495D;
}
</style>

```

Файл UsersController.cs

```

using System.Text.Json;
using OLPNet.Attributes;
using OLPNet.Controllers.Extensions;
using OLPNet.Controllers.Users.Dtos;
using OLPNet.Database;
using OLPNet.Database.Tables;
using OLPNet.RabbitMq;
using OLPNet.RabbitMq.dto;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace OLPNet.Controllers.Users;

[ApiController]
[Authorize]
[Route("/api/v1/[controller]")]
public class UsersController(DatabaseContext dbContext, UserManager<User> userManager, IRabbitMqContext rabbitMqContext) : ControllerBase
{
    private const string RoutingKey = "notification.mail";

    [HttpPost]
    public async Task<ActionResult> UpdateUserData(UserDataDto userData)
    {
        var userId = HttpContext.User.Claims.First(c => c.Type == ClaimTypes.NameIdentifier).Value;
        var user = await _userManager.FindByIdAsync(userId);
        if (user is null)
            return NotFound("User not found or not accessible");

        var authorizationResult = await _authorizationService
            .AuthorizeAsync(User, user, CrudOperationAuthorizationRequirements.Update);
    }
}

```

```

    if (!authorizationResult.Succeeded)
        return NotFound("User not found or not accessible");

    if (userData.PhoneNumber is not null && userData.PhoneNumber != user.PhoneNumber)
        user.PhoneNumber = userData.PhoneNumber;

    if (userData.FullName is not null && userData.FullName != user.FullName)
        user.FullName = userData.FullName;

    if (userData.Country is not null && userData.Country != Enum.GetName(typeof(Country), user.Country))
        user.Country = (Country)Enum.Parse(typeof(Country), userData.Country);

    var result = await _userManager.UpdateAsync(user);
    return result.Succeeded
        ? Ok()
        : StatusCode(500);
}
}

```

Реалізація функціональної вимоги управління курсами

Файл CourseEditView.vue

```

<template>
  <v-container fluid>
    <v-tabs
      v-model="tab"
      color="deep-purple-accent-4"
      align-tabs="start"
    >
      <v-tab :value="1">General</v-tab>
      <v-tab :disabled="!courseID" :value="2">Sections</v-tab>
      <v-tab :disabled="!courseID" :value="3">Lectons</v-tab>
      <v-tab :disabled="!courseID" :value="4">Tests</v-tab>
    </v-tabs>
    <v-window v-model="tab">
      <v-window-item
        :value="1"
      >
        <v-container fluid>
          <v-row>
            <v-col>
              <v-card style="padding: 2%; border-radius: 15px">
                <v-form>
                  <v-row>
                    <v-col>
                      <v-text-field
                        v-model="name"

```

```

        :counter="100"
        :rules="[required]"
        label="Name"
        variant="outlined"
      />
    </v-col>
  </v-row>
  <v-row>
    <v-col>
      <v-textarea
        v-model="description"
        :no-resize="true"
        :counter="500"
        :rules="[required]"
        rows="10"
        label="Description"
        variant="outlined"
      />
    </v-col>
  </v-row>
  <v-row align="center" justify="center">
    <v-col>
      <v-checkbox v-model="isPublic" label="Is public"/>
    </v-col>
    <v-col>
      <v-btn @click="visible=true" :disabled="!courseID">{{ photo || 'Choose photo' }}</v-btn>
    </v-col>
  </v-row>
</v-form>

</v-card>
</v-col>
</v-row>
</v-container>
</v-window-item>
<v-window-item :value="2">
  <sections-editor ref="sectionEditor" :course-id="courseID"/>
</v-window-item>
<v-window-item :value="3">
  <v-btn
    rounded
    flat="1"
    color="#f9f9f9"
    :to="/lection-edit/" + courseID"
  >
  <v-icon small>mdi-plus</v-icon>

```

```

    <span>Create lection</span>
  </v-btn>
</v-divider class="mt-2 mb-2"/>
<v-row>
  <v-col
    v-for="(lection, index) in lections"
    :key="index"
    cols="auto"
  >
    <base-card
      :title="lection.name"
      img-src="/logo.svg"
      :button="{ label: 'Open', clickTo: `/lection-edit/${courseID} || `}/${lection.id} || ` }`"
    />
  </v-col>
</v-row>
</v-window-item>
<v-window-item :value="4">
  <v-container fluid>
    <v-btn
      rounded
      flat="1"
      color="#f9f9f9"
      :to="/test-edit/" + courseID
    >
      <v-icon small>mdi-plus</v-icon>
      <span>Create test</span>
    </v-btn>
    <v-divider class="mt-2 mb-2"/>
    <v-row>
      <v-col
        v-for="(test, index) in tests"
        :key="index"
        cols="auto"
      >
        <base-card
          :title="test.name"
          img-src="/logo.svg"
          :button="{ label: 'Open', clickTo: `/test-edit/${courseID} || `}/${test.id} || ` }`"
        />
      </v-col>
    </v-row>
  </v-container>
</v-window-item>
<v-bottom-navigation>
  <v-spacer />

```

```

    <v-btn
      rounded style="color:white; background-color: #0d47a1"
      @click="save"
    >
      <v-icon>mdi-content-save</v-icon>
      <span>Save changes</span>
    </v-btn>
  </v-bottom-navigation>
</v-window>
<v-dialog
  v-model="visible"
  width="50%"
>
  <v-card>
    <file-manager @file:click="onFileClick"/>
  </v-card>
</v-dialog>
</v-container>
</template>

<script>
import FileManager from "@/components/files/FileManager.vue";
import BaseCard from "@/components/cards/BaseCard.vue";
import SlideEditor from "@/components/slides/SlideEditor.vue";
import SectionsEditor from "@/components/user/courses/SectionsEditor.vue";

export default {
  components: {SectionsEditor, SlideEditor, BaseCard, FileManager},
  data() {
    return {
      visible: false,
      tab: 1,
      name: "",
      description: "",
      isPublic: false,
      photo: "",
      courseID: null,
      tests: [],
      lections: [],
    }
  },
  created() {
    if (this.$route.params.courseID) {
      this.courseID = this.$route.params.courseID
      this.loadCourse(this.courseID)
    }
  }
}

```

```

    },
    mounted() {
        // this.courseID = this.$route
    },
    methods: {
        onFileClick(file) {
            this.photo = file.path
            this.visible = false
        },
        async loadCourse(courseID) {
            let result = await fetch(new URL(BACKEND_URL + '/Courses/' + courseID), {
                method: 'GET',
                credentials: 'include',
                headers: new Headers({'content-type': 'application/json'})
            })
            if (result.status !== 200) {
                console.error('Something wrong')
                return
            }
            result = await result.json()
            this.name = result.name
            this.description = result.description
            this.isPublic = result.isPublic
            this.photo = result.photo
            this.tests = result.tests
            this.lections = result.lections
        },

        required (v) {
            return !!v || 'Field is required'
        },
        async save() {
            if (this.saved) {
                this.saved = false
            }

            let result = await fetch(new URL(BACKEND_URL + '/Courses'), {
                method: this.courseID ? 'PUT' : 'POST',
                credentials: 'include',
                headers: new Headers({'content-type': 'application/json'}),
                body: JSON.stringify({
                    id: this.courseID,
                    name: this.name,
                    description: this.description,
                    photo: this.photo,

```



```

        isPublic: this.isPublic
      })
    })
    if (result.status !== 200) {
      console.error("Something wrong")
      return
    }
    this.saved = true
    this.courseID = await result.json()
    setTimeout(() => {
      this.saved = false
      this.$router.push({ name: 'courseEdit', params: { courseID: this.courseID } })
    }, 1000)
  },
}
}
</script>

```

```
<style scoped>
```

```
</style>
```

Файл LectionEdit.vue

```

<template>
<v-container fluid="1" >
  <v-row>
    <v-col>
      <v-breadcrumbs :items="navItems" />
    </v-col>
  </v-row>
  <v-tabs
    v-model="tab"
    color="deep-purple-accent-4"
    align-tabs="start"
  >
    <v-tab :value="1">General</v-tab>
    <v-tab :disabled="!lectureID" :value="2">Slides</v-tab>
  </v-tabs>
  <v-window v-model="tab">
    <v-window-item :value="1">
      <v-row>
        <v-col>
          <v-card style="padding: 2%; border-radius: 15px">
            <v-form>
              <v-row>
                <v-col>

```

```

    <v-text-field
      v-model="name"
      :counter="100"
      :rules="['required']"
      label="Name"
      variant="outlined"
    />
  </v-col>
</v-row>
<v-row>
  <v-col>
    <v-textarea
      v-model="description"
      :no-resize="true"
      :counter="500"
      :rules="['required']"
      rows="10"
      label="Description"
      variant="outlined"
    />
  </v-col>
</v-row>
</v-form>
</v-card>
</v-col>
</v-row>
</v-window-item>
<v-window-item :value="2">
  <slides-table :in-dialog="true" :lection-id="lectionID"/>
</v-window-item>
</v-window>
<v-bottom-navigation>
  <v-spacer />
  <v-btn
    rounded style="color:white; background-color: #0d47a1"
    @click="save"
  >
    <v-icon>mdi-content-save</v-icon>
    <span>Save changes</span>
  </v-btn>
</v-bottom-navigation>
</v-container>
</template>

<script>
import SlidesTable from "@components/orgAdmin/courses/SlidesTable.vue";

```

```

class AddQuestion {
  #questionInd
  constructor({ questions, question }) {
    this.questions = questions
    this.#questionInd = null
    this.question = question ?? {
      text: "",
      edit: true,
      answers: []
    }
  }
  do () {
    this.questions.push(this.question)
    this.#questionInd = this.questions.length - 1
  }

  undo() {
    new RemoveQuestion({
      questions: this.questions,
      index: this.#questionInd
    }).do()
  }
}

class RemoveQuestion {
  constructor({ questions, index }) {
    this.questions = questions
    this.index = index
    this.removed = null
  }
  #remove() {
    return this.questions.splice(this.index, 1)
  }
  do() {
    this.removed = this.questions[this.index]
    this.#remove()
  }
  undo() {
    new AddQuestion({
      question: this.removed,
      questions: this.questions
    }).do()
  }
}

class AddAnswer {
  constructor({ question, answer }) {

```

```

    this.question = question
    this.answer = answer || {
      text: "",
      edit: true
    }
  }
}
do() {
  this.question.answers.push(this.answer)
  this.currentInd = this.question.answers.length - 1
}

undo() {
  new RemoveAnswer({ question: this.question, index: this.currentInd }).do()
}
}

class RemoveAnswer {
  constructor({question, index}) {
    this.question = question
    this.index = index
    this.removed = null
  }
  #remove() {
    return this.question.answers.splice(this.index, 1)
  }
  do() {
    this.removed = this.question.answers[this.index]
    this.#remove()
  }
  undo() {
    new AddAnswer({
      question: this.question,
      answer: this.removed
    }).do()
  }
}

class EditField {
  constructor({ field }) {
    this.field = field
    this.originalVal = null
  }
  do() {
    this.originalVal = JSON.parse(JSON.stringify(this.field.text))
  }
  undo() {
    this.field.text = this.originalVal
  }
}

```

```

}

export default {
  components: {SlidesTable},
  computed: {
    disableAdding () {
      const cmd = this.stack[this.stack.length - 1]
      return cmd instanceof EditField && cmd.field.edit
    }
  },
  data() {
    return {
      navItems: [],
      stack: [],
      questions: [],
      tab: 1,
      name: "",
      description: "",
      lectionID: null
    }
  },
  created() {
    if (this.$route.params.lectionID) {
      this.lectionID = this.$route.params.lectionID
      this.loadLecture(this.lectionID)
    }
    this.navItems.push(
      {
        title: 'Organization courses',
        disabled: false,
        href: '/organization-courses'
      },
      {
        title: 'Course edit',
        disabled: false,
        href: '/course-edit/' + this.$route.params.courseID
      }
    )
  },
  methods: {
    /**
     *
     * @param obj.text - text
     * @param obj.edit -
     */
    edit(obj) {

```

```

const cmd = new EditField({ field: obj })
cmd.do()
obj.edit = true
this.stack.push(cmd)
},
remove(item, index) {
  this.questions.splice(index, 1)
},
addAnswer (question) {
  let cmd = new AddAnswer({ question })
  cmd.do()
  this.stack.push(cmd)
  cmd = new EditField({ field: cmd.question.answers[cmd.question.answers.length - 1] })
  cmd.do()
  this.stack.push(cmd)
},
addQuestion () {
  let cmd = new AddQuestion({ questions: this.questions })
  cmd.do()
  this.stack.push(cmd)
  cmd = new EditField({ field: this.questions[this.questions.length - 1] })
  cmd.do()
  this.stack.push(cmd)
},
onIconSaveClick(obj) {
  if (!obj.text) return
  obj.edit=false
},
onIconCancelClick (obj, type, question) {
  const cmd = this.stack.findLast(c => c.field === obj)
  cmd.undo()
  obj.edit = false
  if (cmd.field.text) {
    return
  }
  switch (type) {
    case 'q':
      new RemoveQuestion({
        questions: this.questions,
        index: this.questions.findIndex(q => q === obj)
      }).do()
      break
    case 'a':
      new RemoveAnswer({
        question,
        index: question.answers.findIndex(a => a === obj)
      }).do()
  }
}

```

```

    }).do()
  }
},

removeAnswer (question, answer) {
  const cmd = new RemoveAnswer({
    question,
    index: question.answers.findIndex(a => a === answer)
  })
  cmd.do()
  this.stack.push(cmd)
},

async loadSlides() {

},

async loadLecture(lectureID){
  let result = await fetch(new URL(BACKEND_URL + '/Lectures/' + lectureID), {
    method: 'GET',
    credentials: 'include',
    headers: new Headers({'content-type': 'application/json'})
  })
  if (result.status !== 200) {
    console.error('Something wrong')
    return
  }
  result = await result.json()
  this.name = result.name
  this.description = result.description
  this.slides = result.slides
},
required (v) {
  return !!v || 'Field is required'
},

async save() {
  if (this.saved) {
    this.saved = false
  }
  let result = await fetch(new URL(BACKEND_URL + '/Lectures'), {
    method: this.lectureID ? 'PUT' : 'POST',
    credentials: 'include',
    headers: new Headers({'content-type': 'application/json'}),
    body: JSON.stringify({
      id: this.lectureID,
      courseID: this.$route.params.courseID,

```

```

      name: this.name,
      description: this.description
    })
  })
  if (result.status !== 200) {
    console.error("Something wrong")
    return
  }
  if (!this.lectionID) {
    this.lectionID = await result.json()
  }
  this.saved = true
  setTimeout(() => {
    this.saved = false
  }, 10000)

},
_create() {
  const url = new URL(BACKEND_URL + '/Tests')

}
}
}
</script>

```

```
<style scoped>
```

```
</style>
```

Файл TestEditView.vue

```

<template>
  <v-container fluid="1" >
    <v-row>
      <v-col>
        <v-breadcrumbs :items="navItems" />
      </v-col>
    </v-row>
    <v-tabs
      v-model="tab"
      color="deep-purple-accent-4"
      align-tabs="start"
    >
      <v-tab :value="1">General</v-tab>
      <v-tab :disabled="!testID" :value="2">Questions</v-tab>
    </v-tabs>
  </v-container>
</template>

```



```

</v-tabs>
<v-window v-model="tab">
  <v-window-item :value="1">
    <v-row>
      <v-col>
        <v-card style="padding: 2%; border-radius: 15px">
          <v-form>
            <v-row>
              <v-col>
                <v-text-field
                  v-model="name"
                  :counter="100"
                  :rules="['required']"
                  label="Name"
                  variant="outlined"
                />
              </v-col>
            </v-row>
            <v-row>
              <v-col>
                <v-textarea
                  v-model="description"
                  :no-resize="true"
                  :counter="500"
                  :rules="['required']"
                  rows="10"
                  label="Description"
                  variant="outlined"
                />
              </v-col>
            </v-row>
          </v-form>
        </v-card>
      </v-col>
    </v-row>
  </v-window-item>
  <v-window-item :value="2">
    <v-col>
      <draggable
        @start="true"
        @end="drag = false"
        :list="questions"
      >
        <template #item="{ element: item }">
          <v-row>
            <v-col>

```

```

      <question
        :stack="stack"
        :disable-adding="disableAdding"
        :question="item"
        @remove="removeQuestion"
      />
    </v-col>

  </v-row>
</template>
</draggable>

<v-row>
  <v-col style="padding: 2%">
    <v-btn :disabled="disableAdding" @click="addQuestion">Add question</v-btn>
  </v-col>
</v-row>
</v-col>
</v-window-item>
</v-window>
<v-bottom-navigation>
  <v-spacer />
  <v-btn
    rounded style="color:white; background-color: #0d47a1"
    @click="save"
  >
    <v-icon>mdi-content-save</v-icon>
    <span>Save changes</span>
  </v-btn>
</v-bottom-navigation>
</v-container>
</template>

<script>
import draggable from 'vuedraggable'
import {EditField} from "@/components/user/courses/commands/tests/EditField";
import {AddQuestion} from "@/components/user/courses/commands/tests/AddQuestion";
import {RemoveQuestion} from "@/components/user/courses/commands/tests/RemoveQuestion";
import Question from "@/components/user/courses/Question.vue";

export default {
  components: {
    Question,
    draggable
  },

```

```

computed: {
  disableAdding () {
    const cmd = this.stack[this.stack.length - 1]
    return cmd instanceof EditField && cmd.field.edit
  }
},
data() {
  return {
    navItems: [],
    stack: [],
    questions: [],
    tab: 1,
    name: "",
    description: "",
    testID: null,
    drag: false
  }
},
created() {
  if (this.$route.params.testID) {
    this.testID = this.$route.params.testID
    this.loadTest(this.testID)
  }
  this.navItems.push(
    {
      title: 'Organization courses',
      disabled: false,
      href: '/organization-courses'
    },
    {
      title: 'Course edit',
      disabled: false,
      href: '/course-edit/' + this.$route.params.courseID
    }
  )
},
methods: {
  /**
   *
   * @param obj.text - text
   * @param obj.edit -
   */
  edit(obj) {
    const cmd = new EditField({ field: obj })
    cmd.do()
    obj.edit = true
  }
}

```

```

    this.stack.push(cmd)
  },
  removeQuestion(item) {
    const index = this.questions.findIndex(i => i === item)
    this.questions.splice(index, 1)
  },
  addQuestion () {
    let cmd = new AddQuestion({ questions: this.questions })
    cmd.do()
    this.stack.push(cmd)
    cmd = new EditField({ field: this.questions[this.questions.length - 1] })
    cmd.do()
    this.stack.push(cmd)
  },

  async loadTest(testID){
    let result = await fetch(new URL(BACKEND_URL + '/Tests/' + testID), {
      method: 'GET',
      credentials: 'include',
      headers: new Headers({'content-type': 'application/json'})
    })
    if (result.status !== 200) {
      console.error('Something wrong')
      return
    }
    result = await result.json()
    this.name = result.name
    this.description = result.description
    this.questions = result.questions
  },
  required (v) {
    return !!v || 'Field is required'
  },
  async save() {
    if (this.saved) {
      this.saved = false
    }
    let result = await fetch(new URL(BACKEND_URL + '/Tests'), {
      method: this.testID ? 'PUT' : 'POST',
      credentials: 'include',
      headers: new Headers({'content-type': 'application/json'}),
      body: JSON.stringify({
        id: this.testID,
        courseID: this.$route.params.courseID,
        name: this.name,
        description: this.description,

```

```

      questions: this.testID ? this.questions : []
    })
  })
  if (result.status !== 200) {
    console.error("Something wrong")
    return
  }
  if (!this.testID) {
    this.testID = await result.json()
  }
  this.saved = true
  setTimeout(() => {
    this.saved = false
  }, 10000)

},
_create() {
  const url = new URL(BACKEND_URL + '/Tests')

}
}
}

```

```
</script>
```

```
<style scoped>
```

```
</style>
```

Файл SectionEditor.vue

```

<template>
  <v-container fluid="1">
    <div v-if="!isReadOnly">
      <v-btn
        rounded
        flat="1"
        color="#f9f9f9"
        @click="createBaseSection"
      >
        <v-icon small>mdi-plus</v-icon>
        <span>Create section</span>
      </v-btn>
      <v-divider class="mt-2 mb-2"/>
    </div>
  </v-container>

```

```

<v-row
  v-for="(section, index) in sections"
  :key="index"
  align="start"
>
  <v-col>
    <course-section
      :is-read-only="isReadOnly"
      :section="section"
      @add="add"
      @save="updateSection"
      @delete="remove"
    />
  </v-col>
</v-row>
</v-container>
<base-choose-dialog
  v-if="dialogType === DIALOG_TYPES.Lections"
  title="Choose lection"
  :get-url="/api/v1/Lections?courseId=${courseId}"
  @save="saveItem"
  @close="dialogType = DIALOG_TYPES.None"
/>
<base-choose-dialog
  v-if="dialogType === DIALOG_TYPES.Tests"
  title="Choose test"
  :get-url="/api/v1/tests?courseId=${courseId}"
  @save="saveItem"
  @close="dialogType = DIALOG_TYPES.None"
/>
<v-dialog
  v-model="fileDialogVisible"
  width="50%"
>
  <v-card>
    <file-manager @file:click="onFileClick"/>
  </v-card>
</v-dialog>
<v-dialog
  v-model="createSectionDialogVisible"
  :close-on-content-click="false"
  :close-on-back="false"
  :persistent="true"
  width="25%"
>
  <v-card style="border-radius: 15px">

```

```

    <v-card-title>
      Enter section name
    </v-card-title>
    <v-card-item>
      <v-row>
        <v-col>
          <v-text-field
            v-model="sectionName"
            variant="outlined"
          />
        </v-col>
      </v-row>
    </v-card-item>
    <v-card-actions>
      <v-row justify="end" class="mb-0">
        <v-col cols="auto">
          <v-btn flat="1" @click="sectionName=''; createSectionDialogVisible=false">Close</v-btn>
        </v-col>
        <v-col cols="auto">
          <v-btn flat="1" @click="saveSection">Ok</v-btn>
        </v-col>
      </v-row>
    </v-card-actions>
  </v-card>

</v-dialog>
</template>
<script>
import BaseChooseDialog from "@components/user/courses/BaseChooseDialog.vue";
import CourseSection from "@components/user/courses/CourseSection.vue";

import { BAG_TYPES } from './types'
import FileManager from "@components/files/FileManager.vue";

const DIALOG_TYPES = {
  None: null,
  Lectons: 'lections',
  Tests: 'Tests'
}

export default {
  name: "SectionsEditor",
  components: {
    FileManager,
    CourseSection,
    BaseChooseDialog
  }
}

```

```

    },
    props: {
      isReadOnly: {
        type: Boolean,
        default: false
      },
      courseId: {
        type: Number,
        default: 0
      }
    },
    data() {
      return {
        DIALOG_TYPES,
        sections: [],
        dialogType: DIALOG_TYPES.None,
        currentSection: null,
        fileDialogVisible: false,
        isBaseSection: false,
        sectionName: "",
        createSectionDialogVisible: false
      }
    },
    async created() {
      if (!this.courseId) {
        return
      }
      try {
        let result = await fetch(new URL(BACKEND_URL + `/Sections?courseID=${this.courseId}`), {
          method: 'GET',
          credentials: 'include',
          headers: new Headers({'content-type': 'application/json'})
        })
        if (result.status !== 200) return
        const serverSections = await result.json()

        let courseResults = []

        if (this.isReadOnly) {
          result = await fetch(new URL(BACKEND_URL + `/Courses/${this.courseId}/results`), {
            method: 'GET',
            credentials: 'include',
            headers: new Headers({'content-type': 'application/json'})
          })
          courseResults = await result.json()
        }
      }
    }
  }

```



```

    for (const serverSection of serverSections) {
      this.sections.push(this.convertDataToComponentFormat(serverSection, courseResults))
    }
  } catch (e) {
    console.error(e)
  }
},
methods: {
  createBaseSection() {
    this.openDialog(true)
  },

  openDialog(baseSection) {
    this.isBaseSection = baseSection
    this.createSectionDialogVisible = true
  },

  prepareSectionToSave(section) {
    const sectionResult = {
      name: section.text,
      lections: [],
      tests: [],
      files: [],
      sections: []
    }
    for (const bagItem of section.bag) {
      switch (bagItem.type) {
        case BAG_TYPES.Lection:
          sectionResult.lections.push(bagItem)
          break
        case BAG_TYPES.Test:
          sectionResult.tests.push(bagItem)
          break
        case BAG_TYPES.File:
          sectionResult.files.push(bagItem)
          break
        case BAG_TYPES.Section:
          sectionResult.sections.push(this.prepareSectionToSave(bagItem))
          break
        default:
          throw new Error('Invalid bag item type')
      }
    }
    return sectionResult
  },

```

```

async updateSection (section) {
  let result
  try {
    result = await fetch(new URL(BACKEND_URL + '/Sections'), {
      method: 'PUT',
      credentials: 'include',
      headers: new Headers({'content-type': 'application/json'}),
      body: JSON.stringify({
        id: section.id,
        name: section.name,
        courseId: section.courseId || this.courseId,
        order: section.order || 1
      })
    })
  } catch (e) {
    console.error(e)
    return
  }

  if (result.status !== 200) {
    console.error('Something went wrong while deleting section')
  }
},

getSectionsToSave() {
  const res = []
  for (const section of this.sections) {
    res.push(this.prepareSectionToSave(section))
  }
  return res
},

convertDataToComponentFormat(serverSection, courseResults) {

  const courseResultsByItemId = (courseResults || []).reduce((acc, val) => {
    if (!acc[val.itemId]) {
      acc[val.itemId] = val.isOk
    }
    return acc
  }, {})

  const addType = (type, arr)=> {
    arr.forEach(i => {
      i.type = type
      i.isOk = courseResultsByItemId[i.id] || false
    })
  }
}

```

```

    })
    return arr
  }
  const result = serverSection
  serverSection.bag = []
  result.bag.push(...addType(BAG_TYPES.Test, serverSection.tests || []))
  result.bag.push(...addType(BAG_TYPES.Lecture, serverSection.lectures || []))
  result.bag.push(...addType(BAG_TYPES.File, serverSection.files || []))
  // result.bag.push(...addType(BAG_TYPES.Section, (serverSection.subSections ||
[]).map(this.convertDataToComponentFormat)))
  return result
},

```

```

async addBaseSection() {
  if (!this.sectionName) return
  try {
    let result = await fetch(new URL(BACKEND_URL + '/Sections/'), {
      method: 'POST',
      credentials: 'include',
      headers: new Headers({'content-type': 'application/json'}),
      body: JSON.stringify({
        name: this.sectionName,
        courseId: this.$route.params.courseID
      })
    })
    if (result.status !== 200) {
      console.error('Failed to create section')
      return
    }
    result = await result.json()
    this.sections.push({
      id: result,
      name: this.sectionName,
      bag: []
    })
  } catch (e) {
    console.error(e)
  } finally {
    close()
  }
},
async remove(section) {
  if (!section) return
  let result
  try {

```

```

    result = await fetch(new URL(BACKEND_URL + '/Sections'), {
      method: 'DELETE',
      credentials: 'include',
      headers: new Headers({'content-type': 'application/json'}),
      body: section.id
    })
  } catch (e) {
    console.error(e)
    return
  }

  if (result.status !== 200) {
    console.error('Something went wrong while deleting section')
    return
  }

  this.sections = this.sections.filter(s => s !== section)
},
async saveItem(item) {
  switch (this.dialogType) {
    case DIALOG_TYPES.Tests:
      item.type = BAG_TYPES.Test
      break
    case DIALOG_TYPES.Lections:
      item.type = BAG_TYPES.Lection
      break
  }
  let result = await fetch(new URL(BACKEND_URL + '/Sections/Bag'), {
    method: 'POST',
    credentials: 'include',
    headers: new Headers({'content-type': 'application/json'}),
    body: JSON.stringify({
      sectionId: this.currentSection.id,
      itemId: item.id,
      type: item.type,
      path: item.path ?? "",
      name: item.name ?? ""
    })
  })
  item.id = await result.json()
  this.currentSection.bag.push(item)
  this.dialogType = DIALOG_TYPES.None
},
add(section, type) {
  switch (type) {
    case BAG_TYPES.Lection:

```

```

        return this.addLecture(section)
    case BAG_TYPES.Section:
        return this.addSection(section)
    case BAG_TYPES.Test:
        return this.addTest(section)
    case BAG_TYPES.File:
        return this.addFile(section)
    }
},

async saveSection() {
    const close = () => {
        this.sectionName = ""
        this.createSectionDialogVisible = false
    }
    if (this.isBaseSection) {
        await this.addBaseSection()
        close()
        return
    }
    await this.saveItem({ text: this.sectionName, bag: [], type: BAG_TYPES.Section })
    close()
},

addSection(section) {
    this.currentSection = section
    this.openDialog(false)
    // this.saveItem({ text: 'Enter name', bag: [], type: BAG_TYPES.Section })
},

addTest(section) {
    this.currentSection = section
    this.dialogType = DIALOG_TYPES.Tests
},

addFile(section) {
    this.currentSection = section
    this.fileDialogVisible = true
},

addLecture(section) {
    this.currentSection = section
    this.dialogType = DIALOG_TYPES.Lectures
},

onFileClick(file) {
    this.saveItem({ type: BAG_TYPES.File, name: file.name, path: file.path })
    this.fileDialogVisible = false
}
}

```

```
}
</script>
```

```
<style>
```

```
</style>
```

Файл CoursesController.cs

```
using System.Security.Claims;
using OLpNet.Attributes;
using OLpNet.Controllers.Courses.Dtos;
using OLpNet.Controllers.Extensions;
using OLpNet.Database;
using OLpNet.Database.Tables;
using OLpNet.Database.Tables.Courses;
using OLpNet.Services.Files;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Http.HttpResults;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using Mentor = OLpNet.Attributes.Mentor;
namespace OLpNet.Controllers.Courses;

[ApiController]
[Authorize]
[Route("/api/v1/[controller]")]
public class CoursesController(DatabaseContext dbContext, UserManager<User> userManager, IFileStorageProvider
fileStorageProvider) : ControllerBase
{
    [HttpPost]
    [LocalAdmin]
    public async Task<IActionResult> AddCourse(CourseDto courseDto)
    {
        var course = new Course
        {
            Name = courseDto.Name,
            Description = courseDto.Description,
            IsPublic = courseDto.IsPublic
        };
        dbContext.Courses.Add(course);
        await dbContext.SaveChangesAsync();
        return Ok(course.Id);
    }

    [HttpGet]
```

```

public async Task<IActionResult> GetCourses([FromQuery] bool enrolled=true)
{
    var user = await userManager.FindByNameAsync(HttpContext.User.Identity?.Name ?? "");
    var isAdmin = userManager.IsLocalAdmin(User);
    if (user is null) return Unauthorized();
    IQueryable<Course> coursesQ = dbContext.Courses
        .Include(c => c.Mentors)
        .Include(c => c.Enrolled);
    coursesQ = enrolled
        ? coursesQ.Where(c => c.Enrolled.FirstOrDefault(e => e.UserId == user.Id) != null)
        : coursesQ.Where(c => isAdmin || c.IsPublic);
    var courses = await coursesQ.ToListAsync();
    var result = courses.Select(c => new CourseRegisterDto
    {
        Id = c.Id,
        Enrolled = c.Enrolled.Count,
        Mentors = c.Mentors.Count,
        Description = c.Description,
        Name = c.Name,
        Photo = c.Photo,
    });
    return Ok(result);
}

```

[HttpGet("invites")]

```

public async Task<IActionResult> GetCoursesWithInvite()
{
    var user = await userManager.FindByNameAsync(HttpContext.User.Identity.Name);
    if (user is null) return Unauthorized();
    var courses = await dbContext.Courses
        .Include(c => c.Mentors)
        .Include(c => c.Enrolled)
        .Include(c => c.Invited)
        .Where(c => c.Invited.FirstOrDefault(e => e.InvitedUserId == user.Id) != null)
        .ToListAsync();
    var result = courses.Select(c => new
    {
        c.Id,
        Enrolled = c.Enrolled.Count,
        Mentors = c.Mentors.Count,
        c.Description,
        c.Name,
        InviteId = c.Invited.FirstOrDefault(e => e.InvitedUserId == user.Id)?.Id
    });
    return Ok(result);
}

```

```

[HttpGet("{courseId:int}/results")]
public async Task<IActionResult> CourseResults(int courseId)
{
    var user = await userManager.FindByNameAsync(HttpContext.User.Identity.Name);
    if (user is null) return Unauthorized();

    var course = await dbContext.Courses.FirstOrDefaultAsync(c => c.Id == courseId);
    if (course is null) return NotFound();

    var results = await dbContext.CourseResults
        .Where(r => r.UserId == user.Id)
        .ToListAsync();
    return Ok(results.Select(r => new
    {
        r.CourseId,
        r.ItemId,
        r.IsOk
    }));
}

[HttpGet("{courseId:int}/Users")]
[Mentor]
public async Task<IActionResult> GetCourseUsers(int courseId, [FromQuery] string? name)
{
    var course = await dbContext.Courses
        .Include(c => c.Enrolled)
        .ThenInclude(e => e.User)
        .FirstOrDefaultAsync(c => c.Id == courseId);

    if (course is null)
        return NotFound("Course not exists");

    return Ok(course.Enrolled
        .Where(e => name is null ? true : e.User.UserName.Contains(name))
        .Select(e => new
        {
            Id=e.UserId,
            Name=e.User.UserName,
            e.User.Email
        }));
}

[HttpGet("{courseId:int}")]

```



```

public async Task<IActionResult> GetCoursesById(int courseId)
{
    var user = await userManager.FindByNameAsync(HttpContext.User.Identity.Name);
    if (user is null) return Unauthorized();
    var course = await dbContext.Courses
        .Include(c => c.Enrolled)
        .Include(c => c.Tests)
        .Include(c => c.Lections)
        .Where(c => c.Id == courseId)
        .FirstOrDefaultAsync();
    if (course is null) return NotFound();
    return Ok(new
    {
        IsEnrolled=course.Enrolled.FirstOrDefault(e => e.UserId == user.Id) is not null,
        course.IsPublic,
        course.Description,
        course.Name,
        course.Id,
        Tests = course.Tests.Select(t => new TestDto
        {
            Name = t.Name,
            Id = t.Id
        }),
        Photo = course.Photo,
        Lections=course.Lections.Select(l => new { l.Id, l.Description, l.Name })
    });
}

[HttpPut]
[LocalAdmin]
public async Task<IActionResult> UpdateCourse(CourseDto courseDto)
{
    if (courseDto.Id is null) return NotFound();
    var course = await dbContext.Courses.FirstOrDefaultAsync(c => c.Id == courseDto.Id);
    if (course is null) return NotFound();
    course.Description = courseDto.Description;
    course.Name = courseDto.Name;
    course.IsPublic = courseDto.IsPublic;

    if (courseDto.Photo is not null)
    {
        try
        {
            var f = fileStorageProvider.GetFileContent(new LearnFile
            {
                Path = courseDto.Photo
            })
        }
    }
}

```

```

        });
    }
    catch (Exception e)
    {
        Console.WriteLine(e);
        return NotFound("File not found");
    }
}

course.Photo = courseDto.Photo ?? null;

dbContext.Courses.Update(course);
await dbContext.SaveChangesAsync();
return Ok();
}

[HttpDelete]
[LocalAdmin]
public async Task<IActionResult> RemoveCourse(int courseId)
{
    var isAdmin = userManager.IsLocalAdmin(User);
    if (!isAdmin) return Forbid();

    var course = await dbContext.Courses.FirstOrDefaultAsync(c => c.Id == courseId);
    if (course is null) return NotFound();
    dbContext.Courses.Remove(course);
    await dbContext.SaveChangesAsync();
    return Ok();
}

[HttpGet("archive")]
public async Task<IActionResult> GetArchivedCourses()
{
    var user = await userManager.FindByNameAsync(HttpContext.User.Identity.Name);
    if (user is null) return Unauthorized();

    var enrolledCourses = await dbContext.Enrolls
        .Include(e => e.Course)
        .Where(e => e.IsArchived)
        .Where(e => e.UserId == user.Id)
        .ToListAsync();

    return Ok(enrolledCourses.Select(e => new CourseDto
    {
        Description = e.Course.Description,
        Id = e.Course.Id,

```

```

        Name = e.Course.Name,
        IsPublic = e.Course.IsPublic
    ));
}

[HttpGet("mentor")]
public async Task<IActionResult> GetCoursesWhereUserIsMentor()
{
    var user = await userManager.FindByNameAsync(HttpContext.User.Identity.Name);
    if (user is null) return Unauthorized();

    var courses = await dbContext.Courses
        .Include(c => c.Mentors)
        .Where(c => c.Mentors.Any(m => m.UserId == user.Id))
        .ToListAsync();
    var result = courses.Select(c => new CourseDto
    {
        Id = c.Id,
        Name = c.Name,
        Description = c.Description,
        IsPublic = c.IsPublic
    });
    return Ok(result);
}
}

```

Файл TestsController.cs

```

using OLPNet.Attributes;
using OLPNet.Controllers.Courses.Dtos;
using OLPNet.Database;
using OLPNet.Database.Tables.Courses;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace OLPNet.Controllers.Courses;

[ApiController]
[Authorize]
[Route("/api/v1/[controller]")]
public class TestsController : ControllerBase
{
    private readonly DatabaseContext _dbContext;

    public TestsController(DatabaseContext dbContext)
    {

```

```

        _dbContext = dbContext;

    }

    [HttpPost]
    [LocalAdmin]
    public async Task<IActionResult> AddTest([FromBody] TestDto dto)
    {
        dto.Questions ??= [];
        var test = new Test
        {
            Name = dto.Name,
            Description = dto.Description,
            CourseId = dto.CourseId,
            Questions = dto.Questions.Select(q => new Question
            {
                Text = q.Text,
                Answers = (q.Answers ?? []).Select(a => new Answer
                {
                    IsValid = a.IsValid ?? false,
                    Text = a.Text
                }).ToList()
            }).ToList()
        };
        _dbContext.Tests.Add(test);
        await _dbContext.SaveChangesAsync();
        return Ok(test.Id);
    }

    [HttpGet]
    public async Task<IActionResult> GetTests()
    {
        var test = await _dbContext.Tests
            .ToListAsync();
        return Ok(test);
    }

    [HttpGet("{testId:int}")]
    public async Task<IActionResult> GetTests(int testId)
    {
        var test = await _dbContext.Tests
            .Include(test => test.Questions)
            .ThenInclude(question => question.Answers)
            .FirstOrDefaultAsync(q => q.Id == testId);
        if (test is null) return NotFound();
        return Ok(new TestDto
        {

```

```

        Id = test.Id,
        Name = test.Name,
        Description = test.Description,
        CourseId = test.CourseId,
        Questions = test.Questions.Select(q => new QuestionDto
        {
            Id = q.Id,
            Text = q.Text,
            Answers = q.Answers.Select(a => new AnswerDto
            {
                Id = a.Id,
                Text = a.Text,
                IsValid = a.IsValid
            }).ToList()
        }).ToList()

    });
}
[HttpPut]
[LocalAdmin]
public async Task<IActionResult> UpdateTest([FromBody] TestDto dto)
{
    if (dto.Id is null) return NotFound();
    dto.Questions ??= [];
    var test = await _dbContext.Tests
        .Include(t => t.Questions)
        .FirstOrDefaultAsync(t => t.Id == dto.Id);
    if (test is null) return NotFound();
    test.Name = dto.Name;
    test.Description = dto.Description;
    test.Questions = dto.Questions.Select(q => new Question
    {
        TestId = dto.Id ?? 0,
        Text = q.Text,
        Answers = (q.Answers ?? []).Select(a => new Answer
        {
            IsValid = a.IsValid ?? false,
            Text = a.Text
        }).ToList()
    }).ToList();
    _dbContext.Tests.Update(test);
    await _dbContext.SaveChangesAsync();
    return Ok(test.Id);
}
}

```

Файл SectionsController.cs

```

using OLPSNet.Attributes;
using OLPSNet.Controllers.Courses.Dtos;
using OLPSNet.Database;
using OLPSNet.Database.Interfaces;
using OLPSNet.Database.Tables;
using OLPSNet.Database.Tables.Courses;
using OLPSNet.Services.Files;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace OLPSNet.Controllers.Courses;

[ApiController]
[Authorize]
[Route("/api/v1/[controller]")]
public class SectionsController(DatabaseContext context, IFileStorageProvider fileStorageProvider, UserManager<User>
userManager) : ControllerBase
{

    [HttpGet]
    public async Task<IActionResult> GetSections([FromQuery] int courseId)
    {
        var sections = await context.Sections
            .Include(s => s.Tests)
            .Include(s => s.Lections)
            .Include(s => s.Files)
            .Where(s => s.CourseId == courseId)
            .Where(s => s.ParentSectionId == null)
            .ToListAsync();
        return Ok(sections.Select(s => new
        {
            s.Id,
            s.Name,
            s.Order,
            s.CourseId,
            Tests = s.Tests.Select(t => new
            {
                t.Id,
                t.Name,
                t.Description
            }).ToList(),
            Lections = s.Lections.Select(l => new

```

```

    {
        l.Id,
        l.Name,
        l.Description
    }).ToList(),
    Files = s.Files.Select(f => new
    {
        f.Id,
        f.Name,
        f.Path
    })
    }));
}

```

```

[HttpPost]
[LocalAdmin]
public async Task<IActionResult> AddSection([FromBody] SectionDto dto)
{
    var section = new Section
    {
        Name = dto.Name,
        Order = dto.Order,
        CourseId = dto.CourseId,
    };
    context.Sections.Add(section);
    await context.SaveChangesAsync();
    return Ok(section.Id);
}

```

```

[HttpPut]
[LocalAdmin]
public async Task<IActionResult> UpdateSection([FromBody] SectionDto dto)
{
    var section = await context.Sections.FirstOrDefaultAsync(s => s.Id == dto.Id);
    if (section is null)
    {
        return NotFound("Section not found");
    }

    section.Name = dto.Name;
    section.Order = dto.Order;

    context.Sections.Update(section);
    await context.SaveChangesAsync();
    return Ok();
}

```

```

[HttpPost("Bag")]
[LocalAdmin]
public async Task<ActionResult> AddItemToBag([FromBody]SectionBagItemDto dto)
{

    var sectionQ = context.Sections.Where(s => s.Id == dto.SectionId);
    switch (dto.Type)
    {
        case "lection":
            sectionQ = sectionQ.Include(s => s.Lections);
            break;
        case "test":
            sectionQ = sectionQ.Include(s => s.Tests);
            break;
        case "section":
            sectionQ = sectionQ.Include(s => s.SubSection);
            break;
        case "file":
            sectionQ = sectionQ.Include(s => s.SubSection);
            break;
    }

    var section = await sectionQ.FirstOrDefaultAsync();

    if (section is null)
        return NotFound("Section not found");
    var id = -1;
    switch (dto.Type)
    {
        case "lection":
            id = await AddToBag<Lecture>(section, dto);
            break;
        case "test":
            id = await AddToBag<Test>(section, dto);
            break;
        case "file":
            var file = new SFile
            {
                Name = dto.Name ?? "",
                Path = dto.Path ?? "",
                SectionId = section.Id
            };
            await context.SFiles.AddAsync(file);
            await context.SaveChangesAsync();
            id = file.Id;

```



```

        break;
    }

    if (id == -1) return NotFound($"Lecture with ID {dto.ItemId} not found");
    return Ok(id);
}

private async Task<int> AddToBag<T>(Section section, SectionBagItemDto dto) where T : class, IEntity
{
    var lection = await context.Set<T>().FirstOrDefaultAsync(l => l.Id == dto.ItemId);
    if (lection is null) return -1;
    var info = section
        .GetType()
        .GetProperties()
        .FirstOrDefault(p => p.PropertyType == typeof(List<T>));
    var l = (List<T>) section.GetType().GetProperty(info.Name).GetValue(section);
    l?.Add(lection);
    // context.Set<T>().Add(lection);
    await context.SaveChangesAsync();
    return lection.Id;
}

[HttpDelete]
[LocalAdmin]
public async Task<IActionResult> DeleteSection([FromBody] int sectionId)
{
    var section = await context.Sections.FirstOrDefaultAsync(s => s.Id == sectionId);
    if (section is null)
        return NotFound("Section not found");

    context.Sections.Remove(section);
    await context.SaveChangesAsync();

    return Ok();
}

[HttpDelete("Bag")]
[LocalAdmin]
public async Task<IActionResult> DeleteBagItem(SectionBagItemDto bag)
{
    var sectionQ = context.Sections.Where(s => s.Id == bag.SectionId);
    switch (bag.Type)
    {
        {
            case "lection":
                sectionQ = sectionQ.Include(s => s.Lectures);
                break;

```

```

        case "test":
            sectionQ = sectionQ.Include(s => s.Tests);
            break;
        case "section":
            sectionQ = sectionQ.Include(s => s.SubSection);
            break;
        case "file":
            sectionQ = sectionQ.Include(s => s.Files);
            break;
    }
    var section = await sectionQ.FirstOrDefaultAsync();

    if (section is null)
        return NotFound("Section not found");

    switch (bag.Type)
    {
        case "lection":
            section.Lections.Remove(section.Lections.First(l => l.Id == bag.ItemId));
            break;
        case "test":
            section.Tests.Remove(section.Tests.First(l => l.Id == bag.ItemId));
            break;
        case "file":
            section.Files.Remove(section.Files.First(l => l.Id == bag.ItemId));
            break;
    }

    await context.SaveChangesAsync();
    return Ok();
}

[HttpGet("{sectionId:int}/Bag/Files/{fileId:int}")]
public async Task<IActionResult> GetFileContent(int sectionId, int fileId)
{
    var user = await userManager.FindByNameAsync(HttpContext.User.Identity.Name);
    if (user is null) return Unauthorized();
    var file = await context.SFiles
        .Include(f => f.Section)
        .FirstOrDefaultAsync(f => f.Id == fileId && f.SectionId == sectionId);
    if (file is null) return NotFound("File not found");
    var fileBytes = fileStorageProvider.GetFileContent(new LearnFile
    {
        Path = file.Path
    });
}

```

```
await context.CourseResults.AddAsync(new CourseResults
{
    ItemId = file.Id,
    CourseId = file.Section.CourseId,
    IsOk = true,
    UserId = user.Id
});

await context.SaveChangesAsync();
return File(fileBytes, "application/octet-stream", file.Name);
}
}
```

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2024 р.

**РОЗПОДІЛЕНА СИСТЕМА УПРАВЛІННЯ, АГРЕГАЦІЇ ТА
ПОШИРЕННЯ ДАНИХ**

Програма та методика тестування

КПІ.ПІ-8201.045440.04.51

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Микола ХРАМЧЕНКО

Нормоконтроль:

_____ Тетяна ШУЛЬКЕВИЧ

Виконавець:

_____ Олександр БАЛАЦЕНКО

Київ – 2024

ЗМІСТ

1 ОБ'ЄКТ ВИПРОБУВАНЬ	3
2 МЕТА ТЕСТУВАННЯ.....	4
3 МЕТОДИ ТЕСТУВАННЯ.....	4
4 ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ.....	6

1 ОБ'ЄКТ ВИПРОБУВАНЬ

Об'єктом випробування є розподілена система агрегації, управління та поширення навчальних матеріалів. Тестування можна поділити на чотири етапи: тестування серверу, тестування клієнту, тестування поштових нотифікацій, тестування файлової системи.

Тестування серверу здійснюється мануальним способом, для цього серверу запускається у режимі розробки, в цьому режимі доступний Swagger, що дає можливість легко перевірити необхідні посилання

Оскільки відображенням інтерфейсу застосунку займається браузер, необхідно переконатися у правильності його роботи на різних версіях браузера.

Для тестування поштових нотифікацій був розроблений спеціальний скрипт, який запускається за допомогою засобів командного рядку.

2 МЕТА ТЕСТУВАННЯ

Метою тестування є наступне:

- перевірка правильності роботи програмного забезпечення відповідно до функціональних вимог;
- перевірка збереження даних;
- перевірка сумісності веб-додатку з останніми версіями сучасних браузерів
- знаходження проблем, помилок і недоліків з метою їх усунення;
- перевірка зручності графічного інтерфейсу.

Метриками для оцінки якості ПЗ обрано наступні:

- швидкість;
- надійність;

3 МЕТОДИ ТЕСТУВАННЯ

Для тестування програмного забезпечення використовуються такі методи:

- статичне тестування – перевіряється програма разом з усією документацією, яка аналізується на предмет дотримання стандартів програмування;

- системне тестування – перевіряється усе програмне забезпечення в цілому;

- мануальне тестування – тестування без використання автоматизації, тест-кейси пише особа, що тестує програмне забезпечення;

4 ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ

Тестування виконується мануально, з метою знаходження помилок та недоліків як у функціональній частині програмного забезпечення так і в зручності користування. Для того, щоб перевірити працездатність та відмовостійкість застосунку, необхідно провести наступні тестування:

- тестування на мобільних пристроях з різною версією операційної системи;
- тестування на виведення повідомлень про помилку, коли це необхідно;
- тестування інтерфейсу користувача;
- тестування зручності використання;
- тестування роботи на різних браузерях;
- тестування роботи на екранах з різною розподільною здатністю.

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ” _____ 2024 р.

**РОЗПОДІЛЕНА СИСТЕМА УПРАВЛІННЯ, АГРЕГАЦІЇ ТА
ПОШИРЕННЯ ДАНИХ
Керівництво користувача
КПІ.ПІ-8201.045440.05.34**

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Микола ХРАМЧЕНКО

Нормоконтроль:

_____ Тетяна ШУЛЬКЕВИЧ

Виконавець:

_____ Олександр БАЛАЦЕНКО

Київ – 2024

ЗМІСТ

1 ПРИЗНАЧЕННЯ ПРОГРАМИ.....	3
2 ПІДГОТОВКА ДО РОБОТИ З ПРОГРАМНИМ ЗАБЕЗПЕЧЕННЯМ	4
2.1 Системні вимоги для коректної роботи.....	4
2.2 Завантаження застосунку	4
2.3 Перевірка коректної роботи.....	4
3 ВИКОНАННЯ ПРОГРАМИ	5

1 ПРИЗНАЧЕННЯ ПРОГРАМИ

Розподілена система агрегації, управління та поширення матеріалів – це програма, яка дозволяє компаніям розміщувати файли, лекції та інші матеріали для навчання співробітників.

Метою програми є можливість розвантажити головних співробітників, які зазвичай займаються навчанням та знайомством з об'єктом навчання, а також зробити єдиний хаб де будуть розміщені навчальні матеріали з можливістю їх агрегації у окремі курси.

Для запуску програми користувачу достатньо мати встановлений браузер на операційній системі, якої він користується.

2 ПІДГОТОВКА ДО РОБОТИ З ПРОГРАМНИМ ЗАБЕЗПЕЧЕННЯМ

2.1 Системні вимоги для коректної роботи

Мінімальна конфігурація технічних засобів:

- тип процесору: Intel Core i3;
- об'єм ОЗП: 4 Гб;
- підключення до мережі Інтернет зі швидкістю від 20 мегабіт;
- Пристрої введення

Рекомендована конфігурація технічних засобів:

- тип процесору: Intel Core i7;
- об'єм ОЗП: 16 Гб;
- підключення до мережі Інтернет зі швидкістю від 1000 мегабіт;

2.2 Завантаження застосунку

Для роботи застосунку необхідне стабільне інтернет з'єднання. Відкрити браузер, який встановлений на комп'ютері користувача та ввести доменне ім'я.

2.3 Перевірка коректної роботи

Після вводу доменного ім'я має відкритися сторінка вводу даних для входу в систему. Після вводу даних користувачем та натисканню на кнопку входу повинна відкритися сторінка з переліком доступних курсів для користувача.

3 ВИКОНАННЯ ПРОГРАМИ

Для початку роботи з програмою необхідно ввести доменне ім'я та почекати на відкриття сторінки авторизації, яка показана на рисунку 3.1.

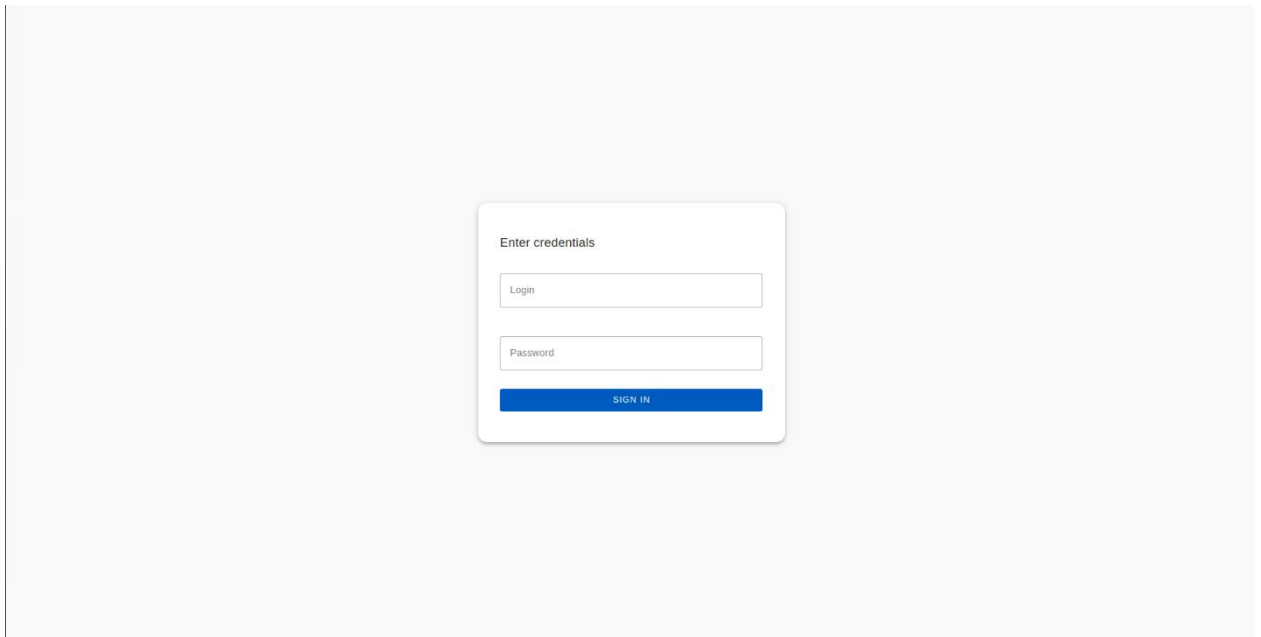


Рисунок 3.1 – Сторінка входу в систему

На цій сторінці користувач вводить дані, які були надіслані на його електронну поштову адресу. Після успішного введення даних користувача буде перенаправлено на сторінку з доступними курсами (Рисунок 3.2).

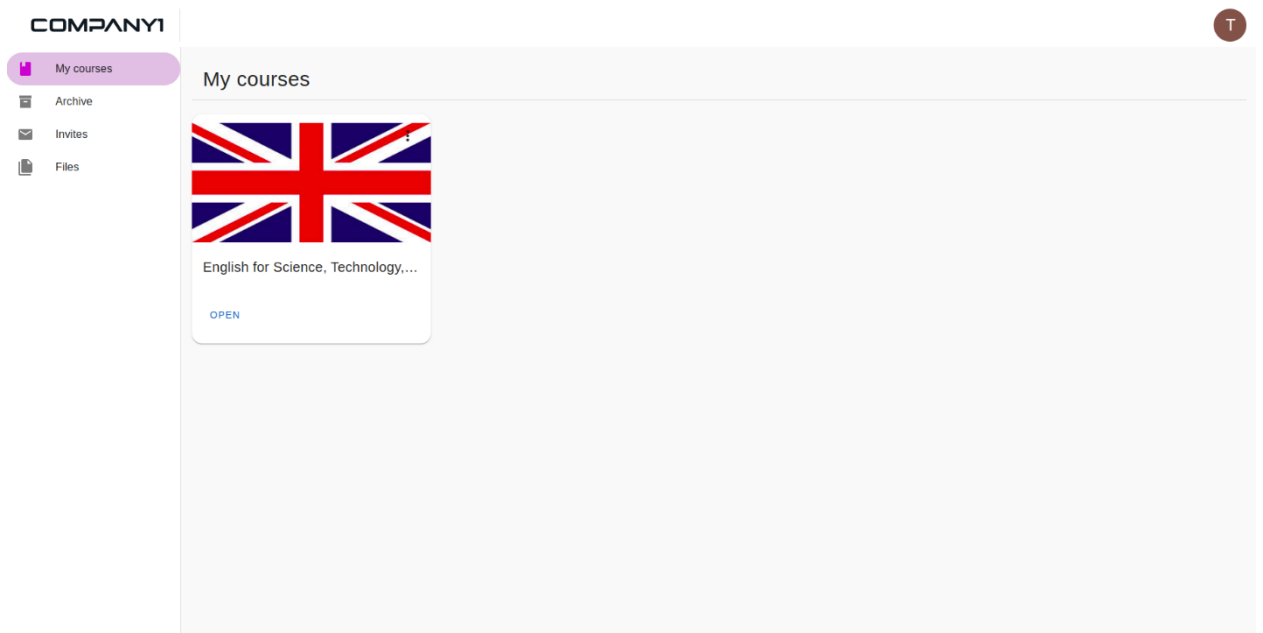


Рисунок 3.2 – Сторінка з курсами

Далі користувач може перейти до одного з курсів, або перейти в один з доступних пунктів меню. При натисканні на кнопку відкриття курсу, яка знаходиться на його картці, користувача буде перенаправлено на сторінку з описом курсу (Рисунок 3.3).

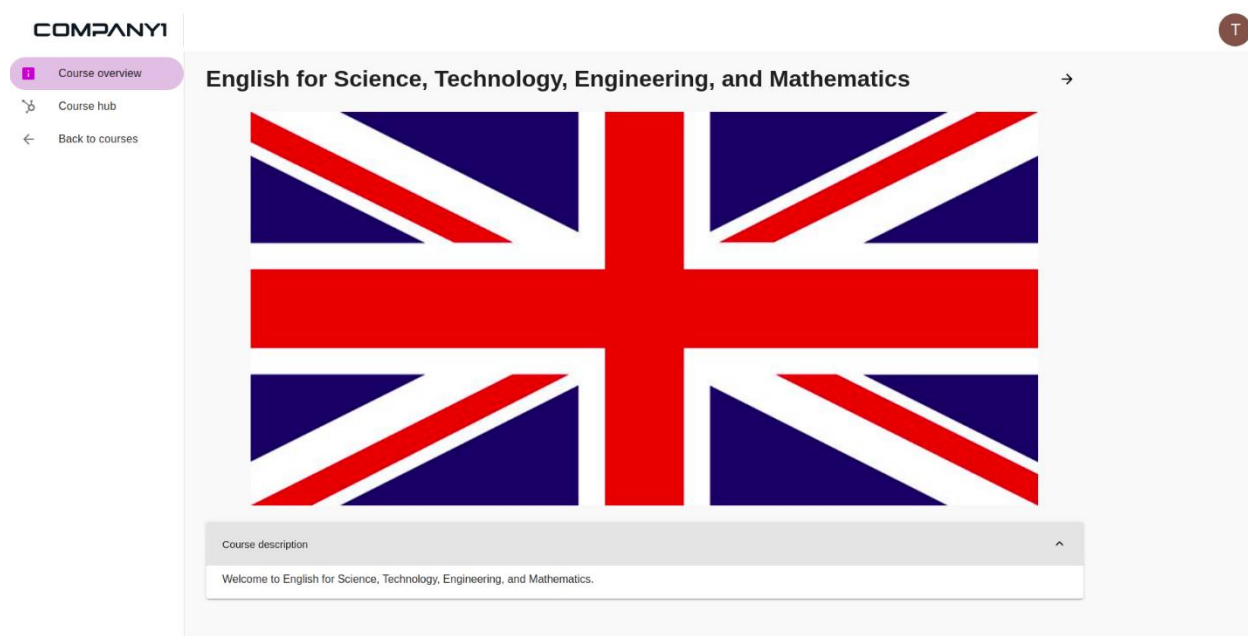


Рисунок 3.3 – Сторінка з описом курсу

З цієї сторінки користувач може перейти до змісту курсу, для цього йому потрібно натиснути на стрілку, яка знаходиться справа від назви курсу, або за допомогою кнопки в меню. Приклад змісту курсу наведений на рисунку 3.4.

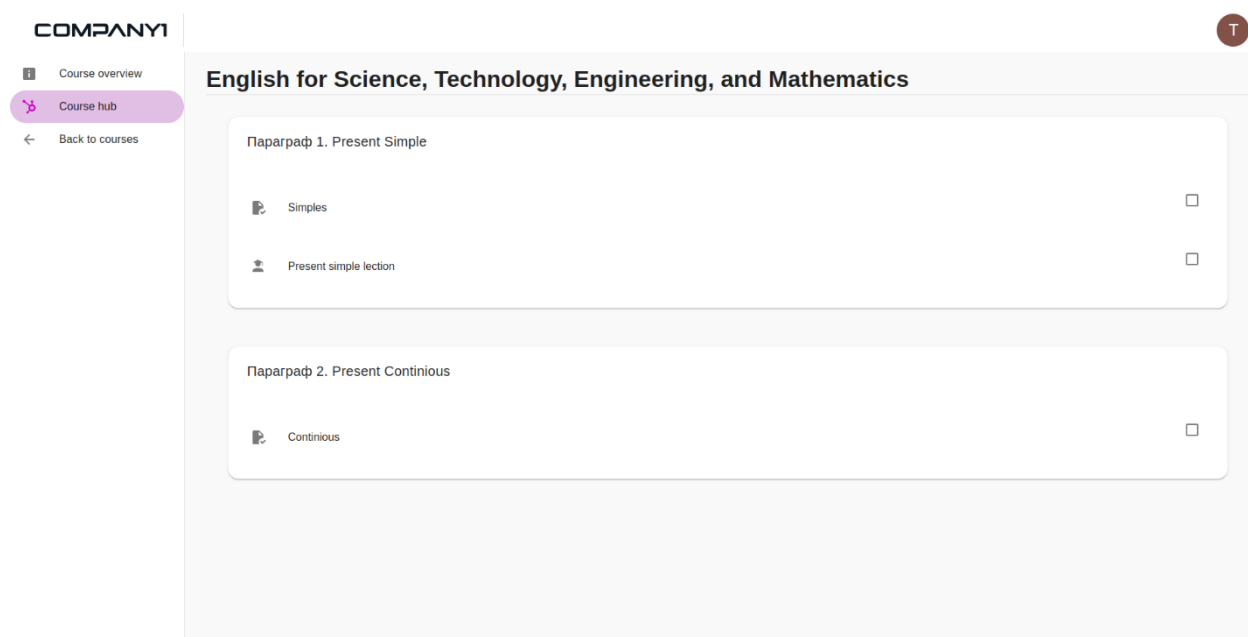


Рисунок 3.4 – Зміст курсу

На цій сторінці користувач може побачити зміст курсу, який поділений на розділи. Кожний розділ може вміщати в собі необмежену кількість пунктів. Пункти діляться на 3 типи: файли, лекції та тести. Якщо користувач натисне на файл, то він буде завантажений на пристрій користувача, якщо натисне на лекції то його буде перенаправлено на сторінку зі слайдами, в іншому випадку при натисканні на тест, його буде перенаправлено на сторінку з тестами(Рисунок 3.5).

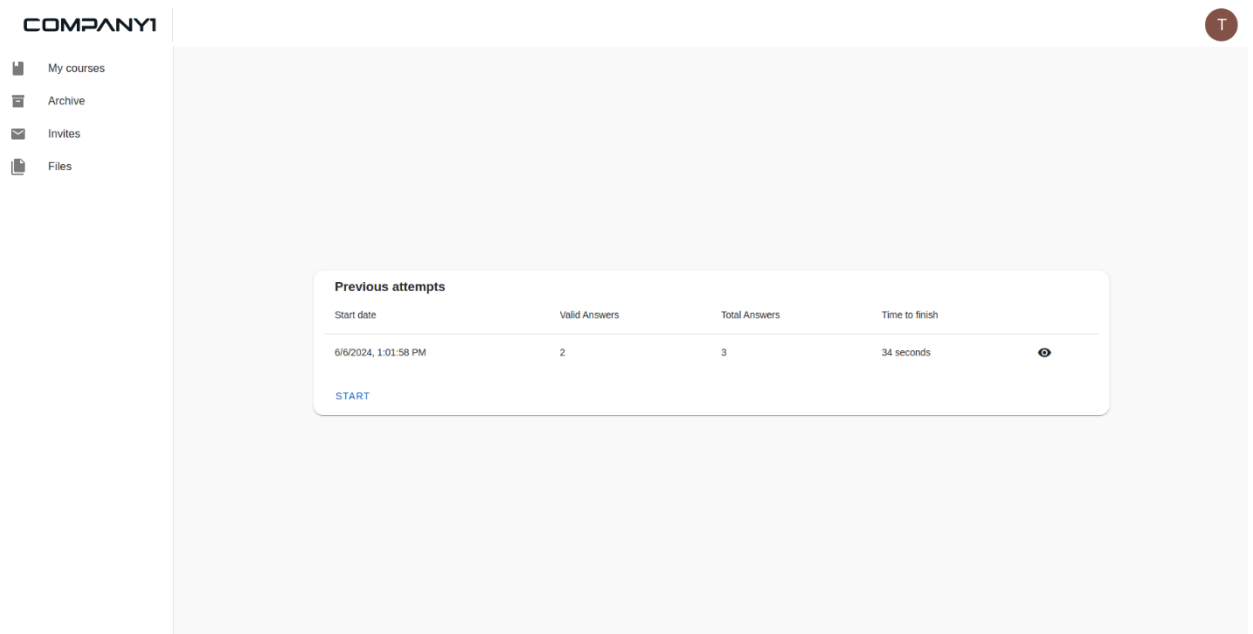


Рисунок 3.5 – Сторінка початку тесту

На цій сторінці користувач може переглянути минулі результати проходження тесту, або розпочати нову спробу, натиснувши на кнопку початку тестування. Після натискання на кнопку початку тесту, розпочнеться тестування (Рисунок 3.6).

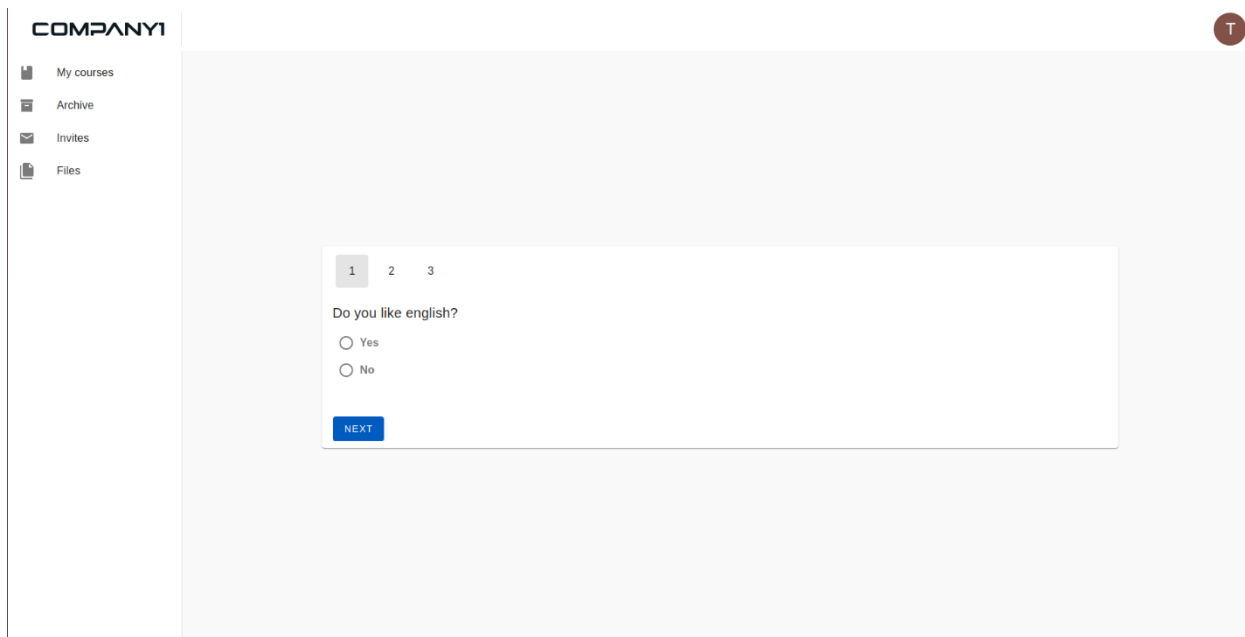


Рисунок 3.6 – Тест

Користувач бачить квадрати з номерами питань, саме питання та варіанти відповіді. Приклад проходження тесту показано на рисунку 3.7.

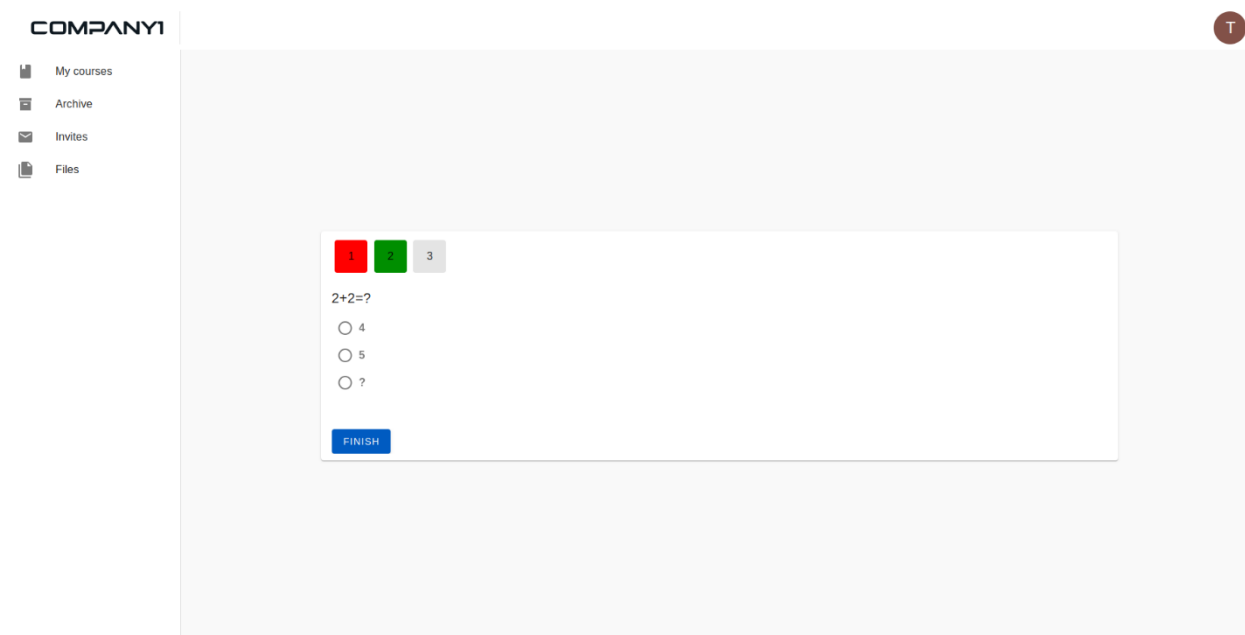


Рисунок 3.7 – Проходження тесту

При проходженні тесту, якщо користувач відповідає на питання правильно, то відповідний квадрат з номером питання змінює колір, у випадку, коли відповідь правильно, то він змінює колір на зелений, у інакшому випадку на червоний. Після проходження всіх питань, користувач натискає на кнопку завершення, та отримує інформацію про витрачений час на тестування(Рисунок 3.8).

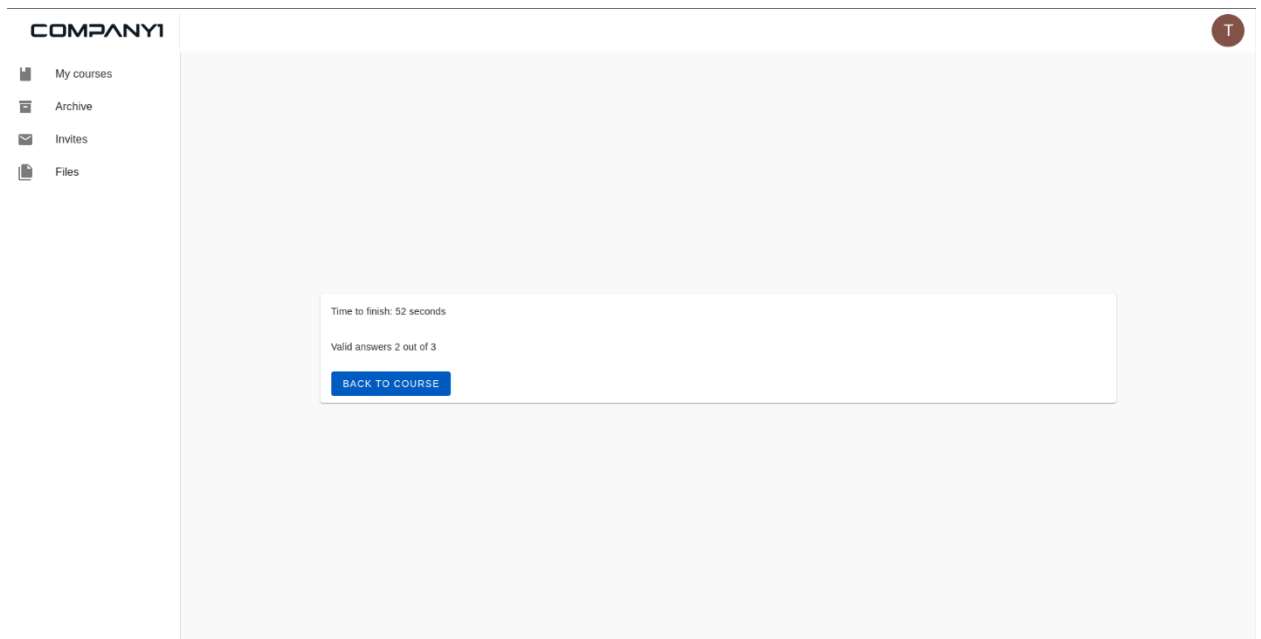


Рисунок 3.8 – Підсумки тестування

Також користувач може завантажувати файли у систему та вивантажувати їх на свій девайс. Для цього йому необхідно перейти у пункт меню «Файли», де він побачить менеджер файлів (Рисунок 3.9).

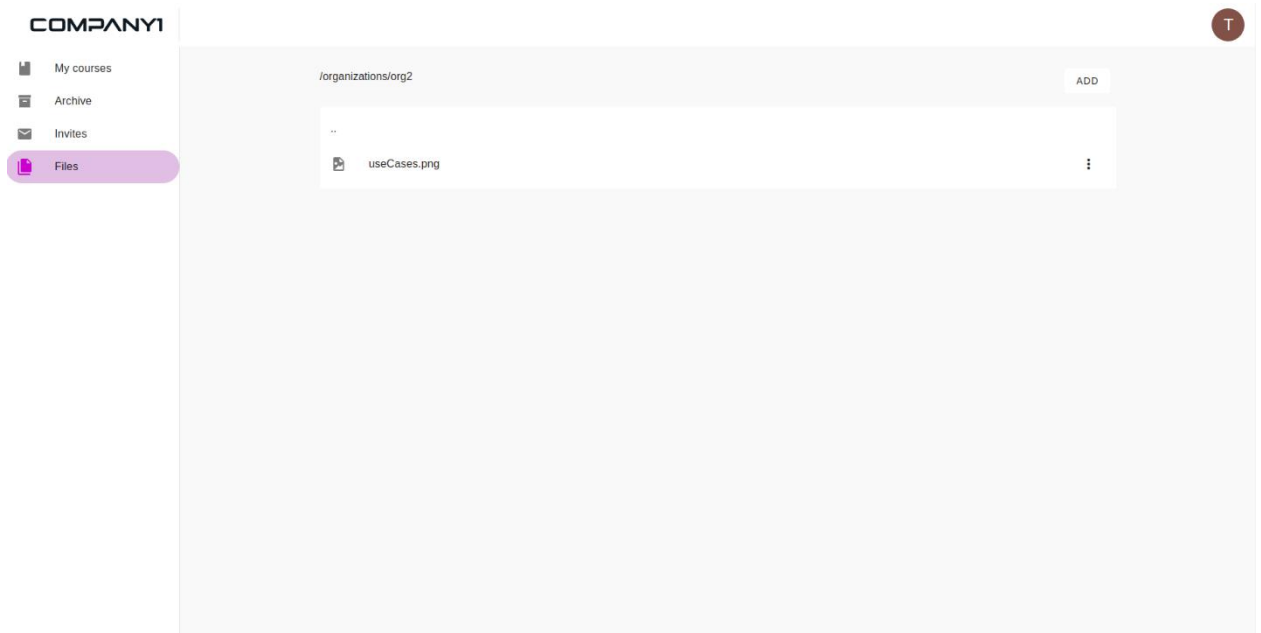


Рисунок 3.9 – Менеджер файлів

Для того щоб отримати доступ до курсу, користувач має перейти до пункту меню «Запрошення». Тут він побачить всі курси до яких його було запрошено (Рисунок 3.10)

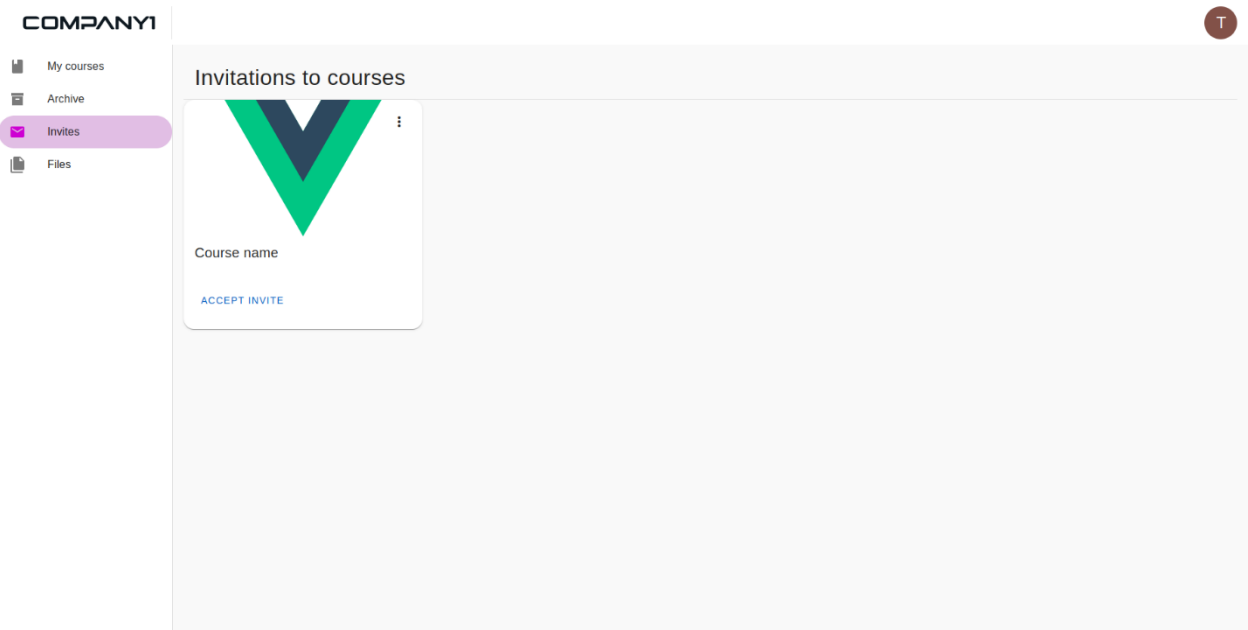


Рисунок 3.10 – Сторінка запрошень до курсів

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2024 р.

**РОЗПОДІЛЕНА СИСТЕМА УПРАВЛІННЯ, АГРЕГАЦІЇ ТА
ПОШИРЕННЯ ДАНИХ**

Графічний матеріал

КПІ.ПІ-8201.045440.06.99

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Микола ХРАМЧЕНКО

Нормоконтроль:

_____ Тетяна ШУЛЬКЕВИЧ

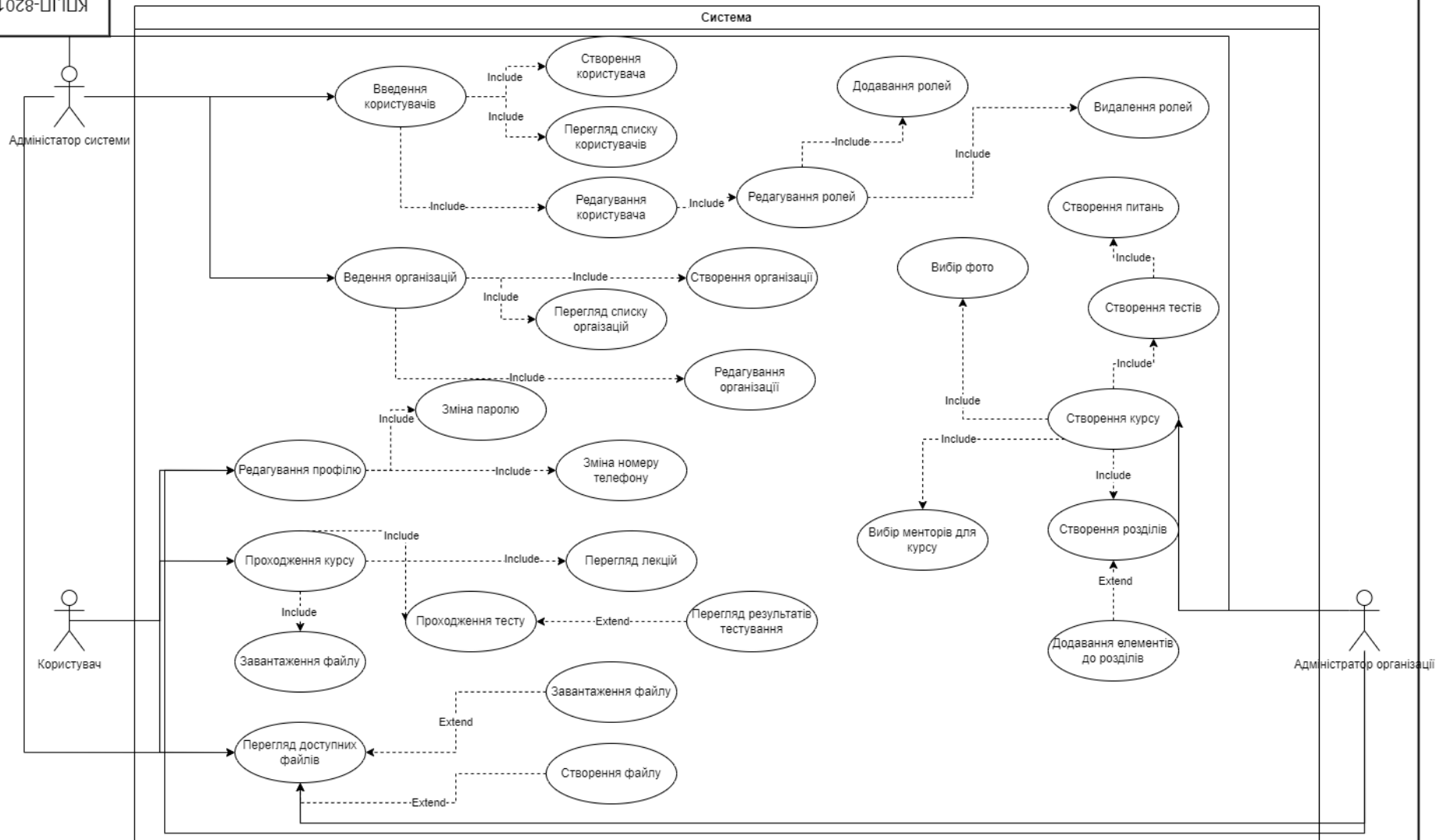
Виконавець:

_____ Олександр БАЛАЦЕНКО

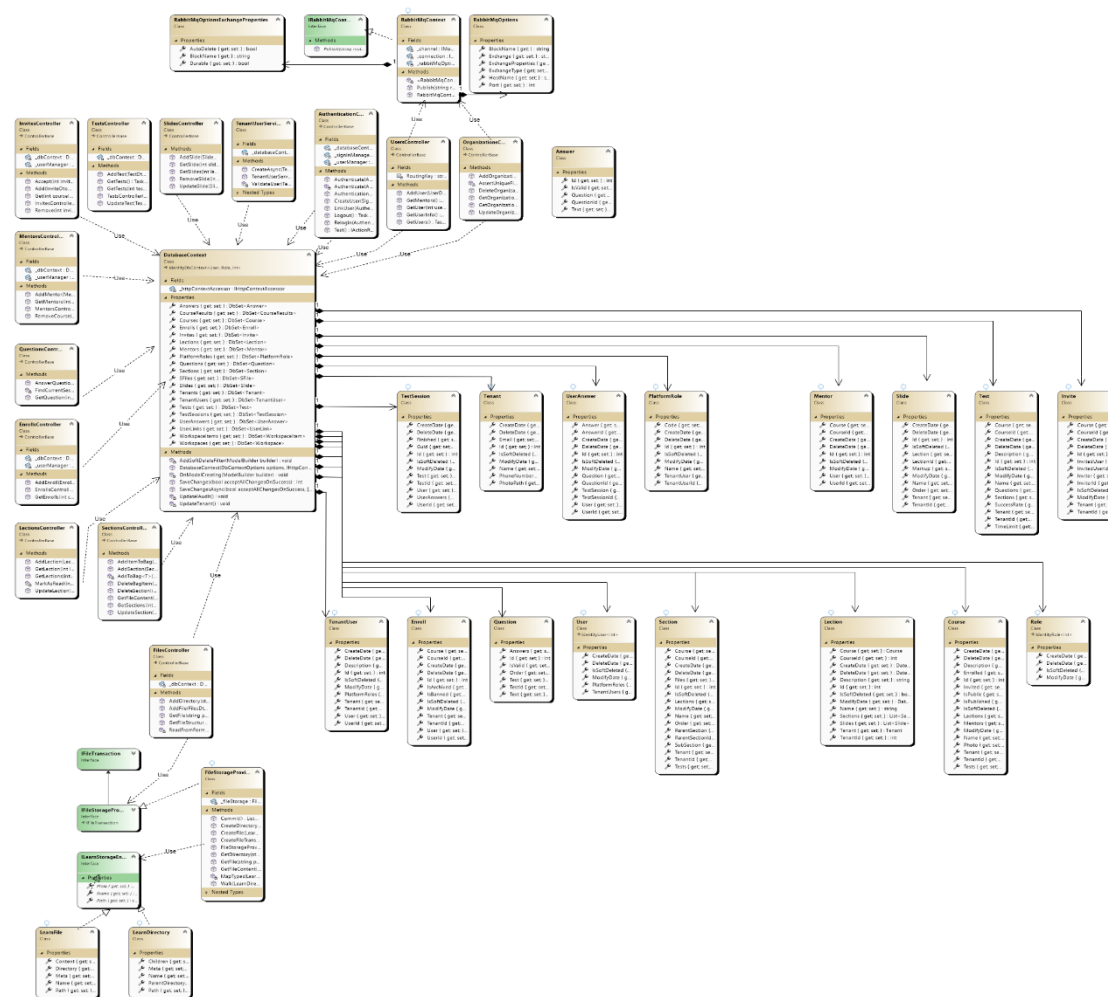
Київ – 2024



КП.ІП-8201.045440.06.99.СБД					Лит.			Маса	Масштаб
Зм.	Арк.	№ докум.	Підп.	Дата	Схема бази даних				
Розробив	Балащенко О.В								
Перевішив	Храмченко М.С								
Т. контр.								Аркуш 1	Аркушів 3
Н. контр.	Шулькевич Т.С				Розподілена система управління, агрегація та поширення даних			КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-02	
Затвердив	Жаріков Е.В.								



					КПІ.ІП-8201.045440.06.99.CCB				
					Схема структурна варіантів використання	Літера		Маса	Масштаб
Зм.	Арк.	№ документа	Підпис	Дата					
Розробив		Балащенко О.В.							
Перевірів		Храмченко М.С.							
Т. контр.						Аркуш 2		Аркуш 3	
Н. контр.		Шулькевич Т.С.			Розподілена система управління, агрегації та поширення даних	КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-02			
Затвердив		Жаріков Е.В.							



				КПІ.ІП-8201.045440.06.99.ССК			
Арк.	№ документа	Підпис	Дата	Схема структурна варіантів використання	Літера	Маса	Масштаб
Розробив	Балащенко О.В.						
Перевірив	Храмченко М.С.						
Т. контр.					Аркуш 3	Аркуші 3	
Н. контр.	Шулькевич Т.С.			Розподілена система управління, агрегація та поширення даних	КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-02		
Затвердив	Жаріков Е.В.						